

On Compressing Non-Additive Correlations

Geoffroy Couteau¹, Alexander Koch², Nikolas Melissaris¹, Peter Scholl³, Sacha Servan-Schreiber⁴,
and Xiayi Ye⁵

¹ CNRS, IRIF, Université Paris Cité, France

² Secorvo Security Consulting GmbH

³ Aarhus University

⁴ Tinfoil

⁵ Tsinghua University

Abstract. Compressing correlated randomness is a key component of secure computation protocols in the silent preprocessing model and has received significant attention in recent years. However, to date, all known constructions are restricted to generating additive correlations, where the parties receive additive shares of a relation. The only known exceptions are constructions from indistinguishability obfuscation.

In this work, we initiate the study of compressing useful forms of correlated randomness for non-additive correlations, without resorting to $i\mathcal{O}$. We provide several constructions for pseudorandom correlation generators and functions, overcoming the long-standing “ $i\mathcal{O}$ -barrier” in the field. Concretely, we focus on two types of non-additive correlations in this work.

First, we introduce pseudorandom *permutation* functions, where each party obtains a portion of a pseudorandom permutation π over the set $\{1, \dots, n\}$, such that no subset of $n-2$ colluding parties learn the full permutation π . We give a three-party construction of a pseudorandom correlation function (PCF) for permutations from homomorphic secret sharing satisfying a special share programmability property. This construction can be instantiated from a variety of standard assumptions and can even be concretely efficient (generating pseudorandom permutations in a fraction of a second). We describe two applications of pseudorandom permutation functions: one in anonymous broadcast and the other in single secret leader election.

Second, we introduce pseudorandom correlation generators for classical correlations (such as Beaver triples) where additive secret sharing is replaced with *threshold* secret sharing. We obtain a pseudorandom correlation function for constant-degree Shamir shares of low-degree correlations from a variety of standard assumptions. We also introduce two new t -out-of- n PCG constructions where either t or $n-t$ is a constant and t denotes the threshold. These constructions rely on the conjunction of the LPN and MQ assumptions.

Keywords. pseudorandom correlation functions, pseudorandom correlation generators, multi-party permutations, threshold secret sharing.

¹ {couteau,nikolas}@irif.fr

² alexander.koch@secorvo.de, work done while the author was at IRIF.

³ peter.scholl@cs.au.dk

⁴ 3s@mit.edu, work done while the author was at MIT.

⁵ yexx23@mails.tsinghua.edu.cn

1 Introduction

Modern secure multi-party computation protocols are generally split into two phases: an input-independent offline phase and an input-dependent online phase. In the offline phase, parties generate preprocessing material, that takes the form of correlated randomness. This correlated randomness is store by the parties for use in the online phase. In the online phase, the parties consume the generated correlated randomness to aid in the computation of the function. This model significantly improves the efficiency of the online phase (reducing communication and rounds of interaction between parties) at the cost of computing a preprocessing phase.

Starting with the work of Boyle et al. [15, 17, 19], a new paradigm called *silent secure computation* was put forth to improve significantly the efficiency of the offline phase (namely the generation of correlated randomness) with the help of *pseudorandom correlation generators* (PCGs) and *pseudorandom correlation functions* (PCFs) [18],⁶ two primitives which allow parties to locally expand a short seed into a large amount of correlated *pseudorandomness*. This paradigm enables efficient deployments of multi-party computation protocols in the preprocessing model by significantly reducing the availability and interaction overheads incurred on the parties in the preprocessing step.

In recent years, many constructions of PCGs and PCFs have been introduced, for various types of useful pseudorandom correlations such as oblivious transfer (OT) correlations [16, 17] and Beaver triple correlations [12, 19]. These constructions have in turn enabled improved designs of useful secure computation protocols such as private set intersection [28, 61, 63], threshold signatures [3, 48, 58], or privacy-preserving machine learning [56, 62]. These constructions even extend beyond secure computation, impacting related primitives such as zero-knowledge proofs [9, 45, 69–71].

1.1 The additive barrier

The type of correlated randomness that best benefits a secure computation protocol is application-dependent, and an important research effort has been devoted to diversifying the kind of correlations that PCGs and PCFs can produce. To date, we have constructions for OT correlations, Beaver triples, vector oblivious linear evaluations [15], inner-product correlations, matrix multiplication triples [19], one-time truth tables [17], general NC^1 additive correlations [60], unit-vector correlations [6], and more. However, all these correlations are special forms of *additive correlations*. In a 2-party additive correlation, the players get random vectors $\mathbf{x}$ and $\mathbf{y}$ (over some ring) respectively, together with *additive shares* of $f(\mathbf{x}, \mathbf{y})$, where f is some function computable by a polysize circuit over the ring. In fact, even when using “heavy hammer” cryptography, such as strong forms of fully-homomorphic encryption [17, 46], the same barrier remains: all known constructions are stuck at additive correlations. As of today, the only known exceptions are constructions based on indistinguishability obfuscation [4, 46].

This state of affairs is unfortunate, as there are many natural types of *non-additive* correlations that would be useful in secure computation protocols. Perhaps the most natural example would be the generalization of additive correlations to *threshold shared* correlations, where n parties receive random vectors $(\mathbf{x}_1, \dots, \mathbf{x}_n)$ together with shares of $f(\mathbf{x}_1, \dots, \mathbf{x}_n)$, but where the sharing is a t -out-of- n threshold secret sharing instead of an additive sharing. PCGs and PCFs for such variant would immediately imply silent secure computation protocols tolerating a broader class of corruption thresholds. This is would allow, for example, to lower the corruption threshold in secure computation protocols by tolerating a number of failures from the participant, and make protocols more resilient to errors.

Other natural motivations for studying non-additive correlations include generating garbled circuit correlations (one party gets a random garbled circuit, and the other party gets the corresponding key) and generating random permutations. Constructions for these types of correlations would imply, among more, silent secure computation with optimal communication and low round complexity, and better constructions of secret leader elections and committee selection protocols.

⁶ Roughly speaking, a PCF can be used to generate a fresh sample from the target correlation “on the fly” via an evaluation algorithm whereas PCGs require expanding all correlations in a single monolithic evaluation.

1.2 Our contributions

In this work, we initiate a study of pseudorandom correlation generators and functions beyond the additive barrier. Our focus is on obtaining PCGs and PCFs for specific but natural target correlations, without relying on indistinguishability obfuscation (or any form of heavy hammer cryptography). More specifically, we focus on two correlations:

- *random permutations*, where for a random permutation π over $\{1, \dots, n\}$, the correlation securely distributes, to each party i , the value $\pi(i)$; and
- *Shamir-shared correlations*, where the correlation securely distributes threshold shares of a secure computation output over pseudorandom inputs (e.g., Shamir secret shares of a Beaver triple).

In both cases, through different techniques, we show how to obtain weak, but non-trivial, forms of the above correlations from standard “non heavy hammer” assumptions. We summarize our results and compare them to the state of the art in Table 1.

Concretely, we show the following:

3-party pseudorandom permutations. We first introduce pseudorandom permutation generators (PPGs) and functions (PPFs) in the 3-party setting. From now on, by a “pseudorandom permutation (correlation)” for three parties we mean the joint distribution obtained by first sampling a permutation $\pi \xleftarrow{R} S_3$ uniformly and then giving party $i \in \{0, 1, 2\}$ the value $r_i = \pi(i)$. A PCF implements this correlation if the joint output (r_0, r_1, r_2) is computationally indistinguishable from $(\pi(0), \pi(1), \pi(2))$ for a uniform π . Each marginal r_i is uniform on $\{0, 1, 2\}$. Any two parties can recover π , while one party learns nothing beyond its own r_i .⁷

Our main construction relies on homomorphic secret sharing for restricted-multiplication straightline (RMS) programs and PRFs in NC^1 . Both assumptions are implied by a variety of standard cryptographic assumptions, such as DCR [60, 64], the hardness of the Joye-Libert encryption scheme, the DDH and DXDH assumptions over class groups, the Hard Subgroup Membership assumption over class groups [2], and the LWE assumption with super-polynomial modulus-to-noise ratio [27]. While our main focus is on feasibility, we additionally show that by replacing the PRF with a suitable low-complexity (weak) PRF, our construction can actually achieve good concrete efficiency. We provide a candidate concrete instantiation for the weak PRF, and estimate that generating a pseudorandom permutation takes less than 100ms on one core of a standard laptop. Our construction tolerates a single corruption: this is the maximum threshold for 3-party permutations, as any coalition of 2 parties necessarily learns the full permutation.

As a secondary contribution, we also describe a 3-party PPG from the hardness of the quasi-abelian syndrome decoding problem, an assumption recently introduced by Bombar et al. [12]. Compared with our main construction, the PPG construction has a few downsides: (1) it is only a PPG (hence all permutations are generated in a one-shot fashion, which limits its usefulness in applications), and (2) it generates a *biased* pseudorandom permutation, where each party can guess the indices received by the other two with probability $2/3$. On the other hand, it is significantly more concretely efficient and can generate a few million permutations per second.

Shamir-shared correlations for a single corruption. As a direct byproduct of the techniques introduced for our pseudorandom permutation function, we obtain pseudorandom correlation functions for distributing 1-out-of- n Shamir shares [65] of any relation computable by an RMS program (a class of programs that captures in particular all NC^1 functions), from the same assumptions.

A recent work by Agarwal et al. [6] shows how to construct distributed point functions outputting 1-out-of- n Shamir shares from OWFs. We note that plugging this construction in the LPN-based DPF-to-PCG compiler of Boyle et al. [17] yields a PCG for 1-out-of- n Shamir shares of constant degree correlations. Our result is incomparable to this approach, since we rely on different assumptions and obtain a pseudorandom correlation *function* (rather than generator) for a much larger class of functions. In contrast, however, our constructions have a worse concrete efficiency.

⁷ This is *not* the block-cipher notion of a keyed PRP. Here the permutation is sampled at correlation time and only revealed per-coordinate.

Primitive	Assumption	Target Correlation	# parties
PCF [46]	$i\mathcal{O}$	Any reverse-samplable	2
PCF [4]	multi-key FHE + $i\mathcal{O}$	Any reverse-samplable	n
PCF [4]	multi-key FHE + $i\mathcal{O}$ + ROM	Any	n
PCG [6]+ [17]	LPN	$\binom{n}{1}$ -Shamir shares of const. degree poly	n
This work			
PCF (Section 4)	HSS+PRF in NC^1	Permutations	3
PCF (Section 4)	HSS+PRF in NC^1	$\binom{n}{1}$ -Shamir shares of NC^1 functions	n
PCG (Section 5)	LPN+MQ	$\binom{n}{t}$ -Shamir shares of const. degree poly	n
PCG (Section 5)	LPN+MQ	$\binom{n}{n-t}$ -threshold shares of const. degree poly	n
PCG (Appendix B)	Quasi-Abelian SD	Permutations (biased)	3

Table 1: Constructions of PCGs and PCFs for non-additive correlations. Colors highlight results that do not require $i\mathcal{O}$. Reverse-samplable means that, given the corrupt parties’ outputs of a correlation sample, the honest parties’ outputs can be efficiently simulated in a way that is indistinguishable from the original sample. The value t denotes a constant.

Shamir-shared correlations for t corruptions with constant t . The previous constructions do not generalize to any number of corruptions above 1. Building upon the recent work of Couteau et al. [40], we construct a pseudorandom correlation generator for t -out-of- n Shamir shares of constant-degree correlations, for any constant t . Our constructions are based on the conjunction of the learning parity with noise (LPN) and multivariate quadratic (MQ) assumptions.

Threshold-shared correlations for $n - t$ corruptions for constant t . We also provide a different approach to build a PCG for $(n - t)$ -out-of- n threshold shares of constant-degree correlations, for any constant t , from the same assumptions (LPN and MQ). In contrast with the previous constructions, however, the underlying (linear) threshold secret sharing scheme is not a Shamir secret sharing. Nevertheless, it enjoys essentially the same applications (as any linear secret sharing with the same threshold can usually be used as a drop-in replacement in MPC applications using Shamir secret sharing).

The techniques we introduce especially around share distinctness and programmability, lay down foundational tools for dealing with richer classes of correlations. In addition, our results already have direct applications, as we outline in Section 1.3.

Future directions. Our constructions demonstrate that some useful and non-trivial non-additive correlations can be generated without resorting to indistinguishability obfuscation; prior to our work, no such result was known (beyond the natural construction of PCG for degree-1 Shamir shares from LPN implied by the recent DPF of [6]). However, our new constructions do come with some limitations: a restriction to 3 parties for our PPF, and a restriction to t or $n - t$ being constant for our PCGs for threshold secret sharing of low-degree correlations. As such, we view our contributions as opening a new and well-motivated research direction by providing an initial set of results and techniques. This creates exciting areas for future work, in particular by highlighting that non-additive PCGs and PCFs are possible without relying on $i\mathcal{O}$, a milestone previously thought out of reach.

1.3 Applications

While a main focus of our work is on the conceptual contribution—demonstrating some first feasibility results for constructing PCGs/PCFs for non-additive correlations without resorting to $i\mathcal{O}$ —our constructions also have concrete applications. We outline two such applications next.

Pseudorandom permutation functions. We show that PPFs have a natural application in anonymous communication protocols and single secret leader election. We find that we can replace the expensive “offline” preprocessing protocol of Studholme and Blake [66] (which is used to

generate pseudorandom permutations) with a PCF, to instantiate a Dining-Cryptography network for anonymous broadcast with optimal communication overheads. In particular, we show that using a PCF, each party can locally determine where to write their message in a bulletin board, in each round. Here, the pseudorandomness of the permutation guarantees that no other party learns the association between messages and the other parties and the permutation itself guarantees each party writes to a uniquely assigned “slot” removing the need for redundancy (a common solution used in prior work [37, 53]). We also show how a PCF for pseudorandom permutations enable a non-interactive single secret leader election (SSLE) protocol, where the leader is elected by having the lowest permutation value. In turn, SSLE has applications to proof-of-stake cryptocurrencies and other problems in distributed systems [13].

PCGs for threshold secret sharing. Plugging our new PCGs in existing secure computation protocols in the correlated randomness model, we demonstrate the existence of n -party secure computation protocol with *silent preprocessing* (concretely, the preprocessing phase can be implemented with $\text{poly}(n) \cdot \text{polylog}(s)$ communication for generating length- s correlations) tolerating either t or $n - t$ corruptions for a constant t . Previously, such a result was only known in the maximal ($n - 1$ out of n) corruption setting. The setting of $n - t$ corruptions is especially appealing, as it translates to MPC protocols with silent preprocessing secure against adversaries corrupting almost all of the parties, but resilient to a small (constant) number of failures ($n - t + 1$ participants suffice to run the protocol to completion). Such resiliency to failures is crucial in applications where a failure to produce an output can have important consequences, e.g., in threshold signatures (and indeed, the use of PCGs for additive correlations has been pointed as a significant limitation of [3]).

The work of [17] also showed that n -party PCGs for constant-degree correlations imply secure computation with almost-sublinear communication $n \cdot s/c$ for circuits of size s and any constant c . Plugging our PCGs in their compiler directly implies the same result for $n - t$ corruptions with resiliency to $t - 1$ failures.

Both applications (to silent MPC and to sublinear MPC) follow immediately by plugging our new PCGs into existing compilers from previous works. Therefore, we refrain from providing a detailed treatment of these applications in the main body of this work and will only note their existence here.

2 Technical Overview

In this section, we provide a detailed technical overview of our approach and construction. In Section 2.1, we briefly overview the syntax of PCGs. In Section 2.2, we overview the main ideas and general approach behind our PCF and PCG for permutation correlations, and in Section 2.3, the ideas behind are PCGs in the threshold setting.

2.1 Background

We start by giving a brief overview of PCGs and PCFs to aid in understanding of the technical overview. See Section 3 for formal definitions. A PCG for a target two-party correlation $\mathcal{C}$ consists of two algorithms:

- $\text{Gen}(1^\lambda) \rightarrow (k_0, k_1)$: a randomized algorithm that generates a pair of short, correlated seeds (k_0, k_1) given a security parameter λ , and
- $\text{Expand}(k_\sigma) \rightarrow R_\sigma$: a deterministic algorithm which stretches a seed k_σ into a long output string R_σ .

Remark 1 (PCFs). In the case of a PCF, Expand is replaced with an evaluation algorithm Eval that takes an additional input x and outputs a single instance of the correlation (computed as a function of x).

The security requirement for a PCG is that the joint outputs (R_0, R_1) should be indistinguishable from $\mathcal{C}$, not only to external parties but also to each of the parties that possesses only one of the seeds. PCFs have an analogous security requirement with the main difference being on-demand evaluation of R_0 and R_1 .

Our constructions will rely on homomorphic secret sharing (HSS). Informally, an n -party HSS for a class of functions $\mathcal{F}$ is a pair of algorithms (Share, Eval) where Share(x) distributes n shares $(x_1, \dots, x_n)$ of an input x , and for any $f \in \mathcal{F}$, Eval(i, x_i, f) produces a share y_i of $f(x)$ —that is, $\text{Rec}(y_1, \dots, y_n) = f(x)$, where Rec is the reconstruction procedure for the output shares. Homomorphic secret sharing with additive reconstruction are known for the class of RMS programs (a superclass of NC^1) from a variety of standard assumptions [2, 20, 23, 27, 60, 64] in the 2-party setting, and for low-depth circuits in the 4-party [21, 34] and 8-party [39] setting. In this work, we will be interested in the setting where the outputs are shared via a t -out-of- n secret sharing scheme, such as Shamir’s scheme [65].

2.2 Pseudorandom permutation correlations and $\binom{n}{1}$ -HSS

For permutation correlations, we focus on the three-party setting with parties Alice, Bob, and Carol. However, we note that our main HSS construction naturally extends to n parties with one corruption.

Pseudorandom permutations: a strawman approach. Examining a three party permutation, we first observe that we can view the problem as generating an additive sharing of the “zero correlation” (modulo 3), where the parties always obtain additive pseudorandom shares of zero, with additional constraints on the shares. To see this, note that a permutation over the set $\{0, 1, 2\}$ can be viewed as giving parties pseudorandom shares of zero if the sum of the permutation values (when parsed as additive shares) is always 3. This is because 3 over the integers is equivalent to zero in $\mathbb{F}_3$. Unfortunately, while all pseudorandom permutations can be viewed as shares of zero, not all pseudorandom shares of zero form a valid pseudorandom permutation. Specifically, in the three-party case, all shares of zero that consist of the same value in $\mathbb{F}_3$ (e.g., where all parties get 2 as the share), result in parties having shares of zero but an invalid permutation (since all parties have the same output, namely 2). Therefore, the problem of constructing a PCG for permutations (in the three party case) boils down to ensuring that all three “pseudorandom shares of zero” held by the parties are distinct.⁸ However, eliminating these “bad” cases turns out to be non-trivial.

Guaranteeing distinct shares. Our first observation is that for just two parties, say Alice and Bob, we can start by having the parties generate shares of a pseudorandom *bit* $\mu \in \{0, 1\}$ over $\mathbb{F}_3$. Then, using shares of μ we can guarantee *distinct shares* between them as follows. Let $a \in \mathbb{F}_3$ be Alice’s share of μ and $b \in \mathbb{F}_3$ be Bob’s share of μ , such that $a + b \pmod{3} = \mu$. If Alice outputs a and Bob outputs $2 - b$, as their components of the pseudorandom permutation, then it is clear that the outputs of Alice and Bob are trivially distinct as $2 - b = a + 2 - \mu \pmod{3}$. Furthermore, Alice cannot distinguish whether Bob outputs $a + 1$ or $a + 2$, ensuring privacy of the permutation so far. However, how can Carol get her share of the permutation? Ideally, we’d like to repeat the same process between Alice and Carol, such that Carol’s output is distinct to both Bob’s and Alice’s output, and Alice’s output/share stays the same. However, this presents a challenge: Carol’s share must somehow correlate with Bob’s share to ensure they also have distinct outputs *and* Alice’s share needs to remain the same. Overcoming this turns out to be the crux of our solution to compressing pseudorandom permutations in the three party setting. However, ignoring this problem for the time being, we first explain how this gives us a template for a three-party construction.

The general template. Notice that if Alice and Bob, and Alice and Carol, can each generate a pseudorandom *subtractive* sharing of a bit μ and of $1 - \mu$, respectively, such that Alice maintains the same share a in both cases, then Carol’s share c will be such that $2 - c \pmod{3}$ is guaranteed to be distinct from both Alice’s and Bob’s output. Specifically, if we expand the outputs of each party, we have that:

$$\begin{array}{ll} \text{Alice outputs:} & a := x \\ \text{Bob outputs:} & b := x + 2 - \mu \\ \text{Carol outputs:} & c := x + 1 + \mu \end{array}$$

⁸ It suffices to ensure that the parties have shares $a + b + c = 0$ and $a \neq b$. This implies a, b and c are pairwise distinct, since if e.g., c was equal to b then we have $a = -2b = b \pmod{3}$, a contradiction.

Note that a, b, c are pairwise distinct. This is because $a = b$ would imply $\mu = 2$; $a = c$ would imply $\mu + 1 = 0$; and $b = c$ would imply $2\mu = 1 \pmod{3}$. All these statements are false for $\mu \in \{0, 1\}$. More precisely, Bob and Carol must be given the ability to generate shares of pseudorandom bits μ and of $1 - \mu$, respectively.

In summary, this brings us to a sufficient condition for compressing a pseudorandom permutation among three parties: Generating pseudorandom shares of a bit $\mu \in \mathbb{F}_3$ and pseudorandom shares of $1 - \mu$ while fixing one of the shares to be the same in both cases. This turns out to be non-trivial to achieve. Below, we elaborate on a natural approach and explain where it gets stuck, before outlining our final solution.

A connection to Shamir sharing. We observe that the above requirement can equivalently be phrased in terms of a variant of 1-out-of-3 Shamir sharing, where the secret bit μ is encoded in the degree-1 coefficient of the polynomial, instead of the constant term.⁹ Concretely, for random $x \in \mathbb{F}_3, \mu \in \{0, 1\}$, consider the polynomial $p(y) = \mu y + x$, and evaluations:

$$p(0) = x \qquad p(-1) = x - \mu \qquad p(1) = x + \mu$$

Clearly, the above shares can be directly converted into the outputs of Alice, Bob and Carol in the above template, leading to a random permutation. Thus, if we had a way of generating pseudorandom Shamir shares of $\{0, 1\}$ values over $\mathbb{F}_3$, without interaction, we would be done. Unfortunately, known techniques for pseudorandom secret sharing [42] do not suffice, as they only generate shares of *uniform* secrets over a field.

Instead, note that this could be solved using a three-party *homomorphic secret sharing* scheme. Homomorphic secret sharing (HSS) can be seen as a distributed analogue of FHE that allows two or more parties to perform computations on inputs that are distributed via a special type of secret sharing. If we have a scheme that (1) supports homomorphic evaluation of a function class that includes a pseudorandom function, and (2) gives Shamir shared outputs, then we could build a 3-party PCF for permutation correlations as follows. Start by secret-sharing a key for a pseudorandom function, then have each party homomorphically evaluate the “arithmetized” circuit that produces one bit of the PRF output over $\mathbb{F}_3$. The Shamir-shared result can then be mapped to a permutation as explained above. We refer to such a scheme as $\binom{3}{1}$ -HSS, or more generally, $\binom{n}{t}$ -HSS. Unfortunately, as of today, no construction of $\binom{n}{t}$ -HSS for $n \leq 2$ is known without resorting to indistinguishability obfuscation. Our core observation, which may be of independent interest, is that many existing HSS schemes satisfy an additional “simulatability” property that allows us to upgrade them to $\binom{n}{1}$ -HSS for any n .

From two-party HSS to $\binom{n}{1}$ -HSS, generically. The most expressive of today’s HSS constructions support evaluation of large classes of functions including arbitrary, polynomially-sized circuits, using LWE [34, 46], or a form of restricted-multiplication circuits (that includes NC1) under group-theoretic assumptions including DDH, DCR and class groups [2, 23, 60, 64]. These schemes are all limited to additively shared outputs. We show that many of these two-party schemes can be used to build an n -party scheme with *Shamir shared outputs*, for threshold $t = 1$, in a generic manner.

The family of constructions relevant for us is the two-party constructions for restricted-multiplication circuits, that is, circuits where every multiplication gate is between an input wire and an intermediate wire of the computation. These constructions feature two types of sharings: *input sharings*, which are typically given as public ciphertexts encrypting an input to the computation, and *memory sharings* for intermediate wires, which typically are additive shares of some secret value.

Towards handling Shamir-shared outputs, we rely on a recent observation of Couteau et al. [41] that these HSS schemes have a property called “simulatable memory shares” which, informally speaking, means that one of the party’s shares of memory values can be sampled independently of the value of that input. Essentially, this holds because of the additive structure of these shares.

⁹ Also referred to as the ‘infinity point’ of the polynomial. Note that using the infinity point allows us to securely give out 3 shares over $\mathbb{F}_3$, while the standard choice of evaluation points $\{0, \dots, n\}$ requires a field strictly larger than the number of parties.

We observe that this can be exploited in the degree-1, Shamir setting, as follows. To share an input x , we generate HSS shares of x using the two-party scheme, giving one share to party P_0 and the other share to every other party P_i , for $i = 1, \dots, n-1$. Now, after homomorphic evaluation, parties (P_0, P_i) can obtain the *same* sharing of the result, namely, shares y_0 and y_1 such that $y_1 = y_0 + f(x)$. Note that P_0 's share can already be seen as an evaluation of the polynomial $p(z) = y_0 + z \cdot f(x)$ at the point 0. To allow P_i to get $p(i)$, we exploit the fact that HSS schemes with simulatable memory shares additionally have a property that allows for a ‘tweaked’ form of homomorphic evaluation, where instead of shares of $f(x)$, the parties get shares of $\alpha \cdot f(x)$ for some α given as memory shares. Crucially, we can achieve this *without* changing P_0 's output share for each of the $n-1$ pairs of HSS evaluations, by relying on the ability to sample P_0 's share of α independently of the value of α .

Overall, by relying on two-party HSS constructions for NC^1 functions, based on LWE, DCR, or assumptions in class groups, we can obtain a $\binom{n}{1}$ -HSS for the same class of circuits. With $n = 3$, we use this observation to directly build a PCF for permutation correlations, by obviously evaluating a pseudorandom function using HSS.

Building $\binom{n}{1}$ -PCFs. In addition to the PCF for 3-party permutation correlations, we remark that $\binom{n}{1}$ -HSS generically gives us a PCF for threshold 1-out-of- n , Shamir-shared randomness for any sampling functionality that admits a sampling algorithm in NC^1 . This follows in the natural way, by distributing $\binom{n}{1}$ -HSS shares of a random seed, followed by first evaluating a PRF inside NC^1 , and then feeding the resulting shares of pseudorandomness into the sampling algorithm.

A weaker, PCG construction from a variant of LPN. We also give an alternative construction of a PCG for 3-party permutations, relying on the quasi-abelian syndrome decoding problem over $\mathbb{F}_3$, a variant of LPN introduced in [12]. In a nutshell, this construction instantiates the general template outlined above, by building a PCG for secret-shared random bits modulo 3. Unfortunately, the random bits are biased (by $\frac{1}{6}$), leading to only a biased, three-party permutation PCG. Nevertheless, the techniques we use here may be useful in future improvements, and the construction is concretely faster than the HSS-based one. We defer the details of this construction to Appendix B.

2.3 PCG for $\binom{n}{t}$ secret sharing

We now describe how we construct PCGs for $\binom{n}{t}$ secret shares. The construction expands on the ideas introduced in Boyle et al. [17] building PCGs from distributed point functions.

Starting point. A distributed point function is a primitive that distributes shares of a point function $f_{\alpha,\beta}$ to n parties (a point function $f_{\alpha,\beta}$ satisfies $f(\alpha) = \beta$ and evaluates to 0 on all $x \neq \alpha$) such that given any input x , the parties can locally compute shares of $f_{\alpha,\beta}(x)$. The pseudorandom correlation generator originally introduced by Boyle et al. [17] can be viewed as a general compiler that, given an n -party DPF over a domain of size $O(N^d)$ and assuming LPN, provides an n -party PCG for degree- d N -variate correlations whose seed length is dominated by $O(\lambda^d)$ DPF keys. State-of-the-art PCGs are instantiations of this template with the PRG-based 2-party additive DPF of Boyle et al. [24]. In this section, we describe extension of this template to correlations forming t -out-of- n shares of constant-degree polynomials.

A first observation. We first note that there is a straightforward construction of a DPF for $\binom{n}{t}$ -sharing from an additive DPF for $\binom{n}{t}$ parties: use the $\binom{n}{t}$ -party additive DPF to generate $\binom{n}{t}$ keys, associate one key to each subset of t parties, and distribute this key to all parties in this subset. This yields a $\binom{n}{t}$ replicated secret sharing of the DPF output, which can be locally converted to $\binom{n}{t}$ Shamir shares. When t is a constant, $\binom{n}{t}$ remains polynomial and this construction is efficient. Then, plugging an n -party DPF for $\binom{n}{t}$ Shamir sharing into the LPN-based compiler of Boyle et al. [17] yields an n -party PCG for $\binom{n}{t}$ Shamir shares of degree- d N -variate polynomials.

Unfortunately, we do not know of an M -party additive DPF for arbitrary M without resorting to $i\mathcal{O}$ or spooky encryption [46]. While a recent work [40] provided the first construction of multiparty DPF from assumptions not known to imply FHE or $i\mathcal{O}$, their construction only tolerates a constant number of parties (or incurs an n^d blowup in the key size for a DPF over domain $[\lambda^d]$, which makes

it nontrivial only for very small values of n). To go beyond this barrier and construct a threshold PCG accommodating an arbitrary polynomial number of parties, our first approach is to use a refined construction based on replicated secret sharing. Our second approach relies on the party emulation framework to compose DPFs with constant-degree Shamir sharings accommodating polynomial number of parties proposed in [6] and threshold PCGs accommodating constant number of parties built as above.

First construction: constant $n - t$, polynomial n . For this construction, we assume that we have at our disposal a DPF for $\binom{n}{t}$ Shamir shares for constant n and t . As outlined above, this can be constructed generically given any $\binom{n}{t}$ -party additive DPF, which in turn can be based on LPN and MQ [40]. For simplicity, assume that we want to generate Shamir shares of a Beaver triple: the n parties should receive (many) $\binom{n}{t}$ Shamir shares of pseudorandom triples $(a, b, a \cdot b)$. Our PCG proceeds as follows: first, for every subset S of size t , generate pseudorandom values a_S, b_S and distribute them to all parties not in S . This can be done succinctly by generating many a_S 's and b_S 's (for a fixed S) as LPN samples, by dealing the corresponding noise vector to all parties outside of S . Concretely, we use the standard LPN-based template of [17] to generate each pseudorandom share: let $\mathbf{e}_S, \mathbf{f}_S$ denote random noise vectors (i.e., sparse vectors) and fix a public random compressing matrix H . Define $\mathbf{a}_S := H \cdot \mathbf{e}_S$ and $\mathbf{b}_S := H \cdot \mathbf{f}_S$. Under the LPN assumption, $(\mathbf{a}_S, \mathbf{b}_S)$ are indistinguishable from random. Furthermore, for all parties in S , it suffices to know $\mathbf{e}_S, \mathbf{f}_S$ to reconstruct them, which can be represented succinctly. Now, we leverage a classical observation from [19]: all component-wise products $\mathbf{a}_S \odot \mathbf{b}_T$ for subsets S, T can be read from the diagonal of $\mathbf{a}_S \cdot \mathbf{b}_T^\top = H \cdot (\mathbf{e}_S \cdot \mathbf{f}_T^\top) \cdot H^\top$, and each entry can be reconstructed given shares of $(\mathbf{e}_S \cdot \mathbf{f}_T^\top)$ (assuming the secret sharing scheme is linear). In turn, because $\mathbf{e}_S, \mathbf{f}_T$ are sparse (weight- w), so is their tensor product (weight- w^2), hence they can be expressed as a sum of w^2 point functions (evaluated on their entire domain).

Now, define $a = \sum_S a_S$ and $b = \sum_S b_S$. We have $a \cdot b = \sum_{S, T} a_S \cdot b_T$. To share $a \cdot b$, we need to share all terms $a_S \cdot b_T$ for all pairs of t -sized subsets S, T . To share all $a_S \cdot b_T$ (for each fixed S, T), we observe that it suffices to distribute to the parties outside of $S \cup T$ the keys of w^2 DPFs for $(m - (n - t))$ -out-of- m sharing, with $m = |([n] \setminus S) \cup ([n] \setminus T)|$, for each entry in $\mathbf{e}_S \otimes \mathbf{f}_T$ (where $\otimes$ denotes the tensor product). Fortunately, m and $n - t$ are constants, hence this can be achieved using the DPF from LPN+MQ outlined above. The full construction involves an $\binom{n}{n-t}^2$ overhead for generating the shares of all cross-terms. Generalizing the construction to arbitrary degree- d correlations (for constant d) is straightforward, at the cost of increasing the overhead to $\binom{n}{n-t}^d$.

In our next construction, we handle the complementary case where t is a constant. Note that our first construction does not generalize to this setting even though there is a polynomial number of subsets S of t parties, because it requires a $(m - (n - t))$ -out-of- m DPF, and m and $(n - t)$ would not be constant anymore. As such, a different strategy is required.

Second construction: polynomial n and constant t . Our second construction is more involved. Towards building an $\binom{n}{t}$ -DPF and compiling it to $\binom{n}{t}$ -PCG with polynomial n and constant t , we rely on the party emulation technique to compose the $\binom{n}{1}$ -DPF with polynomial n recently proposed in [6] and threshold PCGs accommodating constant number of parties (as constructed above). This is inspired by the work of Cohen et al. [36]. Focusing on the threshold-adversary setting, Cohen et al. [36] adapts the player emulation framework established by Hirt and Maurer [55] and manages to efficiently reduce multiparty MPC protocol to MPC protocols for a constant number of parties. More specifically, starting with a 3-party protocol π secure against 1 corruption, it iteratively emulates (virtual) parties inside the protocol with a 3-party protocol. Importantly, representing the 3-party protocol by a single gate Maj_3 , the set of corrupted parties against which can be described by a formulae composed of 3-input majority gates according to the way the virtual parties are emulated. Motivated by this, [36] studies the complexity of computing an n -input majority function (and more generally n -input threshold functions) using only 3-input majority gates (constant-size threshold gates).

Considering security against all-but-one corruption, the nesting approach is used in [21, 46] to overcome the limitation. Intuitively, the evaluation function of a 2-party HSS scheme (inner HSS) is distributed by another 2-party HSS scheme (outer HSS), resulting in an 4-party HSS scheme, which has the same spirit as the party emulation where a virtual party involved in one protocol

(inner scheme) is emulated by another protocol (outer scheme). Moreover, multi-level nesting is extensively used by Couteau and Kumar [39] to build 8-party HSS and 10-party sublinear secure computation.

Going back to our goal of constructing a DPF against a threshold adversary, we consider composing HSS schemes distributed among a constant number of parties (the outer scheme) to compute the evaluation function of an $\binom{m}{1}$ -DPF accommodating polynomial number of parties (the inner scheme). The observation is that if a t -party additive HSS is distributed among every group of t parties, resulting in $\binom{n}{t}$ virtual parties, then any adversary corrupting at most t parties can corrupt at most one virtual party. Hence, an $\binom{m}{1}$ -DPF with $m := \binom{n}{t}$ suffices to ensure the privacy. Moreover, the information held by any subset of $t + 1$ parties can reconstruct t keys of the $\binom{m}{1}$ -DPF scheme and thus recover the whole point function. Then:

- Combining the recent $\binom{m}{1}$ -DPF of [6] with the method introduced in [39] to reduce the complexity of GGM-style constructions under MQ (by using a tree of larger arity and a constant-degree PRG), we obtain an MQ-based $\binom{m}{1}$ -DPF computable by constant degree polynomials, that we use as our inner scheme.
- An HSS scheme can be obtained by going through the LPN-based DPF-to-HSS compiler introduced recently in [40], and used as our outer scheme.

We emphasize that since all parties cannot obtain degree- t Shamir sharings of the truth table for the secret point function in the constructed DPF scheme, the obtained DPF scheme is not a “Shamir-shares” $\binom{n}{t}$ -DPF: the underlying secret sharing scheme is a linear t -out-of- n secret sharing, but it is not Shamir’s secret sharing. We refer the reader to Section 5.5 for the full details of the construction.

3 Preliminaries

Notation. We let $\mathbb{N}$ denote the set of natural numbers. We denote a vector $\mathbf{u}$ using bold lowercase letters and let $\mathbf{u}[i]$ denote the i -th coordinate of $\mathbf{u}$. We denote by $\text{poly}(\cdot)$ the set of all polynomials and by $\text{negl}(\cdot)$ any negligible function. We let $x \xleftarrow{\mathcal{R}} S$ denote a uniformly random sample drawn from S . We let $x \leftarrow \mathcal{A}$ denote assignment from a (possibly randomized) algorithm $\mathcal{A}$ and $x := y$ denote initialization of x to the value of y . By an *efficient* algorithm $\mathcal{A}$ we mean that $\mathcal{A}$ is modeled by a (possibly non-uniform) Turing Machine that runs in probabilistic polynomial time. We write $D_0 \approx_c D_1$ to mean that two distributions D_0 and D_1 are *computationally* indistinguishable to all efficient distinguishers $\mathcal{D}$ and $D_0 \approx_s D_1$ to mean that D_0 and D_1 are *statistically* indistinguishable.

3.1 Homomorphic Secret Sharing

Homomorphic Secret Sharing (HSS) [23] can be seen as a distributed analogue to homomorphic encryption. In contrast to the centralized model where a single server computes on encrypted inputs, in HSS, shares of the input are distributed among multiple parties, who then compute a share of the output through local homomorphic computations. Here, we use a threshold variant of HSS, similar to the definition from [26].

Definition 2 (Homomorphic Secret Sharing [23]). *Let λ be a security parameter and $\mathcal{R}$ be a finite ring. Let $t \in \mathbb{N}$ be some threshold and $n \in \mathbb{N}$ be the number of parties. A (t, n) -Homomorphic Secret Sharing (HSS) scheme for a class of programs $\mathcal{P}$ with input space $\mathcal{I} \subseteq \mathcal{R}$ and share space $\mathcal{S} \subseteq \mathcal{R}$, consists of three efficient (possibly randomized) algorithms $\text{HSS} = (\text{Setup}, \text{Input}, \text{Eval}, \text{Rec})$ such that:*

- $\text{Setup}(1^\lambda) \rightarrow (\text{pk}, \text{ek}^i)$. Takes as input the security parameter λ . Outputs a public key pk and a pair of evaluation keys ek^i for each party $i \in [n]$.
- $\text{Input}(\text{pk}, x) \rightarrow \llbracket x \rrbracket^i$. Takes as input the public key pk and an input $x \in \mathcal{I}$. Outputs HSS input shares $\llbracket x \rrbracket^i$ for each party i .
- $\text{Eval}(\sigma, \text{ek}^\sigma, (\llbracket x_1 \rrbracket^\sigma, \dots, \llbracket x_n \rrbracket^\sigma), P) \rightarrow \langle y \rangle^\sigma$. Takes as input the party identifier σ , an evaluation key ek^σ , a vector of n HSS input shares, and an n -input program $P \in \mathcal{P}$. Outputs a share of the output $y = P(x_1, \dots, x_n)$ in share space $\mathcal{S}$.

- $\text{Rec}_T((\langle y \rangle^i)_{i \in T}) \rightarrow y$, where $T \subset [n]$ has size $\geq t$. Given a set of at least t shares of y , it reconstructs the output y .

The above functionality must satisfy the following properties:

Correctness. For all security parameters $\lambda \in \mathbb{N}$, any $n \in \mathbb{N}$, any $T \subset [n]$ with $|T| \geq t$, and any n -input program $P \in \mathcal{P}$ with input space $\mathcal{I} \subseteq \mathcal{R}$, it holds that for any $x_1, \dots, x_n \in \mathcal{I}$:

$$\Pr \left[\begin{array}{l} y = P(x_1, \dots, x_n) : \\ \begin{array}{l} (\text{pk}, \text{ek}^i) \leftarrow \text{Setup}(1^\lambda) \\ \llbracket x \rrbracket^i \leftarrow \text{Input}(\text{pk}, x) \quad \forall i \in [n] \\ \langle y \rangle^i \leftarrow \text{Eval}(i, \text{ek}^i, (\llbracket x \rrbracket^i)_{i \in [n]}, P) \\ y := \text{Rec}_T((\langle y \rangle^i)_{i \in T}) \end{array} \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Security. For all efficient adversaries $\mathcal{A}$, and any $T \subset [n]$, with $|T| \leq t$ there exists a negligible function $\text{negl}(\cdot)$ such that:

$$\Pr \left[\begin{array}{l} b' = b : \\ \begin{array}{l} (x_0, x_1, \text{st}) \leftarrow \mathcal{A}(1^\lambda) \\ (\text{pk}, \text{ek}^i) \leftarrow \text{Setup}(1^\lambda) \\ b \xleftarrow{R} \{0, 1\} \\ (\llbracket x_b \rrbracket^i) \leftarrow \text{Input}(\text{pk}, x_b) \\ b' \leftarrow \mathcal{A}(\text{st}, \text{pk}, \text{ek}^\sigma, \llbracket x_b \rrbracket^\sigma)_{\sigma \in T} \end{array} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

HSS memory values. In many existing, two-party HSS schemes [23, 41, 44, 49], we can consider so-called “memory shares” of intermediate values during an HSS evaluation held by party σ , which we denote by $\langle\langle x \rangle\rangle^\sigma$. These memory shares can be added together and multiplied with an HSS input share to produce a new memory share. Also, there is an additional efficient algorithm **MemGen** that takes as input an index $\sigma \in [n]$, and evaluation key ek_σ and an input $x \in \mathcal{I}$, and outputs a memory share $\langle\langle x \rangle\rangle^\sigma$ of x .

Extended Evaluation. Moreover, for the two party case specifically, HSS schemes support a so-called “extended evaluation”, as given by [41, Lemma 1]. Informally, there is an additional evaluation algorithm **ExtEval** that is similar to **Eval** but takes one additional input $M_\sigma = \langle\langle z \rangle\rangle^\sigma$ —a memory share of $z \in \mathcal{I}$ —and outputs a $\langle y \rangle^\sigma$, such that upon reconstruction $\langle y \rangle^0 - \langle y \rangle^1$ one obtains the z -multiplied output $z \cdot P(x_1, \dots, x_n)$. We will make use of this multiply our outputs by a $z \in \{-1, 1\}$.

More formally, there the syntax of **ExtEval** is as follows:

- $\text{ExtEval}(\sigma, \text{ek}^\sigma, \langle\langle \alpha \rangle\rangle^\sigma, (\llbracket x_1 \rrbracket^\sigma, \dots, \llbracket x_n \rrbracket^\sigma), P) \rightarrow \langle y \rangle^\sigma$. Takes as input an index σ , an evaluation key ek^σ , a memory share $\langle\langle \alpha \rangle\rangle^\sigma$ of $\alpha \in \mathcal{I}$ and a vector of n HSS input shares, and an n -input program $P \in \mathcal{P}$. Outputs a subtractive share of the output $y = \alpha P(x_1, \dots, x_n)$.

Correctness of this algorithm means that

$$\Pr \left[\begin{array}{l} y = z \cdot P(x_1, \dots, x_n) : \\ \begin{array}{l} (\text{pk}, (\text{ek}^A, \text{ek}^B)) \leftarrow \text{Setup}(1^\lambda) \\ (\llbracket x_i \rrbracket^A, \llbracket x_i \rrbracket^B) \leftarrow \text{Input}(\text{pk}, x_i) \quad \forall i \in [n] \\ M^A \leftarrow \text{MemGen}(A, \text{ek}^A, z) \\ M^B \leftarrow \text{MemGen}(B, \text{ek}^B, z) \\ \langle y \rangle^A \leftarrow \text{ExtEval}(A, \text{ek}^A, M^A, (\llbracket x_i \rrbracket^A)_{i \in [n]}, P) \\ \langle y \rangle^B \leftarrow \text{ExtEval}(B, \text{ek}^B, M^B, (\llbracket x_i \rrbracket^B)_{i \in [n]}, P) \\ y \leftarrow \langle y \rangle^A - \langle y \rangle^B \end{array} \end{array} \right] \geq 1 - \text{negl}(\lambda),$$

i.e., if we run it like **Eval**, but with the additionally generated memory share of a $z \in \mathcal{I}$, we get a subtractive share of $z \cdot P(x_1, \dots, x_n)$.

Programmability of HSS memory values. Here, we follow the definition of Couteau et al. [41, Section 4.3].

Definition 3 (HSS with Simulatable Memory Values). Let $\text{HSS} = (\text{Setup}, \text{Input}, \text{Eval})$ be an HSS scheme with input space $\mathcal{I}$. Then, HSS is simulatable w.r.t. its memory values if there are efficient algorithms $\text{Sim}_0, \text{Sim}_1$, where Sim_0 takes as input 1^λ and outputs a memory value M_0 , and Sim_1 takes as input a memory value M_0 , an element $z \in \mathcal{I}$, and two keys ek^A, ek^B , and outputs a memory value M_1 . Then we want:

Simulation Correctness. For any security parameter λ , and any $z_1, z_2 \in \mathcal{I}$, we want that correctness w.r.t. extended evaluation still holds when the memory values are simulated, i.e.

$$\Pr \left[\begin{array}{l} y = z \cdot P(x_1, \dots, x_n) : \\ \begin{array}{l} (\text{pk}, (\text{ek}^A, \text{ek}^B)) \leftarrow \text{Setup}(1^\lambda) \\ (\llbracket x_i \rrbracket^A, \llbracket x_i \rrbracket^B) \leftarrow \text{Input}(\text{pk}, x_i) \forall i \in [n] \\ M^A \leftarrow \text{Sim}_0(1^\lambda) \\ M^B \leftarrow \text{Sim}_1(M^A, z, (\text{ek}^A, \text{ek}^B)) \\ \langle y \rangle^A \leftarrow \text{ExtEval}(A, \text{ek}^A, M^A, (\llbracket x_i \rrbracket^A)_{i \in [n]}, P) \\ \langle y \rangle^B \leftarrow \text{ExtEval}(B, \text{ek}^B, M^B, (\llbracket x_i \rrbracket^B)_{i \in [n]}, P) \\ y \leftarrow \langle y \rangle^A - \langle y \rangle^B \end{array} \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Simulation Security. There exists an efficient simulator $\mathcal{S}$ such that for all $(\text{pk}, (\text{ek}^A, \text{ek}^B))$ in the image of Setup ,

$$\left\{ M_1 \mid \begin{array}{l} M_0 \leftarrow \text{Sim}_0(1^\lambda) \\ M_1 \leftarrow \text{Sim}_1(M_0, z, (\text{ek}^A, \text{ek}^B)) \end{array} \right\} \approx_c \mathcal{S}(\text{pk}).$$

3.2 Function Secret Sharing and Distributed Point Functions

Function secret sharing (FSS) [22, 24] is an analogue of HSS for sharing functions instead of data. Concretely, FSS allows to split up a function f into shares k_0, k_1 , such that each share individually hides the function f . The shares can be locally evaluated on an input x , such that the evaluations of both shares at the same x form shares of $f(x)$.

Definition 4 (Function Secret Sharing). Let $\mathcal{F} = \{f : I \rightarrow \mathbb{G}\}$ be a class of function descriptions, where the description of each f specifies the input domain I and an Abelian group $(\mathbb{G}, +)$ as the output domain. A (2-party) function secret sharing (FSS) scheme for $\mathcal{F}$ is a pair of algorithms $\text{FSS} = (\text{FSS.Gen}, \text{FSS.Eval})$ with the following syntax:

- $\text{FSS.Gen}(1^\lambda, f)$ is a PPT algorithm that given security parameter λ and description of $f \in \mathcal{F}$ outputs a pair of keys (K_0, K_1) . We assume that the keys specify I and $\mathbb{G}$.
- $\text{FSS.Eval}(b, K_b, x)$ is a polynomial-time algorithm that, given a key K_b for party $b \in \{0, 1\}$, and an input $x \in I$, outputs a group element $y_b \in \mathbb{G}$.

The scheme should satisfy the following requirements:

Correctness. For any $f \in \mathcal{F}$ and $x \in I$, we have $\Pr[(K_0, K_1) \xleftarrow{\$} \text{FSS.Gen}(1^\lambda, f) : \text{FSS.Eval}(0, K_0, x) - \text{FSS.Eval}(1, K_1, x) = f(x)] = 1$.

Security. Let $f_1, f_2, \dots$ be any polynomial-size sequence of functions $f_i \in \mathcal{F}$. For $b \in \{0, 1\}$, consider the following two experiments with a PPT simulator Sim :

- $\text{Real}_b(1^\lambda)$: Run $(k_0, k_1) \xleftarrow{\$} \text{FSS.Gen}(1^\lambda, f_\lambda)$ and output k_b
- $\text{Ideal}_b(1^\lambda)$: Output $k_b = \text{Sim}(1^\lambda, \text{Leak}(f_\lambda))$

We say that FSS is secure if for each $b \in \{0, 1\}$, there exists Sim such that, for any non-uniform adversary $\mathcal{A}$ of size $\text{poly}(\lambda)$, $\Pr[\mathcal{A}(\text{Real}_b(1^\lambda)) = 1] - \Pr[\mathcal{A}(\text{Ideal}_b(1^\lambda)) = 1] \leq \text{negl}(\lambda)$. In the constructions we use, the leakage function $\text{Leak} : \{0, 1\}^* \rightarrow \{0, 1\}^*$ is given by $\text{Leak}(f_\lambda) = (I, \mathbb{G})$, namely it outputs a description of the input and output domains of f .

Remark 5 (Pseudorandomness of FSS Outputs). In some of our constructions, we rely on a property that outputs of FSS.Eval are indistinguishable from random, when the FSS keys remain secret. This property can be obtained, generically, by including a PRF key in each FSS key and having each party add the same PRF evaluation to its output. However, we note that this is already achieved by all the FSS constructions we use.

Distributed Point Functions and FSS for Sums of Point Functions. Let N be a domain size, $\mathbb{G}$ an abelian group, $\alpha \in [N]$, and $\beta \in \mathbb{G}$. A *point function* is a function $f_{\alpha,\beta} : [N] \rightarrow \mathbb{G}$ that evaluates to 0 on all inputs $x \neq \alpha$, and $f_{\alpha,\beta}(x) = \beta$ if $x = \alpha$.

An FSS scheme for the class of point functions of domain size N is known as a *distributed point function* [51]. As in previous work [12, 19], it will be convenient for our constructions to use FSS for the class of functions that can be described as a sum of t point functions, which we define next.

Definition 6 (FSS for sum of point functions (SPFSS)). Let N be a domain size, $\mathbb{G}$ an abelian group, $\alpha \in [N]$, and $\beta \in \mathbb{G}$. A *point function* is a function $f_{\alpha,\beta} : [N] \rightarrow \mathbb{G}$ defined by $f_{\alpha,\beta}(x) = 0$ whenever $x \neq \alpha$, and $f_{\alpha,\beta}(x) = \beta$ if $x = \alpha$. For $S = (s_1, \dots, s_t) \in [N]^t$ and $\mathbf{y} = (y_1, \dots, y_t) \in \mathbb{G}^t$, the sum of point functions $f_{S,\mathbf{y}} : [N] \rightarrow \mathbb{G}$ is defined as

$$f_{S,\mathbf{y}}(x) = \sum_{i=1}^t f_{s_i, y_i}(x).$$

An SPFSS scheme is an FSS scheme for the class of sums of point functions.

3.3 Pseudorandom Correlation Generators

A pseudorandom correlation generator, or PCG, is a way of distributing short seeds to a set of parties, which can later be expanded to produce a large amount of correlated randomness. The type of correlated randomness obtained is specified as a correlation generator, which typically has the reverse-samplable property below. In the following definitions, we adapt the definition of n -party PCG [17] to the threshold case where up to t parties may be corrupted. For a set $T \subset [n]$, we denote by $\bar{T}$ the set $[n] \setminus T$.

Definition 7 (Reverse-samplable $\binom{n}{t}$ -Correlation Generator). A PPT algorithm $\mathcal{C}$ is called a reverse sampleable $\binom{n}{t}$ -correlation generator, if on input 1^λ it outputs elements in $\{0, 1\}^\ell \times \dots \times \{0, 1\}^\ell \in (\{0, 1\}^\ell)^n$ for some length parameter $\ell \in \text{poly}(\lambda)$, and there exists a PPT algorithm RSample such that for any $T \subset [n]$ of size $\leq t$, the correlation obtained via:

$$\left\{ \begin{array}{l} (R_1, \dots, R_n) \xleftarrow{\$} \mathcal{C}(1^\lambda) \\ (R'_\sigma)_{\sigma \in T} := (R_\sigma)_{\sigma \in T} \\ (R'_j)_{j \in \bar{T}} \xleftarrow{\$} \text{RSample}(T, (R_\sigma)_{\sigma \in T}) \end{array} \right\}$$

is computationally indistinguishable from $\mathcal{C}(1^\lambda)$.

Definition 8 (Pseudorandom $\binom{n}{t}$ -Correlation Generator). Let $\mathcal{C}$ be a reverse-samplable $\binom{n}{t}$ -correlation generator. A pseudorandom correlation generator (PCG) for $\mathcal{C}$ is a pair of algorithms $(\text{PCG.Gen}, \text{PCG.Expand})$ with the following descriptions:

- $\text{PCG.Gen}(1^\lambda)$ is a PPT algorithm that given a security parameter λ , outputs seeds $(k_1, \dots, k_n)$.
- $\text{PCG.Expand}(\sigma, k_\sigma)$ is polynomial time algorithm that given a party index $\sigma \in [n]$ and a seed k_σ , outputs a bit string $R_\sigma \in \{0, 1\}^\ell$.

The algorithms $(\text{PCG.Gen}, \text{PCG.Expand})$ should satisfy the following:

Correctness. The correlation obtained via:

$$\{(R_1, \dots, R_n) \mid (k_1, \dots, k_n) \xleftarrow{\$} \text{PCG.Gen}(1^\lambda), R_\sigma \leftarrow \text{PCG.Expand}(\sigma, k_\sigma) \text{ for } \sigma \in [n]\}$$

is computationally indistinguishable from $\mathcal{C}(1^\lambda)$

$\binom{n}{t}$ -Security. For any $T \subset [n]$, with $|T| \leq t$, the following two distributions are computationally indistinguishable:

$$\left\{ \begin{array}{l} ((k_i)_{i \in T}, (R_j)_{j \in [n] \setminus T}) \\ (k_1, \dots, k_n) \xleftarrow{\$} \text{PCG.Gen}(1^\lambda) \\ R_j \leftarrow \text{PCG.Expand}(j, k_j) : j \in \bar{T} \end{array} \right\}$$

and

$$\left\{ \left((k_i)_{i \in T}, (R_j)_{j \in [n] \setminus T} \right) \left| \begin{array}{l} (k_1, \dots, k_n) \xleftarrow{\$} \text{PCG.Gen}(1^\lambda) \\ R_i \leftarrow \text{PCG.Expand}(i, k_i) : i \in T \\ (R_j)_{j \in \overline{T}} \xleftarrow{\$} \text{RSample}(T, (R_i)_{i \in T}) \end{array} \right. \right\},$$

where RSample is the reverse sampling algorithm for the correlation $\mathcal{C}$.

3.4 Pseudorandom Correlation Functions

A PCF [18] can be seen as a PCG where each output of the target correlation can be computed “on-demand,” instead of being expanded all at once. Furthermore, as in a PRF, the number of possible outputs (that is, the domain size) may be exponential in the security parameter.

The syntax of a PCF consists of a key generation algorithm PCF.KeyGen , which outputs a set of correlated keys, as well as a PCF.Eval algorithm that uses a key to compute a single PCF output. In Appendix A, we give the formal correctness and security properties of a PCF with $\binom{n}{t}$ -security.

4 PCF for Permutations

In this section, we describe a construction that is based on two-party HSS with simulatable memory shares to obtain a PCF for unbiased, 3-party permutation correlations. The construction follows the overview from Section 2.2, by building a $\binom{3}{1}$ -HSS and using this to evaluate a pseudorandom function. We start by directly presenting the 3-party PPF construction, and then later describe how the same ideas can be used to build a $\binom{n}{1}$ -HSS construction with Shamir-shared outputs.

For this, let P be an HSS program for a PRF with 1-bit output. Denote it by F_κ , where κ is the PRF key. To run it, we need HSS shares of the key, generated via $(\llbracket \kappa \rrbracket^A, \llbracket \kappa \rrbracket^B) \leftarrow \text{HSS.Input}(\text{pk}, \kappa)$ [29]. Then, Alice gets $\llbracket \kappa \rrbracket^A$, and both, Bob and Carol get $\llbracket \kappa \rrbracket^B$.

Now, when Alice and either Bob or Carol run $\text{Eval}(\sigma, \text{ek}^\sigma, \llbracket \kappa \rrbracket^\sigma, P_x)$, they obtain, upon reconstruction, a pseudorandom bit $\mu = F.\text{Eval}(\kappa, x)$. Recall that we want Alice to obtain a share that, together with the share of Bob, is a share of a pseudorandom bit μ , and, together with the share of Carol, is a share of $1 - \mu$. Hence, we want to compute $\alpha \cdot \mu$, where in one case, $\alpha = 1$, and in the other case, $\alpha = -1$. To wit, it suffices that Carol subtracts 1 from her subtractive share, thereby converting it to a share of $1 - \mu$.

To do this, we can provide the α as memory shares and evaluate P_x for an PRF input x , using our extended evaluation (cf. Section 3.1). And, as we want to keep Alice’s share fixed, we do so by using memory shares M^A, M^B that are simulatable, i.e., Alice uses $M^A \leftarrow \text{Sim}_0(1^\lambda)$ and to get a memory share, whose value is not yet fixed, but can be determined by generating a share using Sim_1 .

Later, this is programmed twice, to encode certain $\alpha_B \in \mathcal{I}$ but also $\alpha_C \in \mathcal{I}$ using Sim_1 , for use with Bob and Carol, respectively. Then, ExtEval will output a share of $\alpha_p \cdot F.\text{Eval}(\kappa, x)$ for $p \in \{B, C\}$, where $F.\text{Eval}(\kappa, x)$ is pseudorandom bit as output by the PRF. As motivated above, we choose $\alpha_B = 1$ and $\alpha_C = -1$. Although our construction internally programs pairwise subtractive shares, the joint distribution (R_0, R_1, R_2) is supported on permutations (a non-linear relation). In particular, it is not an additive sharing over $\mathbb{F}_3$. See the general template in 2.2 for the pairwise-distinctness argument.

Theorem 9. *Let $\text{HSS} = (\text{Setup}, \text{Input}, \text{Eval})$ be an HSS with simulatable memory shares (with algorithms $\text{Sim}_0, \text{Sim}_1$) and with input space $\mathcal{I} = \mathbb{F}_3$ and share space $\mathcal{S} = \mathbb{F}_3$. Moreover, let F be a pseudorandom function and for $x \in \{0, 1\}^k$, let P_x be the HSS program that runs $F.\text{Eval}(\cdot, x)$ when given the PRF key as input. Then, PCF of Figure 1 is a PCF for the 3-correlation generator $\mathcal{C}$ given by Definition 31, where $\mathcal{D}$ is the uniform distribution over S_3 and $m \in \mathbb{N}$ is the output length of F . (For simplicity of notation, we identify S_3 as the set of permutations over the elements of $\mathbb{F}_3$.)*

Proof. We show each property in turn.

Correctness. Let (k_0, k_1, k_2) as output by $\text{PCF.Gen}(1^\lambda)$. Let k_i be parsed as $(\text{pk}, \llbracket \kappa \rrbracket_i, \text{ek}_i, M^i)$ for $i \in \{0, 1, 2\}$. Moreover, let $R_i \leftarrow \text{PCF.Eval}(i, k_i, x)$, for some $x \in \{0, 1\}^k$. More concretely,

$$\begin{aligned} R_0 &= \text{ExtEval}(0, \text{ek}^A, M^A, \llbracket \kappa \rrbracket^A, P_x) \\ R'_1 &= \text{ExtEval}(1, \text{ek}^B, M^B, \llbracket \kappa \rrbracket^B, P_x) \\ R'_2 &= \text{ExtEval}(2, \text{ek}^B, M^C, \llbracket \kappa \rrbracket^B, P_x) - 1 \\ R_1 &= 2 + R'_1 \\ R_2 &= 2 + R'_2. \end{aligned}$$

We need to show that $(R_0, R_1, R_2) \in \mathbb{F}_3^3$ as obtained in this manner, is a uniformly random permutation over the elements of $\mathbb{F}_3$.

By the simulation correctness of the extended evaluation, we know that by construction, with overwhelming probability, $R_0 - R'_1 = F.\text{Eval}(\kappa, x) \in \{0, 1\}$, and $R_0 - R'_2 = 1 - F.\text{Eval}(\kappa, x) \in \{0, 1\}$. Also, because the distinguisher only gets the 3-tuple, by the security of the HSS scheme, they cannot learn anything about κ , and hence, $F.\text{Eval}(\kappa, x)$ is computationally indistinguishable from a random bit. Then, using $R'_1 = R_0 - \mu$, and $R'_2 = R_0 + \mu - 1$, for $\mu := F.\text{Eval}(\kappa, x)$, we get $R_0 + R_1 + R_2 = R_0 + (2 + R'_1) + (2 + R'_2) = R_0 + 2 + (R_0 - \mu) + (R_0 + 1 + \mu) = 3R_0 + 2 - \mu + 1 + \mu = 0 \pmod{3}$, i.e., the R 's form an additive sharing of zero, as required.

Next, we show that they are pairwise distinct. This is because $R_0 = R_1$ implies $R_0 - R'_1 = \mu = 2$, which is false. (The other cases are implied and/or similar.) Hence, (R_0, R_1, R_2) forms a permutation. It remains to show that it is uniformly generated. As R_0 is a fresh random subtractive share, it follows if the HSS satisfies the additional property that the marginal distribution of an output share is computationally close to random. While this does not follow automatically from the HSS definition, it is folklore that any HSS can be trivially modified to satisfy this slightly stronger property by letting HSS.Input additionally output a fresh PRF key (for a PRF with range $\mathbb{F}_3$) in both input shares, and letting HSS.Eval sum the PRF output (evaluated on the RMS program viewed as a string) to its output. This preserves correctness (the PRF evaluations cancel out when subtracting the shares) and guarantees that each share is individually pseudorandom. With this change, R_0 is computationally close to uniform over $\mathbb{F}_3$. Then, given this R_0 , we show that the two possibilities for R_1 are exactly determined by the pseudorandom bit of the PRF, so it is indistinguishable from being uniformly chosen from $\mathbb{F}_3 \setminus \{R_0\}$.

Security. Here, we show PCF security for each $\sigma \in \{0, 1, 2\}$. Note that the proof is different for the different σ 's, because for $\sigma = 0$, the memory share M^A that is part of the key does not need to be simulated. We prove the case where $\sigma = 0$. The other cases follow via an analogous proof.

Hybrid $\mathcal{H}_0$. This hybrid consists of $\text{Exp}_{\mathcal{A}, N, \sigma, 1}^{\text{sec}}(\lambda)$ game as defined in Figure 4.

Hybrid $\mathcal{H}_1$. In this hybrid, we change how the output $R_1^{(i)}$ of Bob is computed (for all $i \in [N]$). First, we reconstruct the PRF key κ from the HSS input shares contained in the PCF keys and then evaluate the PRF on input x_i to get the output μ_i . Then, we use Alice's output $R_0^{(i)}$ to compute Bob's share $R_1^{(i)}$ as $R_0^{(i)} + 2 - \mu_i$.

We note that $\mathcal{H}_1$ is identical to $\mathcal{H}_0$, since the change is merely syntactic. Now, in $\mathcal{H}_1$, Bob's share is no longer computed using k_1 .

Hybrid $\mathcal{H}_2$. In this hybrid, we change how the output of Carol is computed. As before, we first reconstruct the PRF key κ from the HSS input shares contained in the PCF keys and then evaluate the PRF on input x_i to get the output μ_i . Then, we use Alice's output $R_0^{(i)}$ to compute Carol's share $R_1^{(i)}$ as $R_0^{(i)} + 1 + \mu_i$.

We note that $\mathcal{H}_2$ is again identical to $\mathcal{H}_1$, since the change is merely syntactic. Now, in $\mathcal{H}_2$, Carol's share is no longer computed using k_2 .

Hybrid $\mathcal{H}_3$. In this hybrid, we replace the HSS input shares of κ with an input share of 0.

$\mathcal{H}_3 \approx_c \mathcal{H}_2$ by the security of HSS.

Hybrid $\mathcal{H}_4$. In this hybrid, we replace the pseudorandom bit μ_i (as output in $\mathcal{H}_3$ using the PRF) with a uniformly random bit.

$\mathcal{H}_4 \approx_c \mathcal{H}_3$ by the pseudorandomness of the PRF.

Hybrid $\mathcal{H}_5$. This hybrid consists of $\text{Exp}_{\mathcal{A},N,\sigma,0}^{\text{sec}}(\lambda)$ game as defined in Figure 4.

$\mathcal{H}_5$ is identical to $\mathcal{H}_4$ given that in $\mathcal{H}_4$, we already sample $R_1^{(i)}$ and $R_2^{(i)}$ entirely from Alice's output $R_0^{(i)}$. ■

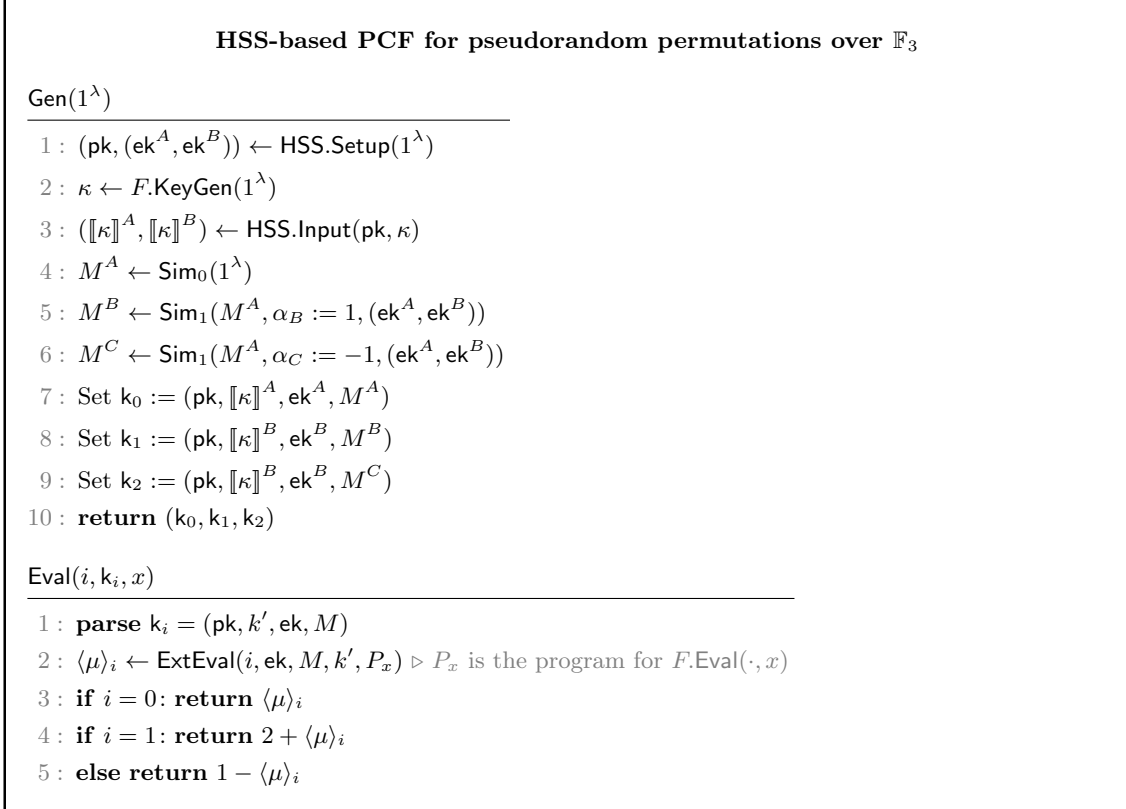


Fig. 1: HSS-based PCF construction

Concrete Performance. Although this construction makes non-black-box use of a PRF, with some care, it can still be reasonably efficient for practical applications. In Appendix B.7, we show that by combining HSS schemes with a suitable, low-complexity weak PRF, we can plausibly obtain constructions that allow expanding short seeds to produce up to 2^{40} pseudorandom permutation correlations, where the PPF for one party's output can be evaluated in under 100ms.

Extension to $\binom{n}{1}$ -HSS. As mentioned in Section 2.2, we can use the same idea in our PPF construction to build an n -party HSS scheme with Shamir-shared outputs, and threshold $t = 1$. The construction supports evaluation of the same class of functions as the underlying 2-party HSS, so when instantiating this under LWE, DCR or class group assumptions, we obtain $\binom{n}{1}$ -HSS for RMS programs (including all of NC1).

The construction, shown in Figure 2, begins with a setup phase essentially the same as the KeyGen algorithm from the PCF, where each party is given a simulated, two-party HSS memory

share that forms a sharing of its evaluation point i , when combined with party 0's share. Then, the HSS input algorithm and evaluation algorithms are essentially the same as the two-party case, except we use the **ExtEval** algorithm, which ensures that output shares are adjusted to multiply the plaintext by the relevant point i .

Theorem 10. *Let $\text{HSS}_2 = (\text{HSS}_2.\text{Setup}, \text{HSS}_2.\text{Input}, \text{HSS}_2.\text{Eval})$ be a two-party HSS with simulatable memory shares (with algorithms $\text{HSS}_2.\text{Sim}_0, \text{HSS}_2.\text{Sim}_1$) and with input space $\mathcal{I}$ and share space $\mathcal{S} = \mathbb{F}_p$ for some prime $p \geq n$. Then, HSS of Figure 2 is a $(1, n)$ -HSS scheme that supports evaluating the same class of programs as HSS_2 , and features Shamir-shared outputs from **Eval**.*

Proof. We prove each property in turn.

Correctness. Let

$$\begin{aligned} y_A &\leftarrow \text{HSS}_2.\text{ExtEval}(0, \text{ek}^A, M^0, \llbracket y_j \rrbracket^A), P) \\ y_B &\leftarrow \text{HSS}_2.\text{ExtEval}(1, \text{ek}^B, M^i, \llbracket y_j \rrbracket^i), P). \end{aligned}$$

Correctness of HSS_2 implies $y_A + y_B = P(x_1, \dots, x_m)$ with probability $1 - \text{negl}(\lambda)$.

In our scheme each party i receives $\langle y \rangle^i = y_A + \alpha_i y_B$, where $\alpha_0 = 0$ and $\alpha_i = i$ for $1 \leq i \leq n-1$. Hence the shares are evaluations of the degree-1 polynomial $p(z) = y_A + y_B z$. Any two distinct indices $T = \{i, j\}$ allow interpolation of p at 0:

$$\text{Rec}_T(\langle y \rangle^i, \langle y \rangle^j) = y_A + y_B.$$

This equals $P(x_1, \dots, x_m)$ except with negligible error.

Security (sketch). We have the following hybrids:

Hybrid $\mathcal{H}_0$. Real execution.

Hybrid $\mathcal{H}_1$. Same as $\mathcal{H}_0$ but replace the memory state M^{i^*} inside ek_{i^*} using $\text{HSS}_2.\text{Sim}_1$. Indistinguishable by the simulatable-memory property of HSS_2 .

Hybrid $\mathcal{H}_2$. Replace the corrupted share $\langle y \rangle^{i^*} = y_A + \alpha_{i^*} y_B$ with a uniform element of $\mathbb{F}$. Because y_B is uniform from the point of view of a single party, the real and modified shares are identically distributed thus $\mathcal{H}_1 \equiv \mathcal{H}_2$.

Simulator. Sim outputs the values of $\mathcal{H}_2$. ■

5 Multiparty PCG for Correlations with Threshold Secret Sharings

In this section, we construct multiparty PCGs for constant-degree correlations with threshold secret sharings from multiparty DPF with additive sharings.

Notations. $\binom{[n]}{t}$ denotes the collection of all subsets in $[n]$ of size t . $\binom{[n]}{\leq t}$ denotes the collection of all subsets in $[n]$ of size at most t . $\binom{[n]}{\geq t}$ denotes the collection of all subsets in $[n]$ of size at least t .

5.1 Threshold secret sharing schemes

In this work, we consider secret sharing schemes over a finite field $\mathbb{F}$ defined below following the definitions in [54].

Definition 11 (Secret sharing scheme). *The syntax of a secret sharing scheme Σ over a finite field $\mathbb{F}$ consists of the following data:*

- A set of parties $\mathcal{I} = \{1, \dots, n\}$.
- A secret space $Z = \mathbb{F}^k$.
- A share space $U = \mathbb{F}^\ell$.
- A sharing space $C \subseteq U^{\mathcal{I}}$, where $U^{\mathcal{I}}$ denotes the indexed Cartesian product $\prod_{i \in \mathcal{I}} U$.

$\binom{n}{1}$ -HSS Scheme for Shamir shared outputs

Setup(1^λ)

```

1 :  $(\text{pk}, (\text{ek}^A, \text{ek}^B)) \leftarrow \text{HSS}_2.\text{Setup}(1^\lambda)$ 
2 :  $M^0 \leftarrow \text{HSS}_2.\text{Sim}_0(1^\lambda)$ 
3 :  $M^i \leftarrow \text{HSS}_2.\text{Sim}_1(M^0, \alpha_i := i, (\text{ek}^A, \text{ek}^B))$ , for  $i \in \{1, \dots, n-1\}$ 
4 :  $\text{ek}_0 := (\text{pk}, \text{ek}^A)$ 
5 :  $\text{ek}_i := (\text{pk}, \text{ek}^B)$ , for  $i \in \{1, \dots, n-1\}$ 
6 : return  $(\text{pk}, (\text{ek}_0, \dots, \text{ek}_{n-1}))$ 

```

Input($1^\lambda, x$)

```

1 :  $(\llbracket x \rrbracket^A, \llbracket x \rrbracket^B) \leftarrow \text{HSS}_2.\text{Input}(1^\lambda, x)$ 
2 : return  $(\llbracket x \rrbracket^A, \llbracket x \rrbracket^B, \dots, \llbracket x \rrbracket^B)$ 

```

Eval($i, \text{ek}_i, (\llbracket y_j \rrbracket^i)_{j=1}^n, P$)

```

1 : parse  $\text{ek}_i = (\text{pk}, \text{ek}', M^i)$ 
2 : return  $\text{HSS}_2.\text{ExtEval}(i, \text{ek}', M^i, (\llbracket y_j \rrbracket^i)_j, P)$ 

```

Rec_T($(\llbracket y \rrbracket^i)_{i \in T}$)

```

1 : Let  $\lambda_i$  be degree-1, Lagrange interpolation coefficients in  $\mathbb{F}_p$  for the points  $T$ 
2 : return  $\sum_{i \in T} \lambda_i \cdot \llbracket y \rrbracket^i$ 

```

Fig. 2: $\binom{n}{1}$ -HSS based on a 2-party HSS scheme with simulatable memory shares, given by algorithms ($\text{HSS}_2.\text{Setup}$, $\text{HSS}_2.\text{Input}$, $\text{HSS}_2.\text{Eval}$, $\text{HSS}_2.\text{Sim}$)

- An injective sharing map: $\text{share} : Z \times \mathbb{F}^r \rightarrow C$, which maps a secret $\mathbf{x} \in Z$ and a random tape $\rho \in \mathbb{F}^r$, to a sharing $\mathbf{X} \in C$.
- A surjective reconstruction map: $\text{rec} : C \rightarrow Z$, which takes as input a sharing $\mathbf{X} \in C$ and outputs a secret $\mathbf{x} \in Z$.

The scheme Σ satisfies that for all $\mathbf{x} \in Z$ and $\rho \in \mathbb{F}^r$, $\text{rec}(\text{share}(\mathbf{x}, \rho)) = \mathbf{x}$.

Then considering a set of shares with indices in A . A is defined to be a privacy set if the set of shares with indices in A reveals no information about the secret and is defined to be a reconstruction set if they fully determine the secret.

Definition 12 (Privacy set and Reconstruction set). Suppose $A \subset \mathcal{I}$ is nonempty. We say A is a privacy set if for all $\mathbf{x}_0, \mathbf{x}_1 \in Z$, and for all vector $\mathbf{v} \in U^A$,

$$\Pr_{\rho}[\pi_A(\text{share}(\mathbf{x}_0, \rho)) = \mathbf{v}] = \Pr_{\rho}[\pi_A(\text{share}(\mathbf{x}_1, \rho)) = \mathbf{v}].$$

We say A is a reconstruction set if there is a map $\text{rec}_A : \pi_A(C) \rightarrow Z$ such that for all $\mathbf{X} \in C$,

$$\text{rec}_A(\pi_A(\mathbf{X})) = \text{rec}(\mathbf{X}).$$

We will focus on threshold secret sharing schemes. For a t -threshold secret sharing scheme, for all $A \in \binom{\mathcal{I}}{t}$, A is a privacy set.

In Shamir secret sharing scheme of degree t , to share a secret $s \in \mathbb{F}$, the sharing map $\text{share} : \mathbb{F} \times \mathbb{F}^t \rightarrow \mathbb{F}^n$ first samples a random polynomial $p(x)$ of degree t over $\mathbb{F}$ such that $p(0) = s$. Then each party P_i takes the evaluation point $p(i)$ as its share s_i . Since the polynomial is of degree t , the reconstruction map takes as input any $t + 1$ shares, interpolates to obtain a degree- t polynomial, and recovers the secret s . As any t shares reveal no information about the secret s , this is a t -threshold secret sharing scheme. The whole sharing $(s_1, \dots, s_n)$ is denoted by $[s]_t^n$. For a vector of Shamir secret sharings, we use the notation $[\mathbf{r}]_t := ([r_1]_t, \dots, [r_m]_t)$, where $\mathbf{r} = (r_1, \dots, r_m)$. Moreover, Shamir secret sharing is linearly homomorphic, i.e., $[x_1]_t + [x_2]_t = [x_1 + x_2]_t$.

To show Shamir secret sharing correlation is reverse-sampleable, we define the following procedure which samples a random degree- t Shamir secret sharing given a secret $r \in \mathbb{F}$ and a set of shares $\{r_j\}_{j \in T}$ for some $T \in \binom{[n]}{\leq t}$.

- $\text{RandShamir}(r, t, T, \{r_j\}_{j \in T})$: The algorithm receives a secret $r \in \mathbb{F}$, a degree parameter t , and a set of values $\{r_j\}_{j \in T}$ for some $T \in \binom{[n]}{\leq t}$. Then it samples a random degree- t polynomial condition on $p(0) = r$ and $p(j) = r_j$ for each $j \in T$ ¹⁰ and defines $r_j := p(j)$ for each $j \in [n] \setminus T$ as well as $[r]_t := (r_1, \dots, r_n)$. Finally, it outputs $(r_j)_{j \in [n] \setminus T}$.

5.2 Multiparty DPF, PCG, HSS with threshold secret sharing Scheme

To deal with threshold adversary, we define multiparty DPF scheme with the truth table of point functions shared by threshold secret sharing schemes, where correctness is defined with respect to the threshold secret sharing scheme and security is against a threshold adversary.

Definition 13 ($\binom{[n]}{t}$ -DPF). An $\binom{[n]}{t}$ -DPF with t -threshold secret sharing scheme Σ for point functions with input domain $[D]$ and output domain a finite field $\mathbb{F}$, is a pair of PPT algorithms $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$ with the following syntax:

- $\text{DPF.Gen}(1^\lambda, \alpha, \beta)$, given security parameter λ and description of a point function $f_{\alpha, \beta}$, outputs n keys $(k_1, \dots, k_n)$;
- $\text{DPF.Eval}(\sigma, k_\sigma, x)$, given party index $\sigma \in [n]$, key k_σ , and input $x \in [D]$, outputs a field element $y_\sigma \in \mathbb{F}$.

The scheme DPF should satisfy the following requirements:

¹⁰ In particular, RandShamir chooses $m := t - |T|$ parties, $P_{i_1}, \dots, P_{i_m}$, that are with m smallest indices and are not in T , samples a random element as r_{i_j} for each $j \in [m]$, and interpolates to get a degree- t polynomial p such that $p(0) = s$ and $p(i) = r_i$ for each $i \in T \cup \{i_1, \dots, i_m\}$.

- **Correctness.** For any $(\alpha, \beta) \in [D] \times \mathbb{F}$ and $x \in [D]$, we have with probability 1 over the randomness of $\text{DPF.Gen}(1^\lambda, \alpha, \beta)$, letting $y_\sigma \leftarrow \text{DPF.Eval}(\sigma, k_\sigma, x)$ for each $\sigma \in [n]$, $(y_1, \dots, y_n)$ forms a valid Σ -sharing with secret $f_{\alpha, \beta}(x)$.
- **Security.** For every set of corrupted parties $T \in \binom{[n]}{\leq t}$, there exists a PPT simulator Sim such that for any family $\mathcal{F} = \{f_{\alpha, \beta} : [D(\lambda)] \rightarrow \mathbb{F}_\lambda\}_{\lambda \in \mathbb{N}}$ of point functions over domain $[D(\lambda)]$ and field $\mathbb{F}_\lambda$, the distributions

$$\{(k_\sigma)_{\sigma \in T} \mid (k_1, \dots, k_n) \leftarrow \text{DPF.Gen}(1^\lambda, \alpha_\lambda, \beta_\lambda)\}$$

and

$$\{(k_\sigma)_{\sigma \in T} \mid (k_\sigma)_{\sigma \in T} \leftarrow \text{Sim}(1^\lambda, D(\lambda), \mathbb{F}_\lambda)\}$$

are computationally indistinguishable.

Notice that an n -party DPF with additive sharing is an $\binom{n}{n-1}$ -DPF with $(n-1)$ -threshold secret sharing.

The following property is useful to argue the correctness of a PCG compiled from a DPF.

Definition 14 ($\binom{n}{t}$ -DPF with pseudorandom outputs). We say that an $\binom{n}{t}$ -DPF scheme $(\text{DPF.Gen}, \text{DPF.Eval})$ with t -threshold secret sharing scheme Σ for family $\mathcal{F} = \{f_{\alpha, \beta} : [D] \rightarrow \mathbb{F}\}$ of point functions satisfies pseudorandomness of outputs, if for all $f \in \mathcal{F}$, $x \in [D]$, and subset of keys $T \in \binom{[n]}{\leq t}$, the following distributions

$$\left\{ (k_\sigma)_{\sigma \in T}, (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta) \\ y_{\tilde{\sigma}} \leftarrow \text{DPF.Eval}(\tilde{\sigma}, k_{\tilde{\sigma}}, x) \text{ for } \tilde{\sigma} \in [n] \setminus T \end{array} \right. \right\}$$

and

$$\left\{ (k_\sigma)_{\sigma \in T}, (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta) \\ y_\sigma \leftarrow \text{DPF.Eval}(\sigma, k_\sigma, x) \text{ for } \sigma \in T \\ (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \leftarrow \text{RandSharing}(f_{\alpha, \beta}(x), t, T, \{y_\sigma\}_{\sigma \in T}) \end{array} \right. \right\}$$

are computationally indistinguishable, where RandSharing is the reverse sampling algorithm for Σ -sharing scheme.

For the sake of completeness, we also formalize the definitions of PCG and HSS when considering threshold secret sharing schemes and threshold adversary as follows.

Definition 15 ($\binom{n}{t}$ -PCG). Let $\mathcal{C}$ be a reverse-sampleable correlation generator. An $\binom{n}{t}$ -PCG for $\mathcal{C}$ is a pair of algorithms $(\text{PCG.Gen}, \text{PCG.Expand})$ with the following syntax:

- $\text{PCG.Gen}(1^\lambda)$ is a PPT algorithm that given a security parameter λ , outputs n seeds $(k_1, \dots, k_n)$.
- $\text{PCG.Expand}(\sigma, k_\sigma)$ is a polynomial-time algorithm that given party index $\sigma \in [n]$ and a seed k_σ , outputs a bit string $R_\sigma \in \{0, 1\}^I$.

The algorithms $(\text{PCG.Gen}, \text{PCG.Expand})$ should satisfy the following:

- **Correctness.** The correlation obtained via:

$$\left\{ (R_1, \dots, R_n) \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \text{PCG.Gen}(1^\lambda) \\ R_\sigma \leftarrow \text{PCG.Expand}(\sigma, k_\sigma) \text{ for } \sigma \in [n] \end{array} \right. \right\}$$

is computationally indistinguishable from $\mathcal{C}(1^\lambda)$.

- **Security.** For any $T \in \binom{[n]}{\leq t}$, the following two distributions

$$\left\{ (k_\sigma)_{\sigma \in T}, (R_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \text{PCG.Gen}(1^\lambda) \\ R_{\tilde{\sigma}} \leftarrow \text{PCG.Expand}(\tilde{\sigma}, k_{\tilde{\sigma}}) \text{ for } \tilde{\sigma} \in [n] \setminus T \end{array} \right. \right\}$$

and

$$\left\{ (k_\sigma)_{\sigma \in T}, (R_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \text{PCG.Gen}(1^\lambda) \\ R_\sigma \leftarrow \text{PCG.Expand}(\sigma, k_\sigma) \text{ for } \sigma \in T \\ (R_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \leftarrow \text{RSampler}(T, \{R_\sigma\}_{\sigma \in T}) \end{array} \right. \right\}$$

are computationally indistinguishable, where RSampler is the reverse sampling algorithm for $\mathcal{C}$.

Definition 16 ($\binom{n}{t}$ -HSS). An $\binom{n}{t}$ -HSS with t -threshold secret sharing scheme Σ for function family $\mathcal{F}$ over a finite field $\mathbb{F}$ is a pair of algorithms $\text{HSS} = (\text{HSS.Share}, \text{HSS.Eval})$ with the following syntax and properties:

- **Share**($1^\lambda, x$): On input 1^λ (the security parameter) and $x \in \mathbb{F}$ (the input), the sharing algorithm **Share** outputs n inputs shares $(x^{(1)}, \dots, x^{(n)})$.
- **Eval**($\sigma, f, x^{(\sigma)}$): On input $i \in [n]$ (the party index), $f \in \mathcal{F}$ (the function to be homomorphically evaluated, implicitly assumed to specify input and output length I, O), and $x^{(\sigma)}$ (the σ -th input share), the evaluation algorithm **Eval** outputs the σ -th output share $y^{(\sigma)} \in \mathbb{F}^O$.
- **Correctness.** For any 1^λ , input $x \in \mathbb{F}^{I(\lambda)}$, and any function $f \in \mathcal{F}$, letting

$$(x^{(1)}, \dots, x^{(n)}) \leftarrow \text{HSS.Share}(1^\lambda, x) \\ y^{(\sigma)} \leftarrow \text{HSS.Eval}(\sigma, f, x^{(\sigma)}) \text{ for } \sigma \in [n],$$

then $(y^{(1)}, \dots, y^{(n)})$ forms valid Σ -sharings of $f(x)$.

- **Security.** For any set of corrupted parties $T \in \binom{[n]}{\leq t}$, there exists a PPT algorithm **Sim**, such that for every sequence of inputs $x_1, x_2, \dots \in \mathbb{F}^{I(\lambda)}$, the following distributions

$$\left\{ (x^{(\sigma)})_{\sigma \in T} \mid (x^{(1)}, \dots, x^{(n)}) \leftarrow \text{HSS.Share}(1^\lambda, x) \right\}$$

and

$$\left\{ (x^{(\sigma)})_{\sigma \in T} \mid (x^{(\sigma)})_{\sigma \in T} \leftarrow \text{Sim}(1^\lambda, n, I, O) \right\}$$

are computationally indistinguishable.

Definition 17 ($\binom{n}{t}$ -HSS with pseudorandom outputs). We say that an $\binom{n}{t}$ -HSS scheme $(\text{HSS.Share}, \text{HSS.Eval})$ with t -threshold secret sharing scheme Σ for function family $\mathcal{F}$ over a finite field $\mathbb{F}$ satisfies pseudorandomness of outputs, if for all $f \in \mathcal{F}, x \in \mathbb{F}^{I(\lambda)}$, and subset of shares $T \in \binom{[n]}{\leq t}$, the following distributions

$$\left\{ (x^{(\sigma)})_{\sigma \in T}, (y^{(\tilde{\sigma})})_{\tilde{\sigma} \in [n] \setminus T} \mid \begin{array}{l} (x^{(1)}, \dots, x^{(n)}) \leftarrow \text{HSS.Share}(1^\lambda, x) \\ y^{(\tilde{\sigma})} \leftarrow \text{HSS.Eval}(\tilde{\sigma}, f, x^{(\tilde{\sigma})}) \text{ for } \tilde{\sigma} \in [n] \setminus T \end{array} \right\}$$

and

$$\left\{ (x^{(\sigma)})_{\sigma \in T}, (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \mid \begin{array}{l} (x^{(1)}, \dots, x^{(n)}) \leftarrow \text{HSS.Share}(1^\lambda, x) \\ y_{\sigma} \leftarrow \text{HSS.Eval}(\sigma, f, x^{(\sigma)}) \text{ for } \sigma \in T \\ (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \leftarrow \text{RandSharing}(f(x), t, T, \{y^{(\sigma)}\}_{\sigma \in T}) \end{array} \right\}$$

are computationally indistinguishable, where **RandSharing** is the reverse sampling algorithm for Σ -sharing scheme.

5.3 $\binom{n}{t}$ -PCG for constant-degree polynomials from additive DPF: constant n and t

We first recall local share conversion from replicated secret sharing scheme to Shamir secret sharing scheme [42]. To share a secret $s \in \mathbb{F}$ with replicated secret sharing, we first additively and randomly decompose s into $\binom{n}{t}$ pieces, where each piece s_T is labeled with a subset $T \in \binom{[n]}{t}$, i.e., $s = \sum_{T \in \binom{[n]}{t}} s_T$. Then we assign each s_T to parties that are not in T . Now for each party P_i , it will obtain a set of shares $\{s_T\}_{T \in \binom{[n]}{t}: i \notin T}$.

To transform from replicated secret sharing of s to degree- t Shamir sharing of s , for each subset $T \in \binom{[n]}{t}$, all parties define a degree- t polynomial $f_T : \mathbb{F} \rightarrow \mathbb{F}$ such that

$$f_T(0) = 1 \text{ and } \forall i \in T, f_T(i) = 0.$$

Let $f : \mathbb{F} \rightarrow \mathbb{F}$ be a degree- t polynomial such that $f = \sum_{T \in \binom{[n]}{t}} s_T \cdot f_T$. Then $f(0) = \sum_{T \in \binom{[n]}{t}} s_T \cdot f_T(0) = \sum_{T \in \binom{[n]}{t}} s_T = s$. Observe that each party P_i can locally compute

$$f(i) = \sum_{T \in \binom{[n]}{t}} s_T \cdot f_T(i) = \sum_{T \in \binom{[n]}{t}: i \notin T} s_T \cdot f_T(i),$$

as $f_T(i) = 0$ when $i \in T$ and s_T is known to P_i for each $T \in \binom{[n]}{t}$ satisfying $i \notin T$.

As for the privacy, for any subset of parties $\mathcal{C} \in \binom{[n]}{\leq t}$, there exists $T_0 \in \binom{[n]}{t}$ such that $\mathcal{C} \subseteq T_0$. Then s_{T_0} and thus s are unknown to parties in $\mathcal{C}$. As for reconstruction property, considering any subset of parties $\mathcal{I} \in \binom{[n]}{\geq t+1}$, for every $T \in \binom{[n]}{t}$, $\mathcal{I} - T \neq \emptyset$ and thus s_T is known to parties in $\mathcal{I}$. Hence, $s = \sum_{T \in \binom{[n]}{t}} s_T$ can be computed by parties in $\mathcal{I}$.

In short, if we can additively share the secret, redundancy can be added by traversing all access structures. Based on this recipe, instead of holding a replicated secret sharing of a single secret s , all parties hold replicated sharing of a point function and are expected to compute their degree- t Shamir shares of the truth table corresponding to the point function. More specifically, starting with an $\binom{n}{t}$ -party additive DPF which distributes a point function $f_{\alpha,\beta}$ using short keys $\{k_T\}_{T \in \binom{[n]}{t}}$, for each subset $T \in \binom{[n]}{t}$, key k_T is assigned to party P_i satisfying $i \notin T$. By evaluating their DPF keys at hand, all parties can obtain replicated secret sharings and hence degree- t Shamir sharings of the truth table of $f_{\alpha,\beta}$ following the share conversion template described above, which results a construction of $\binom{n}{t}$ -DPF from $\binom{n}{t}$ -party additive DPF. Intuitively, the security stems from the security of $\binom{n}{t}$ -party additive DPF and the privacy of replicated secret sharing scheme.

Construction 1: $\binom{n}{t}$ -DPF from $\binom{n}{t}$ -party additive DPF

Let $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$ be an $\binom{n}{t}$ -party additive DPF for family of point functions $\mathcal{F} = \{f_{\alpha,\beta} : [D] \rightarrow \mathbb{F}\}$.

$\binom{n}{t}$ -DPF.Gen(1^λ):

1. $\text{DPF.Gen}(1^\lambda, \alpha, \beta) \rightarrow (K_T)_{T \in \binom{[n]}{t}}$.
2. For each $i \in [n]$, assign DPF keys $k_i \leftarrow \{K_T\}_{T \in \binom{[n]}{t}: i \notin T}$.
3. Output $(k_1, \dots, k_n)$.

$\binom{n}{t}$ -DPF.Eval(i, k_i, x):

1. Parse k_i as $\{K_T\}_{T \in \binom{[n]}{t}: i \notin T}$.
2. For each $T \in \{S \in \binom{[n]}{t} : i \notin S\}$, compute $Y_T \leftarrow \text{DPF.Eval}(T, K_T, x)$.
3. Compute its degree- t Shamir share as $y_i \leftarrow \sum_{T \in \binom{[n]}{t}: i \notin T} Y_T \cdot f_T(i)$ and output y_i .

$\binom{n}{t}$ -DPF from $\binom{n}{t}$ -party additive DPF

$\binom{n}{t}$ -DPF.Gen(1^λ)

- 1 : $(K_T)_{T \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta)$
- 2 : $k_i \leftarrow \{K_T\}_{T \in \binom{[n]}{t}: i \notin T}$ for $i \in \{1, \dots, n\}$
- 3 : **return** $(k_1, \dots, k_n)$

$\binom{n}{t}$ -DPF.Eval(i, k_i, x)

- 1 : **parse** $k_i = \{K_T\}_{T \in \binom{[n]}{t}: i \notin T}$
- 2 : $Y_T \leftarrow \text{DPF.Eval}(T, K_T, x)$ for $T \in \{S \in \binom{[n]}{t} : i \notin S\}$
- 3 : $y_i \leftarrow \sum_{T \in \binom{[n]}{t}: i \notin T} Y_T \cdot f_T(i)$
- 4 : **return** y_i

Lemma 18 (Imported lemma [40]: multiparty DPF with additive sharing). *Let $m, d > 0$ be any constant integers. Assume MQ and the dual $\mathbb{F}$ -LPN assumptions, there exists a secure, m -party DPF with additive sharings $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$ that supports the family of point functions $f_{\alpha, \beta} : [\lambda^d] \rightarrow \mathbb{F}$ with the following characteristics:*

- DPF.Gen generates keys of size $\ell = O(\lambda^3)$.
- DPF.Gen can be computed using $O(\lambda^2 + \text{poly}(\lambda))$ Boolean operations (where the exponent in poly is independent of d, m).
- For any input $x \in [\lambda^d]$, $\text{DPF.Eval}(\cdot, x)$ can be computed by a degree- d' polynomial, where d' is a constant.

Lemma 19 ($\binom{n}{t}$ -DPF: constant n and t). *Let n and t such that $t < n$ be two integers. Let $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$ be an $\binom{n}{t}$ -party additive DPF with pseudorandom outputs as per Definition 14 supporting the family of point functions $f_{\alpha, \beta} : [\lambda^d] \rightarrow \mathbb{F}$ such that*

- DPF.Gen generates keys of size ℓ ,
- for any $x \in [\lambda^d]$, $\text{DPF.Eval}(\cdot, x)$ is computable by degree- d' polynomials.

Then the construction in $\binom{n}{t}$ -DPF is a secure $\binom{n}{t}$ -DPF with degree- t Shamir secret sharing with pseudorandom outputs that supports the same family of the point functions with the following characteristics:

- $\binom{n}{t}$ -DPF.Gen generates keys of size $\binom{n-1}{t} \cdot \ell$.
- $\binom{n}{t}$ -DPF.Gen has the same asymptotic computation complexity as that of DPF.Gen .
- For any input $x \in [\lambda^d]$, $\binom{n}{t}$ -DPF.Eval($\cdot, x$) can be computed by a degree- d' polynomial.

Proof of Lemma 19. The correctness stems from the correctness of $\binom{n}{t}$ -party additive scheme $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$. By invoking $(k_1, \dots, k_n) \leftarrow \binom{n}{t}\text{-DPF.Gen}(1^\lambda, \alpha, \beta)$ and $y_i \leftarrow \binom{n}{t}\text{-DPF.Eval}(i, k_i, x)$ for each $i \in [n]$, notice that $(y_1, \dots, y_n)$ forms a degree- t Shamir secret sharing with secret $\sum_{T \in \binom{[n]}{t}} Y_T \cdot f_T(0) = \sum_{T \in \binom{[n]}{t}} Y_T = \sum_{T \in \binom{[n]}{t}} \text{DPF.Eval}(K_T, x) = f_{\alpha, \beta}(x)$.

The security also inherits from that of $\binom{n}{t}$ -party additive DPF. For any subset $T \in \binom{[n]}{\leq t}$ of corrupted parties, there exists $R \in \binom{[n]}{t}$ such that $T \subseteq R$, which implies the key K_R labeled by R is not included in the set of $\binom{n}{t}$ -DPF keys $\{k_i\}_{i \in T}$ that are known to the corrupted parties. Hence, by security of $\binom{n}{t}$ -party additive DPF, there exists a PPT simulator Sim' such that for any point function $f_{\alpha, \beta}$ over domain $[D]$ and finite field $\mathbb{F}$, the distributions

$$\{(K_S)_{S \in \binom{[n]}{t}: S \not\subseteq R} \mid (K_S)_{S \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta)\}$$

and

$$\{(K_S)_{S \in \binom{[n]}{t}: S \not\subseteq R} \mid (K_S)_{S \in \binom{[n]}{t}: S \not\subseteq R} \leftarrow \text{Sim}(1^\lambda, D, \mathbb{F})\}$$

are computationally indistinguishable. Then a PPT simulator Sim can be constructed based on Sim' by first invoking Sim' to generate the keys $\{K'_S\}_{S \in \binom{[n]}{t}: S \not\subseteq R}$ and then assigning the additive DPF keys to each party P_σ in T by setting $k'_\sigma \leftarrow \{K'_S\}_{S \in \binom{[n]}{t}: \sigma \notin S}$, which is feasible since the key K_R of the $\binom{[n]}{t}$ -party additive DPF is absent from the corrupted parties' view. The effectiveness of Sim holds as the distributions

$$\{(k_\sigma)_{\sigma \in T} \mid (k_1, \dots, k_n) \leftarrow \binom{n}{t}\text{-DPF.Gen}(1^\lambda, \alpha, \beta)\}$$

and

$$\{(k'_\sigma)_{\sigma \in T} \mid (k'_\sigma)_{\sigma \in T} \leftarrow \text{Sim}'(1^\lambda, D, \mathbb{F})\}$$

are computationally indistinguishable.

As for pseudorandomness of the outputs, for any subset $T \in \binom{[n]}{\leq t}$ of parties, by definition, our goal is to prove the following two distributions

$$\left\{ (k_\sigma)_{\sigma \in T}, (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \mid \begin{array}{l} (k_1, \dots, k_n) \leftarrow \binom{n}{t}\text{-DPF.Gen}(1^\lambda, \alpha, \beta) \\ y_{\tilde{\sigma}} \leftarrow \binom{n}{t}\text{-DPF.Eval}(\tilde{\sigma}, k_{\tilde{\sigma}}, x) \text{ for } \tilde{\sigma} \in [n] \setminus T \end{array} \right\}$$

and

$$\left\{ (k_\sigma)_{\sigma \in T}, (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \left| \begin{array}{l} (k_1, \dots, k_n) \leftarrow \binom{n}{t}\text{-DPF.Gen}(1^\lambda, \alpha, \beta) \\ y_\sigma \leftarrow \binom{n}{t}\text{-DPF.Eval}(\sigma, k_\sigma, x) \text{ for } \sigma \in T \\ (y_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T} \leftarrow \text{RandShamir}(f_{\alpha, \beta}(x), t, T, \{y_\sigma\}_{\sigma \in T}) \end{array} \right. \right\}$$

are computationally indistinguishable. The statement already holds when $|T| = t$ by correctness. We then consider the case that $|T| < t$ and choose $m := t - |T|$ parties, $P_{i_1}, \dots, P_{i_m}$, that are not in T with m smallest indices. It suffices to prove that for each $j \in [m]$, given the set of keys $\{k_\sigma\}_{\sigma \in T}$ and $\{k_{i_1}, \dots, k_{i_{j-1}}\}$, the output obtained by evaluating $\binom{n}{t}\text{-DPF.Eval}(i_j, k_{i_j}, x)$ is pseudorandom. Observe that we only need to show for any $T' \in \binom{[n]}{\leq t-1}$ and any party P_ℓ such that $P_\ell \notin T'$, it holds that given the set of keys $\{k_\sigma\}_{\sigma \in T'}$, the output obtained by evaluating $\binom{n}{t}\text{-DPF.Eval}(\ell, k_\ell, x)$ is pseudorandom (denoted by Statement *) since the previous statement can be implied by this by induction on j .

To prove Statement *, notice that there exists a subset $R \in \binom{[n]}{t}$ such that $T' \subset R$ but $P_\ell \notin R$, which implies that the additive $\binom{n}{t}$ -party DPF K_R labeled by R is known by party P_ℓ but not to parties in R and thus T' . By the pseudorandom output property of additive $\binom{n}{t}$ -party DPF, the value $Y_R \leftarrow \text{DPF.Eval}(R, K_R, x)$ is pseudorandom given the set of keys $\{k_i\}_{i \in T'}$. Therefore, the value

$$y_\ell = \sum_{A \in \binom{[n]}{t}: \ell \notin A} Y_A \cdot f_A(\ell) = Y_R \cdot f_R(\ell) + \sum_{A \in \binom{[n]}{t}: \ell \notin A, A \neq R} Y_A \cdot f_A(\ell)$$

is pseudorandom given the set of keys $\{k_i\}_{i \in T'}$, which proves Statement *.

Eventually, we conclude the construction in $\binom{n}{t}\text{-DPF}$ is a secure $\binom{n}{t}$ -DPF with degree- t Shamir secret sharing satisfying the property of pseudorandom output. $\blacksquare$

Observed in [17], there is a general transformation from DPF to PCG for constant-degree correlations assuming hardness of dual-LPN, and further to HSS for constant-degree polynomials due to the equivalence between PCG and HSS. Although such a general transformation is originally considered in the setting of two parties and additive sharings, it can be extended to the setting of multiparty and linear secret sharing schemes. Applying this transformation to $\binom{n}{t}$ -DPF with t -threshold Σ -sharing results an $\binom{n}{t}$ -PCG for constant-degree correlations with Σ -sharing and an $\binom{n}{t}$ -HSS with Σ -sharing for constant-degree polynomials.

Lemma 20 ($\binom{n}{t}$ -DPF to $\binom{n}{t}$ -PCG). *Let n, t be two integers and d be a constant integer. Let Σ be a t -threshold linear secret sharing scheme. Assuming the dual $\mathbb{F}$ -LPN(I, I', k) assumption, if there exists an $\binom{n}{t}$ -DPF for point functions with domain size $(I')^d$ with Σ -sharing with pseudorandom outputs with key of size ℓ , there exists a secure $\binom{n}{t}$ -PCG that generates degree- d correlation $\mathcal{C}(n, d, \Sigma)$, where $\mathcal{C}(n, d, \Sigma)$ is defined as the correlation that all parties take Σ -sharings of each of the components in $\otimes_{j=1}^d \mathbf{r}_j$ with each $\mathbf{r}_j \in \mathbb{F}^I$ being pseudorandom.*

The seed size of $\binom{n}{t}$ -PCG is $O(k^d \cdot \ell)$, where k denotes the noise weight of the dual $\mathbb{F}$ -LPN assumption.

As a side-note, the reverse sampling algorithm for $\mathcal{C}(n, d, \Sigma)$ can be constructed by first sampling the secrets $\{\mathbf{r}_j\}_{j \in [d]}$ and invoking the reverse sampling algorithm for Σ -sharing for each component in $\otimes_{j=1}^d \mathbf{r}_j$.

Lemma 21 ($\binom{n}{t}$ -PCG to $\binom{n}{t}$ -HSS). *Let n, t be two integers and d be a constant integer. Let Σ be a t -threshold linear secret sharing scheme. If there exists an $\binom{n}{t}$ -PCG generating degree- d correlation $\mathcal{C}(n, d, \Sigma)$ with seed of size s , there exists a secure $\binom{n}{t}$ -HSS with Σ -sharing and pseudorandom outputs that homomorphically evaluates degree- d multi-variate polynomials $P: \mathbb{F}^I \rightarrow \mathbb{F}^O$. The share size of $\binom{n}{t}$ -HSS is $O(I + s)$.*

Focusing on existing constructions of multiparty DPF with additive sharings, [22] proposes a construction based on OWFs with key size being exponential in the number of parties. By leveraging randomized bipartite graphs, [52] manages to reduce the exponential dependency of key size on the number of parties and thus supports a polynomial number of parties. However,

both schemes retain a key size that scales with the square root of the domain size. This limitation would hinder their applications to point functions over large domain size.

By leveraging more structured assumptions, [40] builds multiparty DPFs with additive sharings from hardness of LPN and MQ with more compact DPF keys. However, this comes at the cost of supporting only a constant number of parties. Consequently, our derived constructions for $\binom{n}{t}$ -DPF, $\binom{n}{t}$ -PCG and $\binom{n}{t}$ -HSS with t -threshold secret sharing schemes above are presently limited to support a constant number of parties. We will explore the possibility of accommodating polynomial number of parties as well as preserving the compactness of seed below.

5.4 $\binom{n}{t}$ -PCG via replicated secret sharings: polynomial n and constant $n - t$

We elaborate our construction of $\binom{n}{t}$ -PCGs for constant-degree correlations accommodating polynomial number of parties by first focusing on the case of degree-2 correlations, e.g., Beaver triples, for simplicity. An observation is that letting all parties share the truth table of point functions and thus are able to distributively compute constant-degree correlations (through DPF-to-PCG compiler) is a bit redundant. In particular, suppose all parties first maintain replicated secret sharings of random values a and b , where $a = \sum_{T \in \binom{[n]}{t}} a_T, b = \sum_{S \in \binom{[n]}{t}} b_S$ and each party P_i owns $\{a_T | T \in \binom{[n]}{t}, i \notin T\}, \{b_S | S \in \binom{[n]}{t}, i \notin S\}$. To distribute $a \cdot b = \sum_{T \in \binom{[n]}{t}, S \in \binom{[n]}{t}} a_T \cdot b_S$, it remains to distribute each $a_T \cdot b_S$ among parties owning at least one of a_T and b_S , i.e., parties in $M := ([n] \setminus T) \cup ([n] \setminus S)$. Importantly, to maintain the privacy of resulting $\binom{n}{t}$ -PCG, it requires that any subset of parties $\mathcal{C} \subseteq ([n] \setminus T) \cup ([n] \setminus S)$ can reconstruct to learn $a_T \cdot b_S$ only if they are able to reconstruct each of a_T and b_S so that the privacy of b_S (a_T) is preserved when parties only have access to a_T (b_S).

This introduces our need of $\binom{m}{m-(n-t)}$ -PCG for constant-degree correlations to distribute $a_T \cdot b_S$ among parties in M , where $m = |M|$ and hence $n - t + 1 \leq m \leq 2 \cdot (n - t)$. Concretely, invoking $\binom{[m]}{t}$ -PCG to distribute $a_T \cdot b_S$ ensures that if parties in $\mathcal{C}$ do not have access to one of a_T and b_S , say a_T , then $\mathcal{C} \cap ([n] \setminus T) = \emptyset$ and $|\mathcal{C}| \leq m - (n - t)$ (otherwise at least one of parties in $\mathcal{C}$ knows a_T). Hence, by security of $\binom{m}{m-(n-t)}$ -PCG, parties in $\mathcal{C}$ have no information about $a_T \cdot b_S$. It is worth nothing that programmability of such an $\binom{m}{m-(n-t)}$ -PCG that enables programming the pseudorandom variables to be distributed is needed to maintain the consistency among different cross-terms. When $n - t$ is a constant, this can be instantiated by the construction above from multiparty DPF with additive sharing.

Hence, by invoking expanding procedure of $\binom{m}{m-(n-t)}$ -PCG, parties in M are able to compute their shares of $[a_T \cdot b_S]_{m-(n-t)}^m$. Observe that if parties outside M take 0 as their shares of $a_T \cdot b_S$, all n shares will define a polynomial with secret equal to $a_T \cdot b_S$ of degree $m - (n - t) + (n - m) = t$. As a result, it can be viewed that all parties hold degree- t Shamir sharing of $a_T \cdot b_S$, i.e., $[a_T \cdot b_S]_t^n$. By linear homomorphism of Shamir secret sharings, all parties can compute their shares of $[a \cdot b]_t^n$.

Based on threshold PCGs accommodating constant number of parties constructed above, when $n - t$ is a constant, we are able to construct $\binom{n}{t}$ -PCG for degree-2 correlations accommodating polynomial number of parties, which can be extended for constant-degree correlations. Both correctness and security follow from that of the replicated secret sharing scheme and the underlying threshold PCG.

Construction 2: $\binom{n}{t}$ -PCG for degree- d correlations

Let $\mathcal{C}$ be a code generation algorithm such that $H_{I', I} \leftarrow_{\$} \mathcal{C}(I', I, \mathbb{F})$ is a random parity-check matrix.

$\binom{n}{t}$ -PCG.Gen(1^λ):

1. **Generating d random variables with replicated secret sharings.** For each $j \in [d]$ and $T \in \binom{[n]}{t}$, randomly generate a sparse vector $e_{j,T} \leftarrow_{\$} \text{HW}_w(\mathbb{F}^{I'})$ expanded by $\alpha_{j,T}$. Let $\mathbf{r}_{j,T} \leftarrow H \cdot \mathbf{e}_{j,T}$ and $\mathbf{r}_j \leftarrow \sum_{T \in \binom{[n]}{t}} \mathbf{r}_{j,T}$.

2. **Distributing cross-terms.** For each $(T_1, \dots, T_d) \in \binom{[n]}{t}^d$, compute

$$(S_{T_1, \dots, T_d}^i)_{i \in M} \leftarrow \binom{m}{m - (n - t)}\text{-PCG.Gen}(1^\lambda, \otimes_{j=1}^d \mathbf{r}_{j, T_j}),$$

where $M := \cup_{j=1}^d ([n] \setminus T_j)$ and $m := |M|$.

3. **Assign seeds for each party.** For each $i \in [n]$, set

$$s_i \leftarrow (\{\alpha_{j, T}\}_{j \in [d], T \in \binom{[n]}{t}: i \notin T}, \{S_{T_1, \dots, T_d}^i\}_{(T_1, \dots, T_d) \in \binom{[n]}{t}^d: i \in \cup_{j=1}^d ([n] \setminus T_j)}).$$

4. Output $(s_i)_{i \in [n]}$.

$\binom{n}{t}$ -PCG.Expand(i, s_i):

1. Parse s_i as

$$s_i \leftarrow (\{\alpha_{j, T}\}_{j \in [d], T \in \binom{[n]}{t}: i \notin T}, \{S_{T_1, \dots, T_d}^i\}_{(T_1, \dots, T_d) \in \binom{[n]}{t}^d: i \in \cup_{j=1}^d ([n] \setminus T_j)}).$$

2. For each $j \in [d]$, compute degree- t Shamir sharings of $\mathbf{r}_j$ as

$$[\mathbf{e}_j]_t^n \leftarrow \sum_{T \in \binom{[n]}{t}: i \notin T} \mathbf{e}_{j, T} \cdot f_T(i) \text{ and } [\mathbf{r}_j]_t^n \leftarrow H \cdot [\mathbf{e}_j]_t^n.$$

3. For each $(T_1, \dots, T_d) \in \binom{[n]}{t}^d$, compute

$$[\otimes_{j=1}^d \mathbf{r}_{j, T_j}]_{m - (n - t)}^m \leftarrow \binom{m}{m - (n - t)}\text{-PCG.Expand}(i, S_{T_1, \dots, T_d}^i),$$

where $m := |\cup_{j=1}^d ([n] \setminus T_j)|$ and $[\otimes_{j=1}^d \mathbf{r}_{j, T_j}]_{m - (n - t)}^m$ can be viewed as $[\otimes_{j=1}^d \mathbf{r}_{j, T_j}]_t^n$.

4. Compute $[\otimes_{j=1}^d \mathbf{r}_j]_t^n \leftarrow \sum_{(T_1, \dots, T_d) \in \binom{[n]}{t}^d} [\otimes_{j=1}^d \mathbf{r}_{j, T_j}]_t^n$.

5. Output $(\{[\mathbf{r}_j]_t^n\}_{j \in [d]}, [\otimes_{j=1}^d \mathbf{r}_j]_t^n)$.

Theorem 22 ($\binom{n}{t}$ -PCG: polynomial n and constant $n - t$). Let n, t be integers such that $n - t$ is a constant. Assuming the dual $\mathbb{F}$ -LPN assumption, based on existence of $\binom{n'}{n' - (n - t)}$ -PCGs with degree- t Shamir secret sharing for all $n - t + 1 \leq n' \leq d \cdot (n - t)$, the construction in $\binom{n}{t}$ -PCG is a secure $\binom{n}{t}$ -PCG that generates degree- d correlation $(\{[\vec{\mathbf{r}}_j]_t^n\}_{j \in [d]}, [\otimes_{j=1}^d \mathbf{r}_j]_t^n)$ with degree- t Shamir sharing for n parties, where $\mathbf{r}_j \in \mathbb{F}^I$ is pseudorandom for all $j \in [d]$.

If the construction for multiparty DPF with additive sharings from LPN and MQ in [40] is used to instantiate $\binom{n'}{n' - (n - t)}$ -PCG via Lemma 20, the seed size of $\binom{n}{t}$ -PCG is $O(\binom{n}{t}^d \cdot k^d \cdot \lambda^3)$.

Proof of Theorem 22. We first prove the correctness from the dual LPN assumption and PCG correctness of the underlying PCGs. Starting from the real world distribution

$$\begin{aligned} \mathcal{D}_0 &:= \left\{ (R_1, \dots, R_n) \left| \begin{array}{l} (s_1, \dots, s_n) \leftarrow \binom{n}{t}\text{-PCG.Gen}(1^\lambda) \\ (R_i) \leftarrow \binom{n}{t}\text{-PCG.Expand}(i, s_i) \text{ for each } i \in [n] \end{array} \right. \right\} \\ &= \left\{ (R_1, \dots, R_n) \left| \begin{array}{l} \mathbf{e}_{j, T} \leftarrow \text{HW}_w(\mathbb{F}^{I'}), \mathbf{r}_{j, T} \leftarrow H \cdot \mathbf{e}_{j, T} \text{ for each } j \in [d], T \in \binom{[n]}{t} \\ \mathbf{r}_j \leftarrow \sum_{T \in \binom{[n]}{t}} \mathbf{r}_{j, T} \text{ for each } j \in [d] \\ (S_{T_1, \dots, T_d}^i)_{i \in \cup_{j=1}^d ([n] \setminus T_j)} \leftarrow \text{PCG.Gen}(1^\lambda, \otimes_{j=1}^d \mathbf{r}_{j, T_j}) \\ s_i \leftarrow (\{\alpha_{j, T}\}_{j \in [d], T \in \binom{[n]}{t}: i \notin T}, \{S_{T_1, \dots, T_d}^i\}_{(T_1, \dots, T_d) \in \binom{[n]}{t}^d: i \in \cup_{j=1}^d ([n] \setminus T_j)}) \\ (R_i) \leftarrow \binom{n}{t}\text{-PCG.Expand}(i, s_i) \text{ for each } i \in [n] \end{array} \right. \right\}, \end{aligned}$$

our goal is to prove $\mathcal{D}_0$ is computationally indistinguishable from the distribution

$$\left\{ (R_1, \dots, R_n) \left| \begin{array}{l} \mathbf{r}_j \leftarrow_{\$} \mathbb{F}^I \text{ for each } j \in [d] \\ (R_1, \dots, R_n) \leftarrow \text{RandShamir}(\{\mathbf{r}_j\}_{j \in [d]}, \otimes_{j=1}^d \mathbf{r}_j, t, \emptyset, \emptyset) \end{array} \right. \right\},$$

which will be shown using the following hybrid arguments.

Hyb₀. In this hybrid, we consider the real world distribution $\mathcal{D}_0$.

Hyb₁. In this hybrid, for each $(T_1, \dots, T_d) \in \binom{[n]}{t}^d$, instead of invoking $\binom{m}{m-(n-t)}$ -PCG.Gen($1^\lambda, \otimes_{j=1}^d \mathbf{r}_{j,T_j}$) to generate seeds, we directly sample random degree- t Shamir sharings with v , for v being each component of $\otimes_{j=1}^d \mathbf{r}_{j,T_j}$. The distribution in **Hyb₁** is computationally indistinguishable from that in **Hyb₀** due to the PCG correctness of $\binom{m}{m-(n-t)}$ -PCG.Gen.

Hyb₂. In this hybrid, for each $j \in [d], T \in \binom{[n]}{t}$, instead of randomly sampling a sparse vector $\mathbf{e}_{j,T} \leftarrow_{\$} \text{HW}_w(\mathbb{F}^{I'})$ and computing $\mathbf{r}_{j,T} \leftarrow H \cdot \mathbf{e}_{j,T}$, we directly sample $\mathbf{r}_{j,T} \leftarrow_{\$} \mathbb{F}^I$. The distribution in **Hyb₂** is computationally indistinguishable from that in **Hyb₁** assuming the dual-LPN assumption.

Hyb₃. In this hybrid, for each $j \in [d]$, instead of sampling $\mathbf{r}_{j,T} \leftarrow_{\$} \mathbb{F}^I$ for each $T \in \binom{[n]}{t}$ and computing $\mathbf{r}_j \leftarrow \sum_{T \in \binom{[n]}{t}} \mathbf{r}_{j,T}$, we directly randomly sample $\mathbf{r}_j \leftarrow_{\$} \mathbb{F}^I$, $\{\mathbf{r}_{j,S}\}_{j \in [d], S \in \binom{[n]}{t}, S \neq T_0}$ and set $\mathbf{r}_{j,T_0} = \mathbf{r}_j - \sum_{S \in \binom{[n]}{t}: S \neq T_0} \mathbf{r}_{j,S}$ for each $j \in [d]$, where $T_0 = \{1, \dots, t\}$. The distribution in **Hyb₃** is identical to that in **Hyb₂** due to the local conversion from replicated secret sharing with secret to degree- t Shamir secret sharing and the randomness of $\{\mathbf{r}_{j,T_0}\}_{j \in [d]}$ in **Hyb₂**, which corresponds to the target distribution.

As for the security, we prove this from PCG security of the underlying PCGs. For any set of corrupted parties $T \in \binom{[n]}{\leq t}$, starting from the real world distribution

$$\mathcal{D}_0 := \left\{ (k_\sigma)_{\sigma \in T}, (R_{\tilde{\sigma}})_{\tilde{\sigma} \notin T} \left| \begin{array}{l} (S_1, \dots, S_n) \leftarrow \binom{n}{t}\text{-PCG.Gen}(1^\lambda) \\ R_{\tilde{\sigma}} \leftarrow \binom{n}{t}\text{-PCG.Expand}(\tilde{\sigma}, S_{\tilde{\sigma}}) \text{ for each } \tilde{\sigma} \notin T \end{array} \right. \right\},$$

our goal is prove this is computationally indistinguishable from the distribution

$$\left\{ (k_\sigma)_{\sigma \in T}, (R_{\tilde{\sigma}})_{\tilde{\sigma} \notin T} \left| \begin{array}{l} (S_1, \dots, S_n) \leftarrow \binom{n}{t}\text{-PCG.Gen}(1^\lambda) \\ R_\sigma \leftarrow \binom{n}{t}\text{-PCG.Expand}(\sigma, S_\sigma) \text{ for each } \sigma \in T \\ (R_{\tilde{\sigma}}) \leftarrow \text{RSample}(T, \{R_\sigma\}_{\sigma \in T}) \end{array} \right. \right\},$$

which will be shown using hybrid arguments.

Hyb₀. In this hybrid, we consider the real world distribution $\mathcal{D}_0$.

Hyb₁. In this hybrid, for $S \in \binom{[n]}{t}$ such that $T \subseteq S$, for each $\ell \in [d]$ and $(T_1, \dots, T_d) \in \binom{[n]}{t}^d$ with $T_\ell = S$, instead of invoking $\binom{m}{m-(n-t)}$ -PCG.Gen($1^\lambda, \otimes_{j=1}^d \mathbf{r}_{j,T_j}$) to generate PCG seeds for parties in $M := \cup_{j=1}^d ([n] \setminus T_j)$, where $m := |M|$, we invoke the reverse sampling algorithm of $\binom{m}{m-(n-t)}$ -PCG. In particular, let $(s_{T_1, \dots, T_d}^i)_{i \in M} \leftarrow \binom{m}{m-(n-t)}$ -PCG.Gen($1^\lambda, \otimes_{j=1}^d \mathbf{r}_{j,T_j}$), $R_{T_1, \dots, T_d}^\sigma \leftarrow \binom{m}{m-(n-t)}$ -PCG.Expand($\sigma, s_{T_1, \dots, T_d}^\sigma$) for each $\sigma \in T \cap M$ and $(R_{T_1, \dots, T_d}^{\tilde{\sigma}})_{\tilde{\sigma} \in M-T} \leftarrow \text{RSample}(T, \{s_{T_1, \dots, T_d}^\sigma\}_{\sigma \in T \cap M})$. The distribution in **Hyb₁** is computationally indistinguishable from that in **Hyb₀** due to the PCG security of $\binom{m}{m-(n-t)}$ -PCG.

Hyb₂. In this hybrid, instead of invoking the reverse sampling algorithm **RSample** of $\binom{m}{m-(n-t)}$ -PCG for each $S \in \binom{[n]}{t}$ such that $T \subseteq S$ and each $(T_1, \dots, T_d)$ such that there exists some $j \in [d]$ satisfying $T_j = S$ as in **Hyb₁**, for each component v in $\otimes_{j=1}^d \mathbf{r}_j$, we first compute the shares of corrupted parties of $[v]_t^n$ as $\{v_\sigma\}_{\sigma \in T}$ and invoke the reverse sampling algorithm of Shamir secret sharings **RandShamir**($v, 1, t, T, \{v_\sigma\}_{\sigma \in T}$) to obtain the whole sharing $(v_{\tilde{\sigma}})_{\tilde{\sigma} \in [n] \setminus T}$. The distribution in **Hyb₂** is identical to that in **Hyb₁**.

Hyb₃. In this hybrid, for each $j \in [d]$ and $T \in \binom{[n]}{t}$, instead of randomly sampling a sparse vector $\mathbf{e}_{j,T} \leftarrow_{\$} \text{HW}_w(\mathbb{F}^{I'})$ and computing $\mathbf{r}_{j,T} \leftarrow H \cdot \mathbf{e}_{j,T}$, we directly sample $\mathbf{r}_{j,T} \leftarrow_{\$} \mathbb{F}^I$. The distribution in **Hyb₃** is computationally indistinguishable from that in **Hyb₂** assuming the dual-LPN assumption.

Hyb₄. In this hybrid, for each $j \in [d]$, instead of sampling $r_{j,T} \leftarrow_{\$} \mathbb{F}^I$ for each $T \in \binom{[n]}{t}$ and computing $r_j \leftarrow \sum_{T \in \binom{[n]}{t}} r_{j,T}$, we directly randomly sample $r_j \leftarrow_{\$} \mathbb{F}^I$. The distribution in **Hyb₃** is identical to that in **Hyb₄** due to the local conversion from replicated secret sharing with secret to degree- t Shamir secret sharing, which corresponds to the target distribution. $\blacksquare$

5.5 $\binom{n}{t}$ -PCG via party emulation: polynomial n and constant t

To formalize our construction, we consider the following so-called composed secret sharing scheme. To share a secret $s \in \mathbb{F}$, the sharing map $\text{share} : \mathbb{F} \times \mathbb{F}^{1+\binom{n}{t} \cdot t} \rightarrow (\mathbb{F}^{\binom{n-1}{t-1}})^{\mathcal{T}}$ first samples a random degree-1 Shamir secret sharing $(s_T)_{T \in \binom{[n]}{t}}$ with secret s distributed among all t -subsets $\{T\}_{T \in \binom{[n]}{t}}$. Then for each $T \in \binom{[n]}{t}$, it samples a random additive secret sharing of s_T as $(s_T^i)_{i \in T}$ distributed among all parties in T . Then each party P_i obtains the set of additive shares $S_i \leftarrow (s_T^i)_{T \in \binom{[n]}{t}: i \in T} \in \mathbb{F}^{\binom{n-1}{t-1}}$ that are involved with i as its share of s . The reconstruction map takes as input any subset A of $t+1$ shares, since there are $t+1$ t -subsets contained in A , then $t+1$ shares of $(s_T)_{T \in \binom{[n]}{t}}$ can be recovered and thus s can be reconstructed. As any t shares of s reveal no information about the secret s , this is a t -threshold secret sharing scheme. The whole sharing $(S_1, \dots, S_n)$ is denoted by $\langle S \rangle_t$. Notice that this composed secret sharing scheme is linearly homomorphic, i.e., $\langle x_1 \rangle_t + \langle x_2 \rangle_t = \langle x_1 + x_2 \rangle_t$.

To show that composed secret sharing correlation is reverse-sampleable, we define the following procedure which samples a random threshold- t composed secret sharing given a secret $r \in \mathbb{F}$ and a set of shares $\{R_j\}_{j \in T}$ for some $T \in \binom{[n]}{\leq t}$.

- **RandComposed**($r, t, T, \{R_j\}_{j \in T}$): The algorithm first forms a subset $S \in \binom{[n]}{t}$ such that $T \subseteq S$ by further including $t - |T|$ parties with smallest indices that are not in T . Then it randomly samples elements as shares of parties in $S \setminus T$. Then the share r^S labeled with S of degree-1 Shamir sharing can be computed and the whole degree-1 Shamir sharing of r denoted by $(r_{S'})_{S' \in \binom{[n]}{t}}$ is determined by the secret r and the share r_S . Then for each $S' \in \binom{[n]}{t}$ with $S' \neq S$, it randomly samples an additive sharing of $r_{S'}$ according to the secret $r_{S'}$ and known shares with indices in $S \cap S'$. Till now, the threshold- t composed secret sharing of r denoted by $(R_i)_{i \in [n]}$ is fully determined. Finally, it outputs $(R_j)_{j \in [n] \setminus T}$.

As mentioned in Section 2.3, to construct an $\binom{n}{t}$ -DPF and compile it to an $\binom{n}{t}$ -PCG for constant-degree correlations, the party emulation framework [55] is used to combine an $\binom{n}{t}$ -party DPF against one corruption and t -party HSS with additive sharings. More specifically, the point function is first distributed to $\binom{n}{t}$ keys using the $\binom{n}{t}$ -party DPF, where each key corresponds to a subset of t parties. Then parties in each subset apply a t -party HSS scheme to homomorphically evaluate the evaluation function of the $\binom{n}{t}$ -party DPF. As a result, the point function is shared among all parties with threshold- t composed secret sharings. The construction is shown below.

Construction 3: $\binom{n}{t}$ -DPF

$\binom{n}{t}$ -DPF.Gen($1^\lambda, \alpha, \beta$):

1. Invoke $\binom{n}{1}$ -DPF to generate DPF keys. $\text{DPF.Gen}(1^\lambda, \alpha, \beta) \rightarrow (k_T)_{T \in \binom{[n]}{t}}$. Denote the function family $\{g_{T,x} : k_T \mapsto \text{DPF.Eval}(T, k_T, x) \mid x \in [D], T \in \binom{[n]}{t}\}$ by $\mathcal{G}$.
2. For each $T \in \binom{[n]}{t}$, invoke t -party additive HSS for $\mathcal{G}$ to generate secret sharings $(x_T^{(i)})_{i \in T} \leftarrow \text{HSS.Share}(1^\lambda, k_T)$.
3. For each $i \in [n]$, set $K_i \leftarrow \{x_T^{(i)}\}_{T \in \binom{[n]}{t}: i \in T}$.
4. Output $(K_i)_{i \in [n]}$.

$\binom{n}{t}$ -DPF.Eval(i, K_i, x):

1. Parse K_i as $\{x_T^{(i)}\}_{T \in \binom{[n]}{t}: i \in T}$.

2. For each $T \in \{S \in \binom{[n]}{t} : i \in S\}$, compute $y_T^{(i)} \leftarrow \text{HSS.Eval}(i, g_{T,x}, x_T^{(i)})$.
3. Output $Y_i \leftarrow (y_T^{(i)})_{T \in \binom{[n]}{t} : i \in T}$.

Lemma 23 (Imported lemma [6]: $\binom{n}{1}$ -DPF). *Let $n \geq 1$ and ℓ be integers, let $\mathbb{F}$ be a finite field, and let $G : \{0, 1\}^\lambda \rightarrow \{0, 1\}^{2(\lambda + \lceil \log n \rceil)}$ be a PRG computable by constant-degree polynomials. Then, there exists an $\binom{n}{1}$ -DPF for $f_{\alpha, \beta} : \{0, 1\}^\ell \rightarrow \mathbb{F}$ with the following efficiency features:*

- Per-party key size $\lambda + (2^{\lceil \log n \rceil} - 1)(\lambda + 2\lceil \log n \rceil)\ell + 2^{\lceil \log n \rceil}(n - 1)\lceil \log |\mathbb{F}| \rceil$ bits.
- Gen invokes G at most $n(\ell + \lceil \frac{n \log |\mathbb{F}|}{2(\lambda + \lceil \log n \rceil)} \rceil)$ times.
- Eval invokes G at most $\ell + \lceil \frac{n \log |\mathbb{F}|}{2(\lambda + \lceil \log n \rceil)} \rceil$ times.

Theorem 24 ($\binom{n}{t}$ -DPF: polynomial n and constant t). *Let n, t be integers. If $\text{DPF} = (\text{DPF.Gen}, \text{DPF.Eval})$ is a secure $\binom{n}{1}$ -DPF scheme with degree-1 Shamir secret sharing with pseudorandom outputs and $\text{HSS} = (\text{HSS.Share}, \text{HSS.Eval})$ is a t -party HSS with additive sharing for function family $\mathcal{G}$ with pseudorandom outputs, where $\mathcal{G} := \{g_{T,x} : k_T \mapsto \text{DPF.Eval}(T, k_T, x) \mid x \in [D], T \in \binom{[n]}{t}\}$, then the construction in $\binom{n}{t}$ -DPF is a secure $\binom{n}{t}$ -DPF with threshold- t composed secret sharing.*

Proof of Theorem 24. The correctness immediately comes from correctness of the $\binom{n}{1}$ -DPF scheme and the t -party additive HSS scheme. In fact, notice that letting $y_T := \sum_{i \in T} y_T^{(i)} = \sum_{i \in T} \text{HSS.Eval}(i, x_T^{(i)}) = g_{T,x}(k_T) = \text{DPF.Eval}(T, k_T, x)$ by HSS correctness of HSS and $(y_T)_{T \in \binom{[n]}{t}}$ forms a degree-1 Shamir secret sharing of $f_{\alpha, \beta}(x)$ by DPF correctness of DPF. Hence, $(Y_i)_{i \in [n]}$ forms a threshold- t composed secret sharing of $f_{\alpha, \beta}(x)$.

As for the security, w.o.l.g., for any set $T \in \binom{[n]}{t}$ of corrupted parties, the PPT. simulator Sim performs as follows. It first invokes the simulator for $\binom{n}{1}$ -DPF Sim_{DPF} to simulate the DPF key k_T corresponding to T . Then it honestly runs $\text{HSS.Share}(1^\lambda, k_T)$ to generate HSS shares $(x_T^{(i)})_{i \in T}$ of input k_T for parties in T . Then for each $S \in \binom{[n]}{t}$ with $S \cap T \neq \emptyset$ and $S \neq T$, it invokes the PPT. simulator for t -party additive HSS Sim_{HSS} to simulate HSS shares $(x_S^{(i)})_{i \in S \cap T}$. For each $i \in T$, set $Y_i \leftarrow (x_S^{(i)})_{S \in \binom{[n]}{t} : i \in S}$. Then it outputs $(Y_i)_{i \in [n]}$. The effectiveness of Sim follows from the HSS security of HSS and the DPF security of DPF.

As for the pseudorandomness of the outputs, for any $T \in \binom{[n]}{\leq t}$, our goal is to prove the following two distributions

$$\mathcal{D}^{\text{real}} := \left\{ (K_i)_{i \in T}, (Y_i)_{i \in [n] \setminus T} \left| \begin{array}{l} (k_R)_{R \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta) \\ (x_R^{(i)})_{i \in R} \leftarrow \text{HSS.Share}(1^\lambda, k_R, g_{R,x}) \text{ for } R \in \binom{[n]}{t} \\ y_R^{(i)} \leftarrow \text{HSS.Eval}(i, x_R^{(i)}) \text{ for } R \in \binom{[n]}{t}, i \in R \setminus T \\ Y_i \leftarrow (y_R^{(i)})_{R \in \binom{[n]}{t} : i \in R} \text{ for } i \in [n] \setminus T \end{array} \right. \right\}$$

and

$$\mathcal{D}^{\text{ideal}} := \left\{ (K_i)_{i \in T}, (Y'_i)_{i \in [n] \setminus T} \left| \begin{array}{l} (k_R)_{R \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta) \\ (x_R^{(i)})_{i \in R} \leftarrow \text{HSS.Share}(1^\lambda, k_R, g_{R,x}) \text{ for } R \in \binom{[n]}{t} \\ y_R^{(i)} \leftarrow \text{HSS.Eval}(i, x_R^{(i)}) \text{ for } R \in \binom{[n]}{t}, i \in R \cap T \\ Y_i \leftarrow (y_R^{(i)})_{R \in \binom{[n]}{t} : i \in R} \text{ for } i \in T \\ (Y'_i)_{i \in [n] \setminus T} \leftarrow \text{RandCompose}(f_{\alpha, \beta}(x), t, T, \{Y_i\}_{i \in T}) \end{array} \right. \right\}$$

are computationally indistinguishable.

By pseudorandomness of the outputs of the HSS scheme, we have that the distribution $\mathcal{D}^{\text{real}}$ is computationally indistinguishable with the following distribution

$$\mathcal{D}_1 := \left\{ (K_i)_{i \in T}, (Y'_i)_{i \in [n] \setminus T} \left| \begin{array}{l} (k_R)_{R \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta) \\ y_R \leftarrow \text{DPF.Eval}(R, k_R, x) \text{ for } R \in \binom{[n]}{t} \\ (x_R^{(i)})_{i \in R} \leftarrow \text{HSS.Share}(1^\lambda, k_R, g_{R,x}) \text{ for } R \in \binom{[n]}{t} \\ y_R^{(i)} \leftarrow \text{HSS.Eval}(i, x_R^{(i)}) \text{ for } R \in \binom{[n]}{t}, i \in R \cap T \\ (y_R'^{(i)})_{i \in R \setminus T} \leftarrow \text{RandAdditive}(y_R, R \cap T, \{y_R^{(i)}\}_{i \in R \cap T}) \text{ for } R \in \binom{[n]}{t} \\ Y'_i \leftarrow (y_R'^{(i)})_{R \in \binom{[n]}{t}: i \in R} \text{ for } i \in [n] \setminus T \end{array} \right. \right\},$$

where $\text{RandAdditive}(x, A, \{x_i\}_{i \in A})$ randomly samples an additive sharing $(x'_i)_{i \in A}$ according to the secret x and a set of shares $\{x_i\}_{i \in A}$ and outputs $(x'_i)_{i \in A}$.

Then by pseudorandomness of the outputs of the DPF scheme, we have that the distribution in $\mathcal{D}_2$ is computationally indistinguishable with the following distribution

$$\mathcal{D}_1 := \left\{ (K_i)_{i \in T}, (Y'_i)_{i \in [n] \setminus T} \left| \begin{array}{l} (k_R)_{R \in \binom{[n]}{t}} \leftarrow \text{DPF.Gen}(1^\lambda, \alpha, \beta), y_S \leftarrow \text{DPF.Eval}(S, k_S, x) \\ (y_R')_{R \in \binom{[n]}{t}: R \neq S} \leftarrow \text{RandShamir}(f_{\alpha, \beta}(x), t, \{S\}, y_S) \\ (y_R'^{(i)})_{i \in R \setminus T} \leftarrow \text{RandAdditive}(y_R, R \cap T, \{y_R^{(i)}\}_{i \in R \cap T}) \\ Y'_i \leftarrow (y_R'^{(i)})_{R \in \binom{[n]}{t}: i \in R} \text{ for } i \in [n] \setminus T \end{array} \right. \right\},$$

which corresponds to the distribution $\mathcal{D}^{\text{ideal}}$. ■

6 Applications of Pseudorandom Permutations

Permutation correlations represent an important class of non-additive correlations with practical and theoretical significance. They arise naturally in settings where uniqueness, anonymity, or ordering matter, such as anonymous communication and leader election. Establishing PCGs/PCFs for this class of correlations demonstrates that non-linear, non-additive randomness can be efficiently compressed. In this section, we describe two applications of our constructions. The first application is for slot assignment in Dining Cryptographer networks (DC-nets). The second application is to single secret leader election. While currently limited to the three party setting, future constructions of PCGs and PCFs for pseudorandom permutations capable of handling more parties would automatically extend these applications to a multi-party setting.

6.1 Anonymous broadcast via DC-nets

In a DC-net [33, 53], parties use secret sharing to hide the provenance of a message. DC-nets form the backbone of anonymous broadcast protocols [1, 37, 47, 59]. The main idea is that each party P_i with message $m_i \in \{0, 1\}^\ell$ generates a one-hot vector $\mathbf{v}_i$ with the message m_i in the j -th coordinate, where j is chosen randomly by party P_i . The vector $\mathbf{v}_i$ is then additively secret-shared with all other parties. Given all the secret shares, the parties can locally generate a secret share of the combined vector $\mathbf{v}$ by adding all the secret shared vectors together. That is, they compute a share of $\mathbf{v} := \sum_{i=1}^n \mathbf{v}_i$. Finally, by broadcasting their shares of $\mathbf{v}$ (or posting them to a bulletin board), the parties obtain the vector $\mathbf{v}$ in the clear, which reveals the set of messages while hiding the provenance of each message. In particular, DC-nets provide unlinkability between parties and their message since the position of each party's message in $\mathbf{v}$ was chosen randomly and independently.

A practical challenge with the DC-net architecture is handling *message collisions*—an artifact of each party randomly selecting the “slot” of $\mathbf{v}$ they write their message into. Concretely, the problem is that if two parties P_i and P_j write their messages in the same non-zero coordinate of the vector $\mathbf{v}$, then the corresponding messages are added together resulting in a “clobbered” output. The standard solution to this problem is to either (1) run a special protocol to allow each party to “reserve” a random slot in the output vector [59, 66] or (2) increase the size of the vector to make the probability of a collision sufficiently small [1, 37]. However, these solutions are undesirable given

that they both add communication and computation overheads on the parties. As highlighted by Golle and Juels [53]: “There is no good non-interactive means of enabling all players to select distinct message position.”

Here, we show that our constructions of PCGs and PCFs for permutations, provide such a *non-interactive* solution to address collisions, as originally imagined by Golle and Juels: Each party obtains a pseudorandom index automatically assigning them to a unique slot. Therefore, after a one-time setup protocol, the parties can use a DC-net without the risk of collisions. Moreover, this approach avoids the need for increasing the size of the message vector or running an allocation protocol at each round.

While this application to DC-nets is immediate for our unbiased PCF construction, we need to be a little more careful with our biased PCG construction from Appendix B. We show that in the context of DC-nets, where parties broadcast over multiple rounds, we can use a randomness extractor to make the permutation unbiased, while only modestly increasing the communication overhead on the parties (in particular, the parties only need to broadcast a few additional bits in each round, independent of the message).

Dealing with bias via the Von Neumann extractor. We first recall that the bias of the permutation comes from the pseudorandom bit μ being biased towards one. The Von Neumann extractor [68] is a simple way of unbiasing any Bernoulli distribution and works as follows: Given a sequence of biased random bits, examine each consecutive pair of bits and discard all pairs where the xor of the two bits is zero. The remaining bits are then guaranteed to be unbiased. Our idea is to use the multi-round DC-net setting to allow parties to unbias the bits “on the fly” at round r for use in round $r + 1$. The main idea is that the parties can use the biased bit PCG from Figure 5 to generate a sequence of $2k$ biased bits at round r . Then, at the end of round r , they reveal k sums corresponding to the sum of each pair of consecutive bits (the sum is computed over $\mathbb{F}_3$). Then, at round $r + 1$, the parties use the permutation generated using the first bit μ from a pair whose bits sum to one. By the guarantees of the Von Neumann extractor, all the bits in these pairs are distributed as unbiased pseudorandom bits (if the sum is $1 \in \mathbb{F}_3$, then the two bits in the pair are not equal). Moreover, because the parties only reveal the sum of the bits in the pair, no party learns whether the value of the first bit in the pair is zero or one.¹¹

Overall, the protocol proceeds as follows:

One-time Setup. In the one-time setup, the i -th party is given a PCG key k_i for Figure 5. The setup additionally distributes a uniformly random starting permutation to all parties. The round number r is initialized to zero. Given the PCG key k_i , the i -th party runs `PRBits.Expand` to generate a sequence of $\ell \cdot 2k$ biased pseudorandom bits and obtains the associated $2k$ pseudorandom biased permutation outputs.

Step 1: Broadcast. In the $r + 1$ -st round, the parties use the r -th permutation as their slot assignment for the DC-net protocol and compute shares of the vector $\mathbf{v}$ containing all their messages. Then, each party parses the r -th block of $2k$ consecutive shares of bits output by `PRBits.Expand` as a sequence of k pairs, and broadcasts the k shares obtained by summing the bits in each pair in this sequence. Each party outputs their share of $\mathbf{v}$ along with their shares of the k pair sums.

Step 2: Generating the next permutation. With the k sums revealed, let j denote the index of the first sum that equals 1 in the sequence of k sums. The parties select the $2j$ -th permutation output by `PCG.Expand` for the r -th round, which corresponds to selecting the pseudorandom permutation generated using the $2j$ -th bit output by `PRBits.Expand`.

Analysis. For each pair that sums to 1, the parties do not learn which bit in the pair was zero vs. one. Therefore, the sum of the two consecutive shares reveals whether or not the two bits are equal in $\mathbb{F}_3$, and nothing else. Then, by the properties of the Von Neumann extractor, we have the guarantee that all the pairs that sum to 1 are unbiased. By selecting the first such pair, we have the guarantee that both the bits in that pair are unbiased. Therefore, the permutation associated with the first bit of that sequence is also guaranteed to be unbiased.

¹¹ Indeed, all they learn is that the two bits are distinct. Moreover, all pairs that sum to 0 or 2 are discarded and therefore do not affect the protocol.

Determining the value of k . We still need to analyze how large k needs to be in order to guarantee, with high probability, that at least one pair has two differing bits (so that the sum of the bits is one). We compute k using a simple Chernoff bound. The probability that a bit is 1 is $\frac{2}{3}$. Therefore, the probability of getting two differing consecutive bits is $(\frac{1}{3} \cdot \frac{2}{3}) + (\frac{2}{3} \cdot \frac{1}{3}) = \frac{4}{9}$. Let X be the random variable representing the number of pairs of consecutive different bits in k trials. We want to find k such that $\Pr[X > 0] \geq 1 - 2^{-\lambda}$ so as to guarantee a negligible correctness failure in the security parameter. The expected value of X is $4k/9$ and by the Chernoff bound, we have that: $\Pr[X = 0] \leq e^{-2k/9}$. Therefore, by solving for k such that $2^{-\lambda} \leq e^{-2k/9}$, we get that we need $k \geq \frac{9}{2}\lambda \ln(2)$ so that $\Pr[X > 0] \geq 1 - 2^{-\lambda}$. For $\lambda = 40$, this requires $k \approx 120$ to ensure that the parties can extract a single unbiased bit from the k pairs of bits in each round.

6.2 Single Secret Leader Election

Here, we describe an application of our PCF construction to single secret leader election (SSLE) [8, 13, 30, 31, 35]. The problem of SSLE appears in many distributed systems contexts, and in particular, in proof-of-stake based cryptocurrencies. Informally, SSLE requires a set of parties to secretly agree on a single leader, such that only the elected leader knows that they were elected. Additionally, SSLE requires the leader to be able to prove, at some later point in time, that they were elected. The definition of SSLE is complex and so we refer the reader to Boneh et al. [13] for the full formalization. Here, we sketch how a PCF for pseudorandom permutations coupled with a NIZK [10] gives us a simple SSLE protocol.

The basic idea is that we can let the leader be decided by having the lowest permutation value (i.e., zero). By the security of the PCF, no party (besides the leader) learns who was elected in the round. Then, the only missing component is the ability for the elected leader to prove to the other parties that they obtained the lowest permutation value (i.e., that the output of `PCF.Eval` is zero). This can be achieved by having the output of the setup give all the parties commitments to the set of keys $\{k_0, k_1, k_2\}$, which then allows the i -th party to prove via a NIZK that the output of `PCF.Eval` (using key k_i) results in zero.

Overall, the SSLE protocol with n parties proceeds in three steps:

One-time Setup. In the one-time setup, the i -th party is given a PCF key k_i and a commitment to all other parties' PCF keys k_j for $j \in [n]$. The round number r is initialized to zero.

Step 1: Election. In the election step (which is repeated over many rounds), each party runs `PCF.Eval` using the round number r as input. The party with the zero output is determined as the leader at round r .

Step 2: Proof. In the proof step, the elected leader (say, party i) generates a NIZK to prove that, relative to the commitment of k_i , the output of `PCF.Eval` using key k_i and input r outputs zero.

Step 3: Verification. The parties use the public commitment of k_i to verify the NIZK generated by party i in round r .

Remark 25 (Requirement for a PCF). We crucially rely on the PCF to ensure that the elected leader can produce an efficient NIZK that doesn't grow with the total number of rounds. If a proof is not required, then a PCG would suffice to instantiate SSLE. Moreover, we note that our (biased) PCG construction gives us a perfectly *unbiased* SSLE protocol (even though the permutation itself is biased) since the party with the zero share is pseudorandomly distributed across all three parties.

Acknowledgements

Geoffroy Couteau acknowledges the support of the French Agence Nationale de la Recherche (ANR), under the France 2030 ANR Project ANR-22-PECY003 SecureCompute. This work is supported by ERC grant OBELiSC (101115790).

References

1. Ittai Abraham, Benny Pinkas, and Avivay Yanai. Blinder - scalable, robust anonymous committed broadcast. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1233–1252, Virtual Event, USA, November 9–13, 2020. ACM Press.
2. Damiano Abram, Ivan Damgård, Claudio Orlandi, and Peter Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 421–452, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
3. Damiano Abram, Ariel Nof, Claudio Orlandi, Peter Scholl, and Omer Shlomovits. Low-bandwidth threshold ECDSA via pseudorandom correlation generators. In *2022 IEEE Symposium on Security and Privacy*, pages 2554–2572, San Francisco, CA, USA, May 22–26, 2022. IEEE Computer Society Press.
4. Damiano Abram, Peter Scholl, and Sophia Yakubov. Distributed (correlation) samplers: How to remove a trusted dealer in one round. In Orr Dunkelman and Stefan Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 790–820, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
5. Michael Adjedj, Constantin Blokh, Geoffroy Couteau, Arik Galansky, Antoine Joux, and Nikolaos Makriyannis. Two-round 2PC ECDSA at the cost of 1 OLE. Cryptology ePrint Archive, Report 2024/1950, 2024.
6. Amit Agarwal, Elette Boyle, Niv Gilboa, Yuval Ishai, Mahimna Kelkar, and Yiping Ma. Compressing unit-vector correlations via sparse pseudorandom generators. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part VIII*, volume 14927 of *Lecture Notes in Computer Science*, pages 346–383, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
7. Benny Applebaum and Pavel Raykov. Fast pseudorandom functions based on expander graphs. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B: 14th Theory of Cryptography Conference, Part I*, volume 9985 of *Lecture Notes in Computer Science*, pages 27–56, Beijing, China, October 31 – November 3, 2016. Springer, Berlin, Heidelberg, Germany.
8. Michael Backes, Pascal Berrang, Lucjan Hanzlik, and Ivan Pryvalov. A framework for constructing single secret leader election from MPC. Cryptology ePrint Archive, Report 2022/1040, 2022.
9. Carsten Baum, Lennart Braun, Alexander Munch-Hansen, and Peter Scholl. $\text{Moz}\mathbb{Z}_{2^k}$ arella: Efficient vector-OLE and zero-knowledge proofs over $\mathbb{Z}_{2^k}$. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 329–358, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
10. Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications (extended abstract). In *20th Annual ACM Symposium on Theory of Computing*, pages 103–112, Chicago, IL, USA, May 2–4, 1988. ACM Press.
11. Maxime Bombar, Dung Bui, Geoffroy Couteau, Alain Couvreur, Clément Ducros, and Sacha Servan-Schreiber. FOLEAGE: $\mathbb{F}_4$ OLE-based multi-party computation for boolean circuits. In *Advances in Cryptology – ASIACRYPT 2024, Part VI*, *Lecture Notes in Computer Science*, pages 69–101. Springer, Singapore, Singapore, December 7–11, 2024.
12. Maxime Bombar, Geoffroy Couteau, Alain Couvreur, and Clément Ducros. Correlated pseudorandomness from the hardness of quasi-abelian decoding. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part IV*, volume 14084 of *Lecture Notes in Computer Science*, pages 567–601, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
13. Dan Boneh, Saba Eskandarian, Lucjan Hanzlik, and Nicola Greco. Single secret leader election. In *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies*, pages 12–24, 2020.
14. Charles Bouillaguet, Claire Delaplace, Mickaël Hamdad, and Damien Vergnaud. Practical cryptanalysis of pseudorandom correlation generators based on quasi-abelian syndrome decoding. Cryptology ePrint Archive, Paper 2025/892, 2025.
15. Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 896–912, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
16. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure computation. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 291–308, London, UK, November 11–15, 2019. ACM Press.

17. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 489–518, Santa Barbara, CA, USA, August 18–22, 2019. Springer, Cham, Switzerland.
18. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Correlated pseudorandom functions from variable-density LPN. In *61st Annual Symposium on Foundations of Computer Science*, pages 1069–1080, Durham, NC, USA, November 16–19, 2020. IEEE Computer Society Press.
19. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part II*, volume 12171 of *Lecture Notes in Computer Science*, pages 387–416, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland.
20. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, and Michele Orrù. Homomorphic secret sharing: Optimizations and applications. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2105–2122, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
21. Elette Boyle, Geoffroy Couteau, and Pierre Meyer. Sublinear-communication secure multiparty computation does not require FHE. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 159–189, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
22. Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 337–367, Sofia, Bulgaria, April 26–30, 2015. Springer, Berlin, Heidelberg, Germany.
23. Elette Boyle, Niv Gilboa, and Yuval Ishai. Breaking the circuit size barrier for secure computation under DDH. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part I*, volume 9814 of *Lecture Notes in Computer Science*, pages 509–539, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Berlin, Heidelberg, Germany.
24. Elette Boyle, Niv Gilboa, and Yuval Ishai. Function secret sharing: Improvements and extensions. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1292–1303, Vienna, Austria, October 24–28, 2016. ACM Press.
25. Elette Boyle, Niv Gilboa, Yuval Ishai, and Victor I. Kolobov. Programmable distributed point functions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 121–151, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
26. Elette Boyle, Niv Gilboa, Yuval Ishai, Huijia Lin, and Stefano Tessaro. Foundations of homomorphic secret sharing. In Anna R. Karlin, editor, *ITCS 2018: 9th Innovations in Theoretical Computer Science Conference*, volume 94, pages 21:1–21:21, Cambridge, MA, USA, January 11–14, 2018. LIPIcs.
27. Elette Boyle, Lisa Kohl, and Peter Scholl. Homomorphic secret sharing from lattices without FHE. In Yuval Ishai and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 3–33, Darmstadt, Germany, May 19–23, 2019. Springer, Cham, Switzerland.
28. Dung Bui and Geoffroy Couteau. Improved private set intersection for sets with small entries. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II*, volume 13941 of *Lecture Notes in Computer Science*, pages 190–220, Atlanta, GA, USA, May 7–10, 2023. Springer, Cham, Switzerland.
29. Dung Bui, Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Fast public-key silent OT and more from constrained Naor-Reingold. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024, Part VI*, volume 14656 of *Lecture Notes in Computer Science*, pages 88–118, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
30. Dario Catalano, Dario Fiore, and Emanuele Giunta. Adaptively secure single secret leader election from DDH. In Alessia Milani and Philipp Woelfel, editors, *41st ACM Symposium Annual on Principles of Distributed Computing*, pages 430–439, Salerno, Italy, July 25–29, 2022. Association for Computing Machinery.
31. Dario Catalano, Dario Fiore, and Emanuele Giunta. Efficient and universally composable single secret leader election from pairings. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13940 of *Lecture Notes in Computer Science*, pages 471–499, Atlanta, GA, USA, May 7–10, 2023. Springer, Cham, Switzerland.

32. Octavian Catrina and Amitabh Saxena. Secure computation with fixed-point numbers. In Radu Sion, editor, *FC 2010: 14th International Conference on Financial Cryptography and Data Security*, volume 6052 of *Lecture Notes in Computer Science*, pages 35–50, Tenerife, Canary Islands, Spain, January 25–28, 2010. Springer, Berlin, Heidelberg, Germany.
33. David Chaum. The dining cryptographers problem: Unconditional sender and recipient untraceability. *Journal of Cryptology*, 1(1):65–75, January 1988.
34. Ilaria Chillotti, Emmanuela Orsini, Peter Scholl, Nigel P. Smart, and Barry van Leeuwen. Scooby: Improved multi-party homomorphic secret sharing based on FHE. In Clemente Galdi and Stanislaw Jarecki, editors, *SCN 22: 13th International Conference on Security in Communication Networks*, volume 13409 of *Lecture Notes in Computer Science*, pages 540–563, Amalfi, Italy, September 12–14, 2022. Springer, Cham, Switzerland.
35. Miranda Christ, Kevin Choi, Walter McKelvie, Joseph Bonneau, and Tal Malkin. Accountable secret leader election. In *6th Conference on Advances in Financial Technologies (AFT 2024)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
36. Gil Cohen, Ivan Bjerre Damgård, Yuval Ishai, Jonas Kölker, Peter Bro Miltersen, Ran Raz, and Ron D. Rothblum. Efficient multiparty protocols via log-depth threshold formulae - (extended abstract). In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013, Part II*, volume 8043 of *Lecture Notes in Computer Science*, pages 185–202, Santa Barbara, CA, USA, August 18–22, 2013. Springer, Berlin, Heidelberg, Germany.
37. Henry Corrigan-Gibbs, Dan Boneh, and David Mazières. Riposte: An anonymous messaging system handling millions of users. In *2015 IEEE Symposium on Security and Privacy*, pages 321–338, San Jose, CA, USA, May 17–21, 2015. IEEE Computer Society Press.
38. Geoffroy Couteau, Lalita Devadas, Srinivas Devadas, Alexander Koch, and Sacha Servan-Schreiber. QuietOT: Lightweight oblivious transfer with a public-key setup. In *Advances in Cryptology – ASIACRYPT 2024, Part II*, *Lecture Notes in Computer Science*, pages 197–231. Springer, Singapore, Singapore, December 7–11, 2024.
39. Geoffroy Couteau and Naman Kumar. 10-party sublinear secure computation from standard assumptions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024, Part IX*, volume 14928 of *Lecture Notes in Computer Science*, pages 39–73, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
40. Geoffroy Couteau, Naman Kumar, and Xiayi Ye. Multiparty Homomorphic Secret Sharing and More from LPN and MQ. *Cryptology ePrint Archive*, 2025.
41. Geoffroy Couteau, Pierre Meyer, Alain Passelègue, and Mahshid Riahinia. Constrained pseudorandom functions from homomorphic secret sharing. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 194–224, Lyon, France, April 23–27, 2023. Springer, Cham, Switzerland.
42. Ronald Cramer, Ivan Damgård, and Yuval Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In Joe Kilian, editor, *TCC 2005: 2nd Theory of Cryptography Conference*, volume 3378 of *Lecture Notes in Computer Science*, pages 342–362, Cambridge, MA, USA, February 10–12, 2005. Springer, Berlin, Heidelberg, Germany.
43. Ivan Damgård, Matthias Fitzi, Eike Kiltz, Jesper Buus Nielsen, and Tomas Toft. Unconditionally secure constant-rounds multi-party computation for equality, comparison, bits and exponentiation. In Shai Halevi and Tal Rabin, editors, *TCC 2006: 3rd Theory of Cryptography Conference*, volume 3876 of *Lecture Notes in Computer Science*, pages 285–304, New York, NY, USA, March 4–7, 2006. Springer, Berlin, Heidelberg, Germany.
44. Quang Dao, Yuval Ishai, Aayush Jain, and Huijia Lin. Multi-party homomorphic secret sharing and sublinear mpc from sparse lpn. In *Annual International Cryptology Conference*, pages 315–348. Springer, 2023.
45. Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. *LIPICs*, pages 5:1–5:24. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik, 2021.
46. Yevgeniy Dodis, Shai Halevi, Ron D. Rothblum, and Daniel Wichs. Spooky encryption and its applications. In Matthew Robshaw and Jonathan Katz, editors, *Advances in Cryptology – CRYPTO 2016, Part III*, volume 9816 of *Lecture Notes in Computer Science*, pages 93–122, Santa Barbara, CA, USA, August 14–18, 2016. Springer, Berlin, Heidelberg, Germany.
47. Saba Eskandarian, Henry Corrigan-Gibbs, Matei Zaharia, and Dan Boneh. Express: Lowering the cost of metadata-hiding communication with cryptographic privacy. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021: 30th USENIX Security Symposium*, pages 1775–1792. USENIX Association, August 11–13, 2021.
48. Sebastian Faust, Carmit Hazay, David Kretzler, Leandro Rometsch, and Benjamin Schlosser. Non-interactive threshold bbs+ from pseudorandom correlations. In *Cryptographers’ Track at the RSA Conference*, pages 198–222. Springer, 2025.

49. Nelly Fazio, Rosario Gennaro, Tahereh Jafarikhah, and William E Skeith. Homomorphic secret sharing from paillier encryption. In *Provable Security: 11th International Conference, ProvSec 2017, Xi'an, China, October 23-25, 2017, Proceedings 11*, pages 381–399. Springer, 2017.
50. Ximing Fu, Mo Li, Shihan Lyu, and Chuanyi Liu. Bit-fixing correlation attacks on Goldreich’s pseudorandom generators. Cryptology ePrint Archive, Report 2024/1594, 2024.
51. Niv Gilboa and Yuval Ishai. Distributed point functions and their applications. In Phong Q. Nguyen and Elisabeth Oswald, editors, *Advances in Cryptology – EUROCRYPT 2014*, volume 8441 of *Lecture Notes in Computer Science*, pages 640–658, Copenhagen, Denmark, May 11–15, 2014. Springer, Berlin, Heidelberg, Germany.
52. Aarushi Goel, Mingyuan Wang, and Zhiheng Wang. Multiparty distributed point functions. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 140–173, Cham, 2025. Springer Nature Switzerland.
53. Philippe Golle and Ari Juels. Dining cryptographers revisited. In Christian Cachin and Jan Camenisch, editors, *Advances in Cryptology – EUROCRYPT 2004*, volume 3027 of *Lecture Notes in Computer Science*, pages 456–473, Interlaken, Switzerland, May 2–6, 2004. Springer, Berlin, Heidelberg, Germany.
54. Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Sharing transformation and dishonest majority MPC with packed secret sharing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part IV*, volume 13510 of *Lecture Notes in Computer Science*, pages 3–32, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
55. Martin Hirt and Ueli Maurer. Player simulation and general adversary structures in perfect multiparty computation. *J. Cryptol.*, 13(1):31–60, January 2000.
56. Zhicong Huang, Wen jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure two-party deep neural network inference. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022: 31st USENIX Security Symposium*, pages 809–826, Boston, MA, USA, August 10–12, 2022. USENIX Association.
57. Yuval Ishai, Mahimna Kelkar, Varun Narayanan, and Liav Zafar. One-message secure reductions: On the cost of converting correlations. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 515–547, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
58. Yashvanth Kondi, Claudio Orlandi, and Lawrence Roy. Two-round stateless deterministic two-party Schnorr signatures from pseudorandom correlation functions. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology – CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 646–677, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
59. Zachary Newman, Sacha Servan-Schreiber, and Srinivas Devadas. Spectrum: High-bandwidth anonymous broadcast. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 229–248, 2022.
60. Claudio Orlandi, Peter Scholl, and Sophia Yakoubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 678–708, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
61. Srinivasan Raghuraman and Peter Rindal. Blazing fast PSI from improved OKVS and subfield VOLE. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 2505–2517, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
62. Mayank Rathee, Conghao Shen, Sameer Wagh, and Raluca Ada Popa. ELSA: Secure aggregation for federated learning with malicious actors. In *2023 IEEE Symposium on Security and Privacy*, pages 1961–1979, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press.
63. Peter Rindal and Phillipp Schoppmann. VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part II*, volume 12697 of *Lecture Notes in Computer Science*, pages 901–930, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
64. Lawrence Roy and Jaspal Singh. Large message homomorphic secret sharing from DCR and applications. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part III*, volume 12827 of *Lecture Notes in Computer Science*, pages 687–717, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
65. Adi Shamir. How to share a secret. *Communications of the Association for Computing Machinery*, 22(11):612–613, November 1979.
66. Chris Studholme and Ian Blake. Multiparty computation to generate secret permutations. Cryptology ePrint Archive, Report 2007/353, 2007.
67. The OpenSSL Project. OpenSSL cryptography and SSL/TLS toolkit. <https://www.openssl.org/>, 2024. Accessed: 2024-02-12.

- 68. John Von Neumann et al. Various techniques used in connection with random digits. *John von Neumann, Collected Works*, 5:768–770, 1963.
- 69. Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091, San Francisco, CA, USA, May 24–27, 2021. IEEE Computer Society Press.
- 70. Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. QuickSilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In Giovanni Vigna and Elaine Shi, editors, *ACM CCS 2021: 28th Conference on Computer and Communications Security*, pages 2986–3001, Virtual Event, Republic of Korea, November 15–19, 2021. ACM Press.
- 71. Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020: 27th Conference on Computer and Communications Security*, pages 1607–1626, Virtual Event, USA, November 9–13, 2020. ACM Press.

Supplementary Materials

Table of Contents

On Compressing Non-Additive Correlations	1
Geoffroy Couteau ¹ , Alexander Koch ² , Nikolas Melissaris ¹ , Peter Scholl ³ , Sacha Servan-Schreiber ⁴ , and Xiaxi Ye ⁵	
1 Introduction	2
1.1 The additive barrier	2
1.2 Our contributions	3
1.3 Applications	4
2 Technical Overview	5
2.1 Background	5
2.2 Pseudorandom permutation correlations and $\binom{n}{1}$ -HSS	6
2.3 PCG for $\binom{n}{t}$ secret sharing	8
3 Preliminaries	10
3.1 Homomorphic Secret Sharing	10
3.2 Function Secret Sharing and Distributed Point Functions	12
3.3 Pseudorandom Correlation Generators	13
3.4 Pseudorandom Correlation Functions	14
4 PCF for Permutations	14
5 Multiparty PCG for Correlations with Threshold Secret Sharings	17
5.1 Threshold secret sharing schemes	17
5.2 Multiparty DPF, PCG, HSS with threshold secret sharing Scheme	19
5.3 $\binom{n}{t}$ -PCG for constant-degree polynomials from additive DPF: constant n and t	21
5.4 $\binom{n}{t}$ -PCG via replicated secret sharings: polynomial n and constant $n - t$	25
5.5 $\binom{n}{t}$ -PCG via party emulation: polynomial n and constant t	28
6 Applications of Pseudorandom Permutations	30
6.1 Anonymous broadcast via DC-nets	30
6.2 Single Secret Leader Election	32
A Definition of Pseudorandom Correlation Functions	38
B Constructing a PCG for ϵ -Biased Permutations	39
B.1 High-Level Overview	39
B.2 Additional Background	40
B.3 Permutation PCG From PCG for Biased Bits	41
B.4 Instantiating the (1/6)-Biased Bits PCG from Quasi-Abelian Syndrome Decoding	43
B.5 Implementation and parameters	46
B.6 Benchmarks	47
B.7 Performance Estimate for the Unbiased Construction	47

A Definition of Pseudorandom Correlation Functions

Here, we include the formal definition of a PCF for $\binom{n}{t}$ threshold security.

Definition 26 (Pseudorandom Correlation Function [18]). *Let λ be a security parameter, $\mathcal{C}$ be a reverse-sampleable n -correlation with output length $\ell(\lambda) \in \text{poly}(\lambda)$, and $\lambda \leq k = k(\lambda) \in \text{poly}(\lambda)$ be an input length. A Pseudorandom Correlation Function (PCF) for $\mathcal{C}$ is defined by a pair of algorithms (PCF.KeyGen, PCF.Eval) with the following functionality:*

- PCF.KeyGen(1^λ) $\rightarrow$ $(k_1, \dots, k_n)$. Takes as input the security parameter λ . Outputs an n -tuple of keys $(k_1, \dots, k_n)$.

$\text{Exp}_{\mathcal{A},N,0}^{\text{pr}}(\lambda):$ $\text{for } i = 1 \text{ to } N(\lambda):$ $x_i \xleftarrow{\mathcal{R}} \{0,1\}^\ell$ $(R_1^{(i)}, \dots, R_n^{(i)}) \leftarrow \mathcal{C}(1^\lambda)$ $b' \leftarrow \mathcal{A}(1^\lambda, (x_i, R_1^{(i)}, \dots, R_n^{(i)})_{i \in [N(\lambda)]})$ $\text{return } b'$	$\text{Exp}_{\mathcal{A},N,1}^{\text{pr}}(\lambda):$ $(k_1, \dots, k_n) \leftarrow \text{PCF.KeyGen}(1^\lambda)$ $\text{for } i = 1 \text{ to } N(\lambda):$ $x_i \xleftarrow{\mathcal{R}} \{0,1\}^\ell$ $\text{for } \sigma \in [n]:$ $R_\sigma^{(i)} \leftarrow \text{PCF.Eval}(\sigma, k_\sigma, x_i)$ $b' \leftarrow \mathcal{A}(1^\lambda, (x_i, R_1^{(i)}, \dots, R_n^{(i)})_{i \in [N(\lambda)]})$ $\text{return } b'$
--	---

Fig. 3: Pseudorandom $\mathcal{C}$ -correlated outputs of a PCF.

$\text{Exp}_{\mathcal{A},N,T,0}^{\text{sec}}(\lambda):$ $(k_1, \dots, k_n) \leftarrow \text{PCF.KeyGen}(1^\lambda)$ $\text{for } i = 1 \text{ to } N(\lambda):$ $x_i \xleftarrow{\mathcal{R}} \{0,1\}^\ell$ $\text{foreach } \sigma \in T:$ $R_\sigma^{(i)} \leftarrow \text{PCF.Eval}(\sigma, k_\sigma, x_i)$ $(R_\sigma^{(i)})_{\sigma \in \bar{T}} \leftarrow \text{RSample}(1^\lambda, T, (R_\sigma^{(i)})_{\sigma \in T})$ $b' \leftarrow \mathcal{A}(1^\lambda, (k_\sigma)_{\sigma \in T}, (x_i, (R_\sigma^{(i)})_{\sigma \in \bar{T}})_{i \in [N(\lambda)]})$ $\text{return } b'$	$\text{Exp}_{\mathcal{A},N,T,1}^{\text{sec}}(\lambda):$ $(k_1, \dots, k_n) \leftarrow \text{PCF.KeyGen}(1^\lambda)$ $\text{for } i = 1 \text{ to } N(\lambda):$ $x_i \xleftarrow{\mathcal{R}} \{0,1\}^\ell$ $\text{foreach } \bar{\sigma} \in \bar{T}:$ $R_{\bar{\sigma}}^{(i)} \leftarrow \text{PCF.Eval}(\bar{\sigma}, k_{\bar{\sigma}}, x_i)$ $b' \leftarrow \mathcal{A}(1^\lambda, (k_\sigma)_{\sigma \in T}, (x_i, (R_{\bar{\sigma}}^{(i)})_{\bar{\sigma} \in \bar{T}})_{i \in [N(\lambda)]})$ $\text{return } b'$
--	---

Fig. 4: $(n)_t$ -security game for a PCF, with a size- $\leq t$ subset $T \subset [n]$ of corrupted parties. Here, RSample is the algorithm for reverse sampling the correlation $\mathcal{C}$, as defined in Definition 7.

- $\text{PCF.Eval}(\sigma, k_\sigma, x) \rightarrow R_\sigma$. Takes as input $\sigma \in [n]$, a key k_σ , and input string $x \in \{0,1\}^k$. Outputs a string $R_\sigma \in \{0,1\}^\ell$.

We say $\text{PCF} = (\text{PCF.KeyGen}, \text{PCF.Eval})$ is a (weak) PCF for the correlation $\mathcal{C}$, if the following properties hold:

Correctness / Pseudorandom $\mathcal{C}$ -correlated outputs. For every $\sigma \in [n]$, all efficient adversaries $\mathcal{A}$, and all $N \in \text{poly}(\lambda)$, there exists a negligible function negl such that for all sufficiently large λ ,

$$\left| \Pr[\text{Exp}_{\mathcal{A},N,0}^{\text{pr}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A},N,1}^{\text{pr}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A},N,b}^{\text{pr}}(\lambda)$, for $b \in \{0,1\}$, is as defined in Figure 3. In particular, the adversary is given access to $N(\lambda)$ samples.

Security. For all $T \subset [n]$ and all efficient adversaries $\mathcal{A}$, there exists a negligible function negl such that for all sufficiently large λ ,

$$\left| \Pr[\text{Exp}_{\mathcal{A},N,T,0}^{\text{sec}}(\lambda) = 1] - \Pr[\text{Exp}_{\mathcal{A},N,T,1}^{\text{sec}}(\lambda) = 1] \right| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\mathcal{A},N,T,b}^{\text{sec}}(\lambda)$, for $b \in \{0,1\}$, is as defined in Figure 4 (again, with the adversary given $N(\lambda)$ samples).

B Constructing a PCG for ϵ -Biased Permutations

In this section, we present our 3-party PCG for ϵ -biased permutations, with $\epsilon = 1/6$, based on the quasi-abelian syndrome decoding assumption.

B.1 High-Level Overview

Recall the template in Section 2.2: for Alice, Bob and Carol to obtain a permutation, it suffices for Alice and Bob to obtain random, additive shares of a random bit μ , while Alice and Carol

obtain shares of the complement, $1 - \mu$, in such a way that Alice’s share has the *same* value in both instances.

New tool: “Doubly-programmable” PCG for random shared bits. To enable this distribution of shares, we introduce a form a two-party PCG with a new, programmable property. Previous works [17, 19] used a programmable property that allows to generate new PCG seeds that expand to give the same (or, a related), additively shared *value*, as the original seeds, albeit with fresh shares. We extend this to “one-more-level” of programmability by making the seed (resp. key) of one party sampled independently of the correlation. This makes it possible to fix a seed/key k_0 and then generate multiple keys k_1, k_2 for two different programmable correlations $\mathcal{C}_1$ and $\mathcal{C}_2$, respectively. More concretely, coming back to our use case of pseudorandom shared bits, Alice can be given k_0 which allows her to generate a pseudorandom share x while Bob and Carol can be given k_1 and k_2 , respectively, such that:

$$\text{Expand}(k_0) - \text{Expand}(k_1) = \mu \text{ and } \text{Expand}(k_0) - \text{Expand}(k_2) = 1 - \mu.$$

Formally, we define such “doubly-programmable” PCGs and PCFs as being $\mathcal{F}$ -programmable, where (1) one of the seeds/keys can be sampled independently of the ultimate correlation and (2) the correlation itself can be programmed to be some function of the source of pseudorandomness. (In the above example, the programmed function in k_2 maps x to $1 - x$.)

Challenges in generating bits over a field. Generating a pseudorandom bit shared over a larger field is useful for efficient multi-party computation protocols [32, 43, 57] as it enables efficient conversion of secret shares from one field to another. However, despite this usefulness, there are no constructions of PCGs/PCFs for this target correlation, beyond ones that evaluate a PRG or PRF inside HSS to obtain a stream of pseudorandom, shared bits.

To avoid relying on high-degree HSS, we observe that in the case of $\mathbb{F}_3$, a shared, random bit can be obtained from just one OLE correlation in $\mathbb{F}_3$, provided we settle for shares of a *biased* bit in $\mathbb{F}_3$. In particular, consider an OLE correlation $((x_0, y_0), (x_1, y_1))$, where $y_1 = y_0 + x_0 x_1 \bmod 3$. The two parties can define shares of x^2 , where $x := x_0 + x_1$, by letting $z_0 := x_0^2 - 2y_0$ and $z_1 := x_1^2 + 2y_1$. We then have, $z_0 + z_1 = x_0^2 + x_1^2 + 2x_0 x_1 = x^2$. If x is uniform in $\mathbb{F}_3$, then a simple case analysis shows that x^2 is a bit that equals 1 with probability $2/3$, and 0 otherwise.

This observation forms the backbone of our PCG construction, which we show can be instantiated by building off of existing PCGs for OLE correlations in $\mathbb{F}_3$.

In this section, we present the framework for building a PCG/PCF for pseudorandom permutations from a PCG/PCF for pseudorandom bits and then instantiate it from the QA-SD assumption and HSS. In Appendix B.2, we first describe the $\mathcal{F}$ -programmability definition that we will make use of in our constructions.

B.2 Additional Background

Programmable FSS. We use 2-party FSS schemes with a special *programmability* property, meaning that one of the two keys can be sampled independently of the function f being shared. The following definition extends the concept of programmable distributed point function [25] to programmable FSS for general functions.

Definition 27 (Programmable FSS). *We say that an FSS scheme $(\text{FSS.Gen}, \text{FSS.Eval})$ is programmable if FSS.Gen can be split into two algorithms Gen_0 and Gen_1 with the following syntax:*

- $\text{Gen}_0(1^\lambda, I, \mathbb{G}) \rightarrow k_0$. *On input the security parameter and input/output domains of the function f_λ , this samples a key k_0 .*
- $\text{Gen}_1(k_0, f_\lambda) \rightarrow k_1$. *on input key k_0 and the function f_λ , this generates a key k_1 .*

It is easy to see that a programmable DPF immediately also implies programmable FSS for sums of t point functions.

Boyle et al. [25] construct a programmable distributed point function for any domain size $N = N(\lambda) \in \text{poly}(\lambda)$ based on a pseudorandom generator. Their construction starts by building a simple and efficient, but *leaky*, programmable DPF, and then upgrades this to a fully secure construction through privacy amplification techniques. While the final construction is not practical, in ?? we show how to leverage the practical, leaky version of their DPF in our construction to build a concretely efficient $\mathcal{F}$ -programmable PCG.

Doubly-Programmable PCGs. We now introduce our notion of doubly-programmable PCGs. We consider the following class of two-party correlations, where the parties' outputs form subtractive shares of some random variable.

Definition 28 (Two-party subtractive correlation generator). Let $(\mathbb{G}, +)$ be an abelian group and $\ell, n \in \mathbb{Z}$ be functions of λ . Further, let $\mathcal{F} = \{f_\lambda: \mathbb{G}^\ell \rightarrow \mathbb{G}^n\}_\lambda$ be a family of functions. A subtractive correlation generator for $\mathcal{F}$ is a correlation generator $\mathcal{C}_\mathcal{F}$, such that $\mathcal{C}_\mathcal{F}(1^\lambda)$ uniformly samples $X \leftarrow \mathbb{F}^\ell$ and outputs $(R_0, R_1) \in \mathbb{F}^n \times \mathbb{F}^n$, such that R_0 and R_1 are distributed uniformly, conditioned on $R_0 - R_1 = f_\lambda(X)$.

A doubly-programmable PCG allows PCG keys to be generated in a special way, such that both (1) the randomness $f_\lambda(X)$, and (2) the R_0 output of party 0, can be reused across multiple instances. Furthermore, we allow a small “tweak” to be applied to $f_\lambda(X)$ such that it can be modified slightly when reusing it. We formalise this notion by using a special seed from which the randomness in $f_\lambda(X)$ is derived, and an additional input to party 1's key generation algorithm for performing the tweak specified by some function from a family $\mathcal{F}$.

Definition 29 ($\mathcal{F}$ -Programmable). A two-party PCG (resp. PCF) for an additive correlation is $\mathcal{F}$ -programmable for a class of functions $\mathcal{F}$, if PCG.Gen (resp. PCF.Gen) can be split into two distinct algorithms:

- $\text{Gen}_0(1^\lambda)$: a PPT algorithm that on input 1^λ , outputs a key $k_0 \in \{0, 1\}^*$
- $\text{Gen}_1(k_0, \text{seed}, f)$: a deterministic algorithm that on input a key k_0 , $\text{seed} \in \{0, 1\}^\lambda$ and function $f \in \mathcal{F}$, samples and outputs a second key k_1 .

We require the following correctness property, for any $f \in \mathcal{F}$:

$$\Pr \left[\begin{array}{l} \text{seed} \xleftarrow{\$} \{0, 1\}^\lambda \\ k_0 \xleftarrow{\$} \text{PCG.Gen}_0(1^\lambda) \\ R_0 - R_2 = f(R_0 - R_1) : \quad \begin{array}{l} k_1 \leftarrow \text{PCG.Gen}_1(k_0, \text{seed}, \text{id}) \\ k_2 \leftarrow \text{PCG.Gen}_1(k_0, \text{seed}, f) \\ R_j \leftarrow \text{PCG.Expand}(k_j), \quad j \in \{0, 1, 2\} \end{array} \end{array} \right] \geq 1 - \text{negl}(\lambda)$$

The PCG.Gen (resp. PCF.Gen) algorithm then simply runs Gen_0 , followed by Gen_1 on input a random seed and f as the identity function, to output a pair of keys (k_0, k_1) .

B.3 Permutation PCG From PCG for Biased Bits

We now show how to obtain a PCG for biased, three-party permutation correlations, given a special form of two-party PCG for secret-shared, biased random bits.

Definition 30 (Two-party, ε -biased bits correlation generator). An ε -biased bits correlation generator is a subtractive correlation generator $\mathcal{C}$ (from Definition 28), such that $\mathcal{C}(1^\lambda)$ samples a vector $\mathbf{x} \leftarrow \{0, 1\}^\ell$ where each element x_i is sampled from $\text{Bernoulli}(1/2 + \varepsilon)$ and outputs $(\mathbf{R}_0, \mathbf{R}_1) \in \mathbb{Z}_q^\ell \times \mathbb{Z}_q^\ell$, such that the $\mathbf{R}_i$'s are distributed uniformly, conditioned on $\mathbf{R}_0 - \mathbf{R}_1 = \mathbf{x}$.

Definition 31 ($\mathcal{D}$ -biased, n -party permutation correlation generator). A $\mathcal{D}$ -biased, n -party permutation correlation generator is a correlation generator that samples ℓ permutations $(\pi_1, \dots, \pi_\ell) \leftarrow \mathcal{D}^\ell$ where $\mathcal{D}$ is a distribution on the set of all permutations on n elements, S_N . It outputs $\mathbf{R}_1, \dots, \mathbf{R}_n$, where $\mathbf{R}_i(j) = \pi_j[i]$, meaning that in the j -th position of $\mathbf{R}_i$ there is the i -th element of permutation π_j .

Lemma 32. Let $\ell, n \in \mathbb{N}$ and $\mathcal{C}$ be a $\mathcal{D}$ -biased, n -party permutation correlation generator, for some efficiently sampleable distribution $\mathcal{D}$ on S_ℓ . Then, $\mathcal{C}$ is a reverse-sampleable $\binom{n}{t}$ correlation generator for any $t < n$.

Proof. For this, we need to show that there is an efficient algorithm **RSample** that takes as input a party subset $T \subset [n]$ and their correlation samples $R_\sigma =: (e_{\sigma,1}, \dots, e_{\sigma,\ell})$ with $e_i \in [n]$, and outputs a list of samples $(R_i)_{i \in \overline{T}}$ that fulfills Definition 7. **RSample** proceeds by rejection sampling of $\mathcal{D}$ as follows. For each $j \in [\ell]$, it repeatedly samples a permutation π_j using $\mathcal{D}$, until $\pi_j(i) = e_i$ for all $i \in T$. Note that the probability for each permutation assigned by $\mathcal{D}$ is constant, so this process is efficient. Also, clearly, the output is identically distributed to the output of $\mathcal{C}$. $\blacksquare$

When we build a $\mathcal{D}$ -biased correlation generator based on ε -biased bits, this will give the following distribution $\mathcal{D}_\varepsilon$ on S_3 , the set of permutations on $\mathbb{F}_3$. It builds a permutation π by randomly sampling a biased bit $\mu \xleftarrow{R} \text{Bernoulli}(1/2 + \varepsilon)$, and one uniformly random index $i \xleftarrow{R} \mathbb{F}_3$ for $\pi[0]$, and then from the remaining two choices $\mathbb{F}_3 \setminus \{i\}$, assign the smaller one to $\pi[1]$, if $\mu = 1$, and the larger one, otherwise. Assign to $\pi[2]$ the remaining option.

Theorem 33. *Suppose that $(\text{PRBits.Gen}_0, \text{PRBits.Gen}_1, \text{PRBits.Expand})$ is an $\mathcal{F}$ -programmable PCG for ε -biased bits for the function class $\mathcal{F} = \{\text{id}, (x \mapsto -x)\}$, for some $\varepsilon \in [0, 1/2]$. Then, the construction in Figure 5 is a PCG for $\mathcal{D}_\varepsilon$ -biased, three-party permutation correlations, where $\mathcal{D}_\varepsilon$ is the distribution on S_3 given above.*

Proof. We argue each property in turn:

Correctness. Let (k_0, k_1, k_2) be as output by $\text{PCG.Gen}(1^\lambda)$, and $R_i \leftarrow \text{PCG.Expand}(i, k_i)$, i.e. define $R'_i = \text{PRBits.Expand}(i, k_i)$ for $i \in \{0, 1, 2\}$ and then, we have $R_0 = R'_0$, $R_1 = (2, \dots, 2) + R'_1$, $R_2 = (1, \dots, 1) + R'_2$.

Now, the R'_i denote the original subtractive shares of the bit vectors as output by $\text{PRBits.Expand}(i, k_i)$. Then, by construction, $R'_0 - R'_1 = (\mu_1, \dots, \mu_\ell)$ forms a vector of ε -biased bits, and $R'_0 - R'_2 = (-\mu_1, \dots, -\mu_\ell)$, by $\mathcal{F}$ -programmability (cf. Definition 29). Let $k \in [\ell]$, then $R'_1[k] = R'_0[k] - \mu_k$ and $R'_2[k] = R'_0[k] + \mu_k$. Note that $R_0[k] + R_1[k] + R_2[k] = R'_0[k] + R'_1[k] + R'_2[k] = R'_0[k] + (R'_0[k] - \mu_k) + (R'_0[k] + \mu_k) = 0$. Hence, they form an additive sharing of zero for each k . Also, observe that $R_0[k], R_1[k], R_2[k]$ are pairwise distinct, hence they form a permutation.

Using the pseudorandomness of the PCG output, R_0 is a fresh random share in $\mathbb{F}_3$, and hence, R_0 is computationally close to uniform over $\mathbb{F}_3$, as in $\mathcal{D}_\varepsilon$. Then, the two possibilities for R_1 are exactly determined by the ε -biased bit as generated by **PRBits**. This shows that it is indistinguishable from a sample of $\mathcal{D}_\varepsilon$ for each $k \in [\ell]$.

Security.

We prove security via a sequence of hybrid arguments.

Hybrid $\mathcal{H}_0$. This hybrid corresponds to the $(k_i, (R_j)_{j \neq i})$ as computed in Figure 5.

Hybrid $\mathcal{H}_1$. In this hybrid, we reverse sample R'_j for $j \neq i$ from R'_i and compute $(R_j)_{j \neq i}$ as computed in Figure 5.

$\mathcal{H}_1 \approx_c \mathcal{H}_0$ by the security of the **PRBits** PCG.

Hybrid $\mathcal{H}_2$. In this hybrid, we change how the outputs $(R_j)_{j \neq i}$ are computed. In particular, we first sample uniformly random bits $(\mu_1, \dots, \mu_\ell)$. Then, we proceed in one of three cases: (1) if $i = 0$ we let $R_1 := R'_0 + 2 - (\mu_1, \dots, \mu_\ell)$ and let $R_2 := R'_0 + 1 + (\mu_1, \dots, \mu_\ell)$, (2) if $i = 1$ we let $R_0 := R'_1 - 2 + (\mu_1, \dots, \mu_\ell)$ and let $R_2 := R'_0 + 1 + (\mu_1, \dots, \mu_\ell)$, or (3) if $i = 2$ we let $R_1 := R'_0 + 2 - (\mu_1, \dots, \mu_\ell)$ and let $R_0 := R'_2 - 1 - (\mu_1, \dots, \mu_\ell)$.

$\mathcal{H}_2 \approx_c \mathcal{H}_1$ by the pseudorandomness property of the **PRBits** PCG.

Hybrid $\mathcal{H}_3$. In this hybrid, we reverse sample $(R_j)_{j \neq i}$.

$\mathcal{H}_3 \approx_c \mathcal{H}_2$. In particular, note that in $\mathcal{H}_2$, the keys k_j 's for $j \neq i$ are no longer used to compute the outputs. Therefore, by the correctness of the reverse sampling algorithm, the two hybrids are identical. $\blacksquare$

PCG for permutation correlations over $\mathbb{F}_3$

Gen(1^λ)

```

1 :  $k_0 \leftarrow \text{PRBits.Gen}_0(1^\lambda)$ 
2 :  $\text{seed} \xleftarrow{\mathbb{R}} \{0, 1\}^\lambda$ 
3 :  $k_1 \leftarrow \text{PRBits.Gen}_1(k_0, \text{seed}, f_1 = \text{id})$ 
4 :  $k_2 \leftarrow \text{PRBits.Gen}_1(k_1, \text{seed}, f_2 = (x \mapsto -x))$ 
5 : return  $(k_0, k_1, k_2)$ 

```

Expand(i, k_i)

```

1 :  $R'_i = \text{PRBits.Expand}(i, k_i)$ 
2 : if  $i = 0$  : return  $R_i := R'_i$ 
3 : if  $i = 1$  : return  $R_i := (2, \dots, 2) + R'_i$ 
4 : else return  $R_i := (1, \dots, 1) + R'_i$ 

```

Fig. 5: PCG for permutation correlations over $\mathbb{F}_3$

B.4 Instantiating the (1/6)-Biased Bits PCG from Quasi-Abelian Syndrome Decoding

We now show how to instantiate the previously described template, by giving a construction of a programmable PCG for ε -biased bits, where $\varepsilon = \frac{1}{6}$, based on the hardness of the quasi-abelian syndrome decoding problem [12].

B.4.1 QA-SD Background. The main motivation for quasi-abelian syndrome decoding is to obtain a syndrome decoding problem over a ring structure $\mathcal{R}$, where $\mathcal{R}$ is isomorphic to m copies of $\mathbb{F}_q$ for some large m . Using a cyclotomic polynomial ring $\mathcal{R} = \mathbb{F}_q[X]/f(X)$ as in ring-LWE or ring-LPN, this can be done only for $q = \Omega(m)$. Instead, quasi-abelian syndrome decoding allows us to use any $q \geq 3$.

Let $\mathbb{G}$ be a finite abelian group of order N . We define a ring $\mathcal{R}$ as the group algebra of $\mathbb{G}$ with coefficients in the finite field $\mathbb{F}_q$, that is, the set $\mathbb{F}_q[\mathbb{G}]$ of formal linear combinations:

$$\mathbb{F}_q[\mathbb{G}] := \left\{ \sum_{g \in \mathbb{G}} a_g g \mid a_g \in \mathbb{F}_q \right\}$$

Ring multiplication is defined by the convolution:

$$\left(\sum_{g \in \mathbb{G}} a_g g \right) \left(\sum_{g \in \mathbb{G}} b_g g \right) := \sum_{g \in \mathbb{G}} \left(\sum_{h \in \mathbb{G}} a_h b_{h^{-1}g} \right) g$$

The group algebra $\mathbb{F}_q[\mathbb{G}]$ is isomorphic to the vector space $\mathbb{F}_q^N$ via the map $\phi : \sum_{i=0}^{N-1} a_i g_i \mapsto (a_0, \dots, a_{N-1})$. This allows us to define the *weight* of an element $a \in \mathbb{F}_q[\mathbb{G}]$ as the Hamming weight of $\phi(a)$. (Note that the weight is independent of the ordering of the group $\mathbb{G}$ chosen in the definition of ϕ .)

Definition 34 (Decisional quasi-abelian syndrome decoding). *Given a target weight t and compression parameter c , let $\mathcal{D}_t(\mathbb{F}_q[\mathbb{G}])$ be a noise distribution that samples weight- t elements of $\mathbb{F}_q[\mathbb{G}]$. The decisional QASD $_{t,c}$ problem for $\mathcal{D}_t$ is to distinguish between the following two distributions with noticeable advantage:*

$$- (\mathbf{a}', u), \text{ where } \mathbf{a}' \xleftarrow{\mathbb{S}} \mathbb{F}_q[\mathbb{G}]^{c-1}, u \xleftarrow{\mathbb{S}} \mathbb{F}_q[\mathbb{G}]$$

– $(\mathbf{a}', \langle \mathbf{a}, \mathbf{e} \rangle)$, where $\mathbf{a} = (1, \mathbf{a}')$ and $\mathbf{a}' \xleftarrow{\$} \mathbb{F}_q[\mathbb{G}]^{c-1}$, $\mathbf{e} \xleftarrow{\$} \mathcal{D}_t(\mathbb{F}_q[\mathbb{G}])^c$

As in prior works [12], to improve efficiency we can use a *regular* noise distribution where each weight- t vector is divided into N/t consecutive blocks of weight-1.

Instantiating the group algebra. Following [12], we can instantiate $\mathbb{G}$ with a direct product of cyclic groups $\prod_{i=1}^N \mathbb{Z}/(q-1)\mathbb{Z}$. This implies that $\mathbb{F}_q[\mathbb{G}]$ is isomorphic to $\mathbb{F}_q[X_1, \dots, X_N]/(X_1^{q-1} - 1, \dots, X_N^{q-1} - 1)$ via an efficiently-computable isomorphism, and this is isomorphic to $\prod_{i=1}^{(q-1)^N} \mathbb{F}_q$.

Let $\text{QA-Eval}_{q,N} : \mathbb{F}_q[\mathbb{G}] \rightarrow \mathbb{F}_q^{(q-1)^N}$ be the map that takes an element of the group algebra, viewed as a multivariate polynomial in $\mathbb{F}_q[X_1, \dots, X_N]/(X_1^{q-1} - 1, \dots, X_N^{q-1} - 1)$, and evaluates it at every non-zero point. That is, $\text{QA-Eval}_{q,N}(f) = (f(x_1, \dots, x_N))_{(x_1, \dots, x_N) \in \{1, \dots, q-1\}^N}$. Since QA-Eval is a ring homomorphism, for any $f, g, h \in \mathbb{F}_q[\mathbb{G}]$, it holds that

$$\text{QA-Eval}_{q,N}(fg + h) = \text{QA-Eval}(f) * \text{QA-Eval}(g) + \text{QA-Eval}(h)$$

where $*$ denotes the entry-wise product. This allows us to directly convert additive shares of a product over $\mathbb{F}_q[\mathbb{G}]$ into a vector of shares of $(q-1)^N$ products over $\mathbb{F}_q$.

B.4.2 Construction. We want to give the parties shares of a random square $x^2 \in \mathbb{F}_3$. By choosing $x = \langle \mathbf{a}, \mathbf{e} \rangle$, as in the QASD assumption, then we can distribute shares of x^2 to two parties by giving out shares of the tensor product vector $\mathbf{e} \otimes \mathbf{e}$. Since each $e_i \in \mathbb{F}_q[\mathbb{G}]$ is t -sparse, the product $e_i e_j$ is at most t^2 -sparse, and can be shared using FSS for a sum of t^2 point functions. We need to also achieve the $\mathcal{F}$ -programmability feature, so that instead of shares of x^2 , we may additionally distributed shares of $f(x^2)$ for some function $f \in \mathcal{F}$. We will allow $\mathcal{F}$ to be the class of linear maps from $\mathbb{F}_q \rightarrow \mathbb{F}_q$ (applied to each coordinate of x^2 independently, viewing the group algebra as a vector space over $\mathbb{F}_q$). To do this, it suffices to tweak the non-zero values of each tensored noise terms $e_i e_j$ (denoted $\mathbf{b}^i \otimes \mathbf{b}^j$ in Figure 6), applying f to each entry. Since the shares of x^2 obtained in the Expand algorithm are computed as a linear function of the shares of the tensor product, this results in the same linear map being applied to x^2 .

Theorem 35. *Assume that SPFSS is a secure FSS scheme for sums of point functions and that the QA-SD assumption holds. Then, the protocol described in Figure 6 is an $1/6$ -biased bits correlation generator over $\mathbb{F}_3$.*

Proof. We prove each property in turn.

Correctness. We prove correctness via a hybrid argument.

Let $e^i = \sum_{j \in [0, t)} \mathbf{b}^i[j] \mathbf{A}^i[j]$.

Hybrid $\mathcal{H}_0$. In this hybrid, we define $z_0 = \langle \mathbf{a} \otimes \mathbf{a}, \mathbf{u}_0 \rangle$, $z_1 = \langle \mathbf{a} \otimes \mathbf{a}, \mathbf{u}_1 \rangle$ as computed in the real protocol execution and output (z_0, z_1) .

Hybrid $\mathcal{H}_1$. In this hybrid, instead of computing $\mathbf{u}_1$ with SPFSS.Eval , we replace $\mathbf{u}_1$ with $\mathbf{u}_0 - \mathbf{e} \otimes \mathbf{e}$ and compute $z_1 \leftarrow \langle \mathbf{a} \otimes \mathbf{a}, \mathbf{u}_0 - \mathbf{e} \otimes \mathbf{e} \rangle$

$\mathcal{H}_1 \approx_c \mathcal{H}_0$ by the SPFSS correctness property.

Hybrid $\mathcal{H}_2$. In this hybrid, we replace $\mathbf{u}_0$ with a randomly sampled $\tilde{\mathbf{u}}_0$. Compute $\mathbf{u}_1$ and z_0, z_1 as before using the new $\tilde{\mathbf{u}}_0$. We have $\mathbf{u}_1 = \tilde{\mathbf{u}}_0 - \mathbf{e} \otimes \mathbf{e}$, $z_0 = \langle \mathbf{a} \otimes \mathbf{a}, \tilde{\mathbf{u}}_0 \rangle$, and $z_1 = \langle \mathbf{a} \otimes \mathbf{a}, \tilde{\mathbf{u}}_0 - \mathbf{e} \otimes \mathbf{e} \rangle$

$\mathcal{H}_2 \approx_c \mathcal{H}_1$ by the pseudorandomness of the outputs of FSS.Eval (cf. Remark 5).

Hybrid $\mathcal{H}_3$. Here, we compute the output z_1 as $z_0 - \langle \mathbf{a}, \mathbf{e} \rangle^2$. This output is identically distributed to the previous hybrid, since in $\mathcal{H}_2$ we have:

$$z_1 = \langle \mathbf{a} \otimes \mathbf{a}, \tilde{\mathbf{u}}_0 \rangle - \langle \mathbf{a} \otimes \mathbf{a}, \mathbf{e} \otimes \mathbf{e} \rangle = z_0 - \langle \mathbf{a}, \mathbf{e} \rangle^2$$

Hybrid $\mathcal{H}_4$. Here, we sample $z_0, x \xleftarrow{\$} \mathbb{F}_q[\mathbb{G}]$ uniformly at random and compute $z_1 = z_0 - x^2$.

We have $\mathcal{H}_4 \approx_c \mathcal{H}_3$, due to the QASD assumption which says that $x = \langle \mathbf{a}, \mathbf{e} \rangle$ is indistinguishable from random.

Construction PRBits

Parameters: Noise weight $t = t(\lambda)$, compression factor c , field $\mathbb{F}_3$.
A programmable SPFSS scheme $\text{SPFSS} = (\text{SPFSS.Gen}_0, \text{SPFSS.Gen}_1, \text{SPFSS.Eval})$

Public Input: $c - 1$ vectors over $\mathbb{F}_3$, corresponding to the output of $\text{Eval}(a_i)$ for uniformly random $\mathbf{a} = (1, a_1, \dots, a_{c-1})$.

PRBits.Gen₀($1^\lambda, \text{seed}$)

- 1 : **foreach** $i, j \in [0, c)$:
- 2 : $(K_0^{i,j}) \xleftarrow{\$} \text{SPFSS.Gen}_0(1^\lambda)$
- 3 : Let $\mathbf{k}_0 = ((K_0^{i,j})_{i,j \in [0,c)})$
- 4 : Output $\mathbf{k}_0$

PRBits.Gen₁($\text{seed}, \mathbf{k}_0, f$)

- 1 : **foreach** $i \in [0, c)$:
- 2 : $\mathbf{A}^i \xleftarrow{\$} (g_1, \dots, g_t)_{g_i \in \mathbb{G}}, \quad \mathbf{b}^i \xleftarrow{\$} (\mathbb{F}_3^*)^t \quad // \text{ Using randomness from seed}$
- 3 : **foreach** $i, j \in [0, c)$:
- 4 : $(K_1^{i,j}) \xleftarrow{\$} \text{SPFSS.Gen}_1(1^\lambda, \mathbf{A}^i \otimes \mathbf{A}^j, f(\mathbf{b}^i \otimes \mathbf{b}^j))$
- 5 : Let $\mathbf{k}_1 = ((K_1^{i,j})_{i,j \in [0,c)})$
- 6 : Output $\mathbf{k}_1$

PRBits.Expand($\sigma, \mathbf{k}_\sigma$)

- 1 : For $i, j \in [0, c)$:
- 2 : $u_{\sigma, i+cj} \leftarrow \text{SPFSS.Eval}(\sigma, K_\sigma^{i,j})$
- 3 : $z_\sigma \leftarrow \langle \mathbf{a} \otimes \mathbf{a}, \mathbf{u}_\sigma \rangle$
- 4 : Output $\text{QA-Eval}_{3,N}(z_\sigma)$

Fig. 6: PCG for biased bits over $\mathbb{F}_3$

Finally, we observe that this distribution is equivalent to a sample from the ideal $\frac{1}{6}$ -biased bits correlation generator, as each $\mathbb{F}_3$ component of x^2 is zero with probability $\frac{1}{3}$ and one with probability $\frac{2}{3}$.

Security. Security is argued as a sequence of hybrid arguments. We use $\sigma = 1$ and note that the case for $\sigma = 0$ is symmetric. We want to show that:

$$\{(k_1, x, z_0)\} \approx_c \left\{ (k_1, \tilde{x}, \tilde{z}_0) \mid \begin{array}{l} \tilde{x} \xleftarrow{\$} \mathbb{F}_3 \\ \tilde{z}_0 = \tilde{x}^2 + z_1 \end{array} \right\}.$$

Hybrid $\mathcal{H}_0$. This hybrid corresponds to the (k_1, x, z_0) computed as in the real protocol.

Hybrid $\mathcal{H}_1$. In this hybrid, we replace all the FSS keys K_1^{ij} in k_1 with simulated keys.

$\mathcal{H}_0 \approx_c \mathcal{H}_1$ via a straightforward hybrid argument replacing each of the FSS keys one-by-one with a simulated key, and relying on the correctness and security of the FSS scheme.

Hybrid $\mathcal{H}_2$. In this hybrid, we sample $\tilde{x}$ at random instead of computing it using the FSS key K_1^{ij} .

$\mathcal{H}_1 \approx_c \mathcal{H}_2$ by the QA-SD assumption.

Hybrid $\mathcal{H}_3$. In this hybrid, we undo the changes made in $\mathcal{H}_1$ and switch back to the original FSS keys.

$\mathcal{H}_3 \approx_c \mathcal{H}_2$ via a straightforward hybrid argument replacing each of the FSS keys one-by-one, and relying on the correctness and security of the FSS scheme.

At this point, we have that $\mathcal{H}_3$ is distributed exactly as $(k_1, \tilde{x}, \tilde{z}_0)$, concluding the proof.

Programmability. The construction is programmable (that is, we have $z_0 - z_2 = f(z_0 - z_1)$) due to the programmability of the FSS. In particular, this follows from the fact that by scaling the non-zero values for the point functions defined by $\mathbf{b}^i \otimes \mathbf{b}^i$ by some constant $\Delta \in \mathbb{F}_q$ at key generation time, the secret shared output becomes a share of $\Delta \cdot x^2$ instead of x^2 . The programmability of the FSS ensures that we then obtain the necessary $\mathcal{F}$ -programmability feature, since we can keep the share z_0 independent (generated by Gen_0) and get $z_0 - z_2 = \Delta(z_0 - z_1)$. This immediately generalizes to linear functions f .

In addition, as was discussed earlier, for random $x \in \mathbb{F}_3$, x^2 is 1 for $x = 1, 2$ (2/3 of the times) and 0 for $x = 0$ (1/3 of the times). This means that x^2 is distributed according to Bernoulli(1/2+1/6) over $\mathbb{F}_3$ and so our construction is a 1/6-biased bits correlation generator over $\mathbb{F}_3$. ■

B.5 Implementation and parameters

We implement the biased PCG construction in C by adapting the open-source implementation of the FOLEAGE [11] PCG for programmable OLE correlations. In particular, we modify their implementation by adapting it to work over $\mathbb{F}_3$ and use their parameter estimation script to derive a new set of parameters tailored to our construction. The parameters c (compression factor) and t (noise weight) influence the size of the PCG key and evaluation time. We set $c = 5$ and $t = 16$, which requires evaluating $(ct)^2 = 2916$ DPFs to compute PCG.Expand . The choice of $(c = 5, t = 16)$ corresponds to a conservative parameter choice based on the parameter estimation script of [11]. We then replace their use of DPFs with programmable DPFs, to match Figure 6. In particular, we use the $T = 40$, which gives us correctness $1 - 2^{-40}$.

Our programmable DPF implementation takes advantage of the AES-NI instruction to implement a fast PRG G using fixed-key AES (from the OpenSSL library [67]) and the Davies-Meyer transform. However, programmable DPFs do not allow for the optimizations exploited in the implementation of the FOLEAGE PCG [11] and are overall concretely more expensive to evaluate. In particular, the SPFSS (sum of many DPFs) evaluation accounts for 98% of the total computation time of the PCG expansion.

B.5.1 Recent attack on QA-SD parameters Very recently, a new attack has been demonstrated on the QA-SD assumption, which uses sparse multivariate interpolation algorithms to break various parameter sets that were proposed in practice [14]. We caution that our implementation, which uses parameters based on [11], is also vulnerable to the attack, and so insecure. To resist the attack, it would suffice to increase the parameter c from 5 to 8. While we have not yet been able to run our benchmarks with this change, we estimate it would incur roughly a factor-4 slowdown on the current results.

D	PCG.Expand	Key Size (per party)
2^{18}	4 minutes	0.71 MB
2^{20}	14 minutes	0.77 MB
2^{22}	48 minutes	0.84 MB

Table 2: Performance of our PCG implementation on a `c5.metal` EC2 instance. We set the noise parameter to $t = 16$ and set the compression factor $c = 5$, following the parameter selection script of [11].

B.6 Benchmarks

We run a set of benchmarks using AWS `c5.metal` (3.4GHz CPU). All experiments are averaged across ten trials and evaluated on a single core. We report the benchmark results in Table 2. The parameter $D = 2^n$ determines the number of bits we generate in total. Our choice of DPF range 2^{18} , 2^{20} , and 2^{22} correspond to the size of a regular noise block when $D = 2^{22}$, $D = 3^{24}$, and $D = 3^{26}$, respectively. Evaluating the PCG requires $(ct)^2$ calls to the DPF on domain size D/t (using the regular noise optimization described in [11]). The DPF evaluation cost ends up being the dominant factor (approximately 97% of the total computation). However, we obtain an amortized performance of approximately 1,200 permutations per second.

B.7 Performance Estimate for the Unbiased Construction

For the HSS-based construction from Section 4, we additionally give some performance estimates when using an HSS scheme based on Paillier, and an HSS-friendly (weak) PRF.

Instantiating the HSS. We consider the two-party HSS construction from [2], instantiated with ElGamal encryption in the Paillier group $\mathbb{Z}_{N^2}$ for a 2048-bit RSA modulus N . The secret key is a short ($\geq 2\lambda$ -bit) integer exponent. Using a recent result of [5], this yields an HSS whose security reduces to DCR together with the hardness of the short-exponent discrete logarithm assumption. With this construction, a single ElGamal ciphertext has size $4 \log N$ bits, and each input to the HSS requires 2 ciphertexts, for a total of $8 \log N$ bits per input. We assume a setup procedure that has distributed shares of the secret key between the two parties. After this setup, the HSS supports homomorphic evaluation of arbitrary, *restricted-multiplication* circuits on the inputs — namely, arithmetic circuits where every multiplication gate is of fan-in two and includes at least one circuit input wire as an input to the gate.

Instantiating the HSS-friendly PRF. We instantiate the pseudorandom function $(F.\text{KeyGen}, F.\text{Eval})$ using a Goldreich-style weak-PRF [7] of low locality $\ell = O(\log \lambda)$, given by:

$$F_s(x) = P(s_{i_1}, \dots, s_{i_\ell})$$

Here, the indices i_j are randomly determined based on the input x (in practice, using, for instance, a hash function modeled as a random oracle), while the predicate P is a fixed, locality- ℓ function of the PRF key s .

To instantiate P , one way to ensure a relatively high stretch is to use the XOR-majority predicate $P(x) = x_1 \oplus \dots \oplus x_{\ell_1} \oplus \text{MAJ}(x_{\ell_1+1}, \dots, x_{\ell_1+\ell_2})$, where $\ell = \ell_1 + \ell_2$. Although this predicate was recently shown to be susceptible to bit-fixing correlation attacks [50], this can be mitigated by increasing the seed size until the attacks are no longer feasible. Concretely, we consider two instantiations:

- $\ell_1 = 32, \ell_2 = 64$ from [41], with a seed of length 512 (increased from 256 in [41]) and up to 2^{40} pseudorandom output bits.
- $\ell_1 = 5, \ell_2 = 15$ from [38], with a seed length of 2048 and up to 2^{33} output bits.

Both parameter sets should resist the attacks from [50].

To evaluate the PRF as a restricted-multiplication circuit, first, note that since the majority function on k inputs is symmetric, it can be expressed as a degree- k polynomial over any finite field $\mathbf{F}_p$ for $p > k$, by summing the input to obtain a value in $\{0, \dots, k\}$ and interpolating. Therefore, majority can be computed using $k - 1$ multiplications and $O(k)$ additions/scalar multiplications, where in our case, $k = \ell_2$. The XOR part of the PRF can be computed with a chain of $\ell_1 + 1$ multiplication gates, giving a total of $\ell = \ell_1 + \ell_2$ multiplications.

Efficiency. The storage and communication cost for each party consists of $8 \log N$ bits per input to store the HSS ciphertexts. With the first parameter set, we get a total seed size of ≈ 1050 kB per party, and with the second, ≈ 4200 kB. The dominant computational cost of evaluating the PRF inside HSS is the multiplication gates. Each multiplication consists of a distributed discrete logarithm procedure, whose dominant cost can be approximated by 1 exponentiation in $\mathbb{Z}_{N^2}$ with an $\approx 3\lambda$ -bit exponent. Based on estimates for exponentiations from a modern laptop with an Intel i7-1365U CPU, this should cost around 1ms. This gives a total evaluation cost of around 64ms per permutation output, for the first parameter set, or only 15ms for the second. This seems to be a reasonable latency for our applications to single-secret leader election and anonymous broadcast.